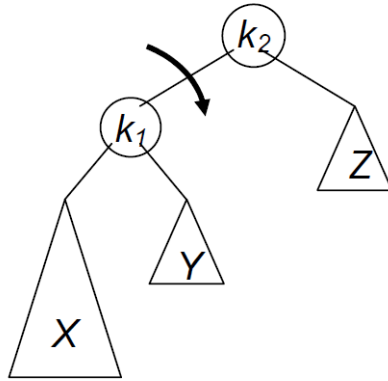


## ڈیٹا سٹرکچر سی ایس 301

### لیکچر نمبر 23

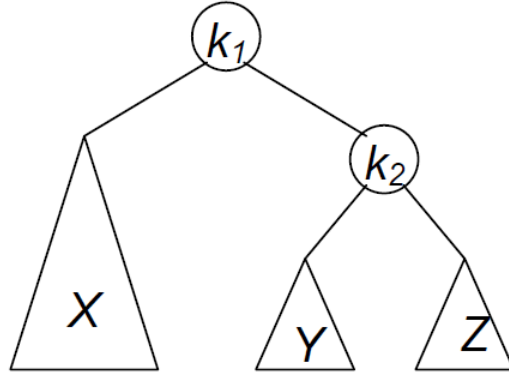
#### Single Right Rotation

جس نوڈ پر رائٹ روٹیشن کرنی ہوتی ہے اس پر یہ سائمن لاگو ہوتا ہے۔ اور اس کو K2 کا نام دیا جاتا ہے۔ سنگل رائٹ روٹیشن کرنے کے بعد فنکشن پوائنٹر کو ریٹرن دیتا ہے۔ ٹری Re-Organized ہو جائے گی اور اس کا روٹ change ہو جائے گا۔ ہم فنکشن میں سب سے پہلے K2 کو چیک کریں گے اگر K2 نل ہو تو فنکشن نہیں ہوگا تو پراسیس سٹارٹ ہو Exit ہو جائے گا اور Null ریٹرن کر دے گا، اگر K2 نل جائے گا۔

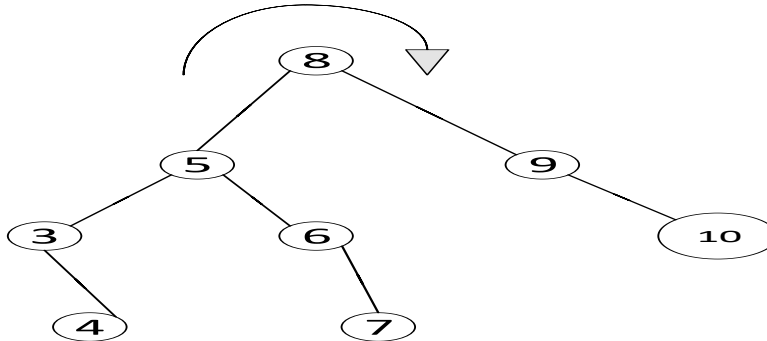


اب اوپر دیئے گئے مثال میں K2 روٹ ہے اس ٹری کا اور رائٹ روٹیشن کا سائمن K2 کے اوپر ہے، اس کا مطلب ہے کہ رائٹ روٹیشن K2 کے اوپر ہوگی اور اس ٹری کا

روٹ نوڈ change ہو جائے گا۔ رائٹ روٹیشن کے بعد K1 اس ٹری کا روٹ نوڈ بن جائے گا۔ روٹیشن کے باڈری کچھ یوں ہوگی،

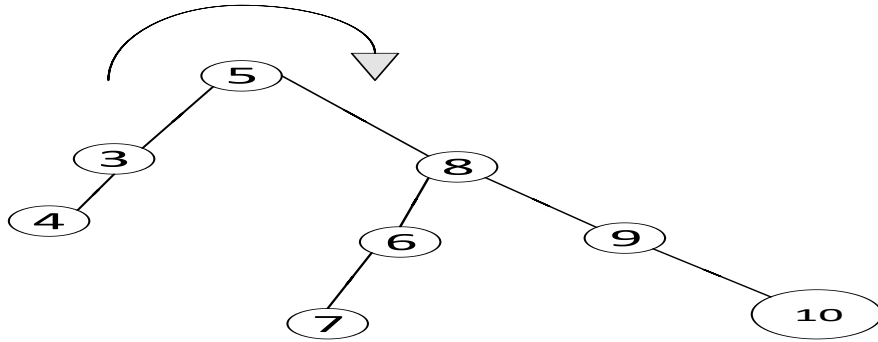


K1 روٹ نوڈ بن گیا اور X اپنی جگہ پر رہے گا جب کہ Y اپنی جگہ تبدیل کر کے رائٹ روٹیشن کر کے K2 کے رائٹ میں چلا جائے گا جیسے کے اوپر کی ٹری میں نظر آ رہا ہے۔ رائٹ روٹیشن کو ایک اور مثال سے سمجھنے کی کوشش کرتے ہیں۔



اس ٹری میں دیکھا جاسکتا ہے کہ 8 نمبر نوڈ اس ٹری کا روٹ نوڈ ہے اور اس کے لفٹ میں 5 ہے اور 5 کے رائٹ میں 6 ہے اور 5 کے لفٹ میں 3 جو، پھر اسی طرح سے نوڈ 6 کے رائٹ میں نوڈ 7 ہے اور نوڈ 3 کے رائٹ میں نوڈ 4 ہے۔ جبکہ روٹ کے رائٹ میں سب ٹری ہے جس میں نوڈ 9 اور نوڈ 9 کے رائٹ میں نوڈ 10 ہے۔ اب جب اس

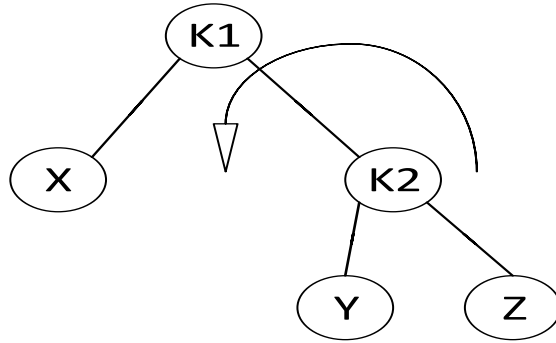
ٹری پر رائٹ روٹیشن apply ہوگی تو اس ٹری میں کیا changing آئی گی؟ رائٹ روٹیشن apply کرنے کے نوڈ 5 اس ٹری کا روٹ نوڈ بن جائے گا اور نوڈ 8 نوڈ 5 کے رائٹ سائیڈ پر چلا جائے گا اور نوڈ 8 کے نیچے جو سب ٹری ہے وہ ویسے ہی رہی گی۔ جو سب ٹری نوڈ 5 کے لفٹ بھی ہے وہ بھی ویسے کے ویسے ہی رہی گی، مگر جو نوڈ 5 کی رائٹ سب ٹری تھی اسکی جگہ تبدیل ہوگی۔ رائٹ روٹیشن میں جو نوڈ روٹ بنتا ہے اسکی لفٹ سب ٹری کو ویسے کا ویسا ہی رکھنا ہے جب کہ اسکی رائٹ ٹری کی جگہ تبدیل ہوگی۔ رائٹ روٹیشن کے بعد اوپر دی گئی ٹری کچھ ایسے ہوگی۔



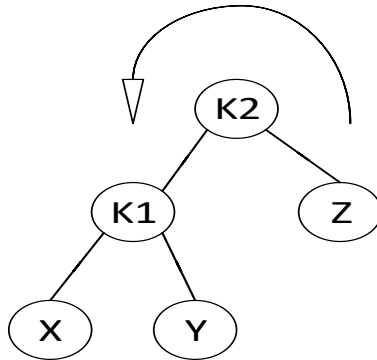
نوڈ 6 اور نوڈ 7 اب نوڈ 8 کے لفٹ سائیڈ میں آگئے ہیں۔ اس روٹیشن کو ہم سنگل رائٹ روٹیشن کہتے ہیں۔ یاد رکھیں اگر ٹری خالی ہوگی تو ریٹرن ویلیو 1- آئی گی۔

## Single Left Rotation

لفٹ روٹیشن بھی رائٹ روٹیشن جیسی ہے مگر اس میں ہم رائٹ کی بجائے لفٹ سائیڈ روٹیشن کرتے ہیں۔ اس کو سمجھنے کیلئے ایک دو مثالیں دیکھتے ہیں تاکہ آسانی سے سمجھ میں آسکے۔



اب اس ٹری پر ہم لفٹ روٹیشن کو apply کرے گے، اس وقت اس ٹری کا روٹ نوڈ K1 ہے مگر لفٹ روٹیشن کے بعد روٹ بھی change ہوگا اور Y بھی اپنی جگہ سے شفٹ ہو جائے گی۔ لفٹ روٹیشن کے بعد ہماری ٹری کچھ یوں نظر آئی گی۔

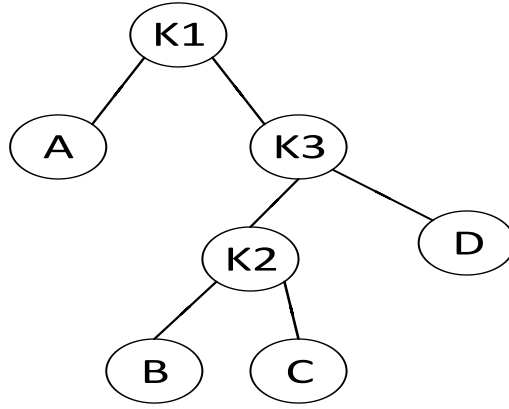


لفٹ روٹیشن apply ہونے کے بعد K2 اس ٹری کا روٹ نوڈ بن گیا جب کے K1 اب روٹ نوڈ K2 کا لفٹ سب ٹری بن گیا ہے۔ مگر نوڈ Z نے اپنی جگہ change نہیں کی ایسا کیوں؟ Z کی جگہ اس لئے change نہیں ہوئی ہے کیونکہ ہم نے لفٹ روٹیشن کو apply کیا ہے اگر ہم رائٹ روٹیشن کو apply کرتے پھر Z کی جگہ change ہوتی

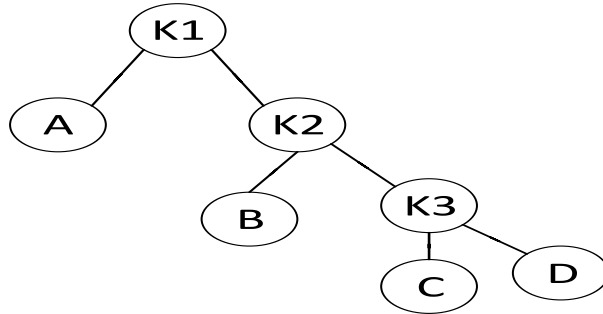
جبکہ ۲ کی جگہ برقرار رہتی۔ لفٹ روٹیشن میں جب نیا روٹ بن جاتا ہے تو نئے روٹ کے لفٹ سب ٹری change ہوئے روٹ کے رائٹ سائیڈ پر چلی جاتی ہے جبکہ رائٹ روٹیشن میں الٹ ہوتا ہے۔

Double Right-Left Rotation:

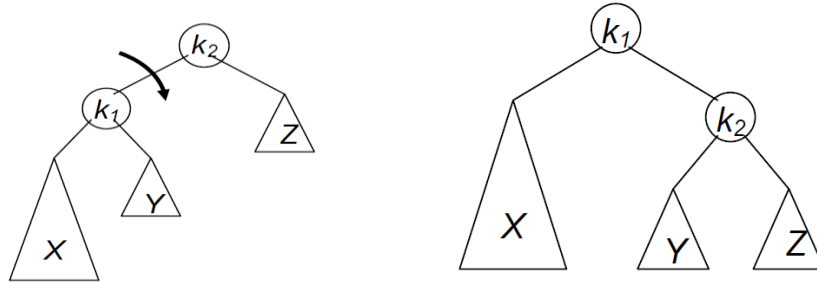
ڈبل رائٹ لفٹ روٹیشن میں سب سے پہلے رائٹ روٹیشن apply ہوتی ہے پھر لفٹ روٹیشن apply ہوتی ہے۔



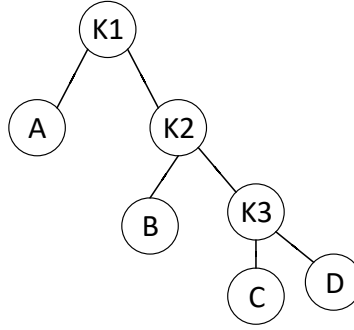
اب اس ٹری پر ہم ڈبل رائٹ لفٹ روٹیشن apply کر کے دیکھیں گے کہ ہماری نئی ٹری کس shape کی بنتی ہے۔



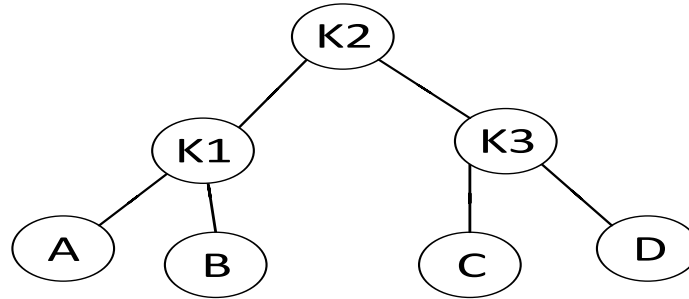
ڈبل رائٹ لفٹ روٹیشن apply ہونے کے بعد ہماری ٹری کی shape اوپر دیکھی جا سکتی ہے۔ اب سوال یہ پیدا ہوتا ہے کہ D کی جگہ change کیوں نہیں ہوئی اور C کی جگہ تبدیل کیوں ہوئی ہے؟ جیسے کہ ڈبل رائٹ لفٹ روٹیشن میں شروع میں بتا چکا ہوں کہ ڈبل رائٹ لفٹ روٹیشن میں سب سے پہلے رائٹ روٹیشن apply ہوتی ہے اور پھر لفٹ روٹیشن apply ہوتی ہے، یہی وجہ ہے کہ D کی جگہ change نہیں ہوئی اور C کی جگہ change ہو گئی۔ K3 کی جگہ change ہوتے ہی رائٹ روٹیشن apply ہو گئی اب رائٹ کے بعد لفٹ روٹیشن apply کرنے کیلئے C کی جگہ change ہو گئی۔ ابھی تک ہم نے سنگل رائٹ روٹیشن apply کی ہے اب ہم سنگل لفٹ روٹیشن apply کریں گے اور پھر جو اوٹ پٹ میں ہمارے پاس ٹری آئی گی اسکو ڈبل رائٹ لفٹ روٹیشن کہیں گے۔ ایک بات یاد رکھیں کہ رائٹ روٹیشن میں جو نوڈ rotate ہو کر اپنی جگہ تبدیل کر دیتا ہے اس کے رائٹ میں جو بھی نوڈ ہوگا صرف اسکی جگہ change ہوگی جبکہ لفٹ نوڈ اپنی جگہ پر قائم رہے گا۔ اس کو مزید سمجھنے کیلئے نیچے دیئے گئے رائٹ روٹیشن کی مثال کو غور سے دیکھیں۔



اس رائٹ روٹیشن میں دیکھا جاسکتا ہے کہ جب K1 روٹ بن گیا ہے تو X اپنی جگہ پر رہا جبکہ Y جو کہ K1 کے رائٹ پر تھا وہ رائٹ روٹیشن ہونے کے بعد K2 کے لفٹ سب ٹری بن گیا ہے۔ اب لفٹ روٹیشن کرنے کے بعد جو ٹری کی shape بنے گی اسکو ہم ڈبل رائٹ لفٹ روٹیشن کہیں گے۔

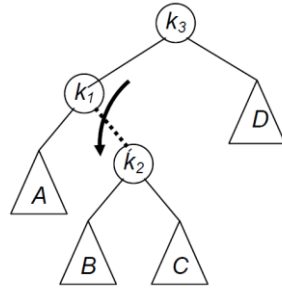


اب اگر اس ٹری کو دیکھیں تو اس کا بیلنس فیکٹر بھی سہی نہیں ہے اور جس بھی ٹری کا بیلنس فیکٹر -1، 0، 1، میں سے نہیں ہوگا وہ AVL ٹری نہیں کہلاتی۔ اس لئے اس کو AVL ٹری بنانے کیلئے ابھی لفٹ سائیڈ آپریشن کرنا پڑے گا۔ ڈبل لفٹ روٹیشن apply کرنے کے بعد ہمارے پاس جو ٹری کی shape بن کر آئی گی وہ نیچے دے گئی ہے۔

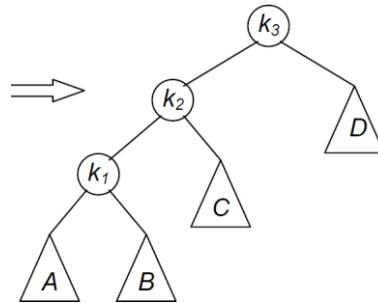


## Double Left-Right Rotation:

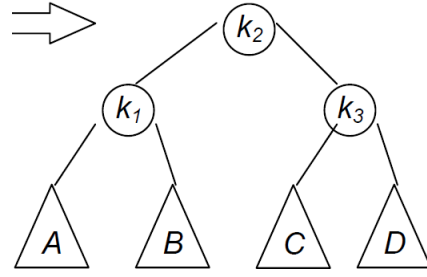
ڈبل لفٹ رائٹ روٹیشن میں سب سے پہلے لفٹ روٹیشن apply ہوتی ہے اور پھر رائٹ روٹیشن apply ہوتی ہے۔



اس ٹری پر پہلے لفٹ روٹیشن کو apply کرے گے اور پھر رائٹ روٹیشن کو apply کر کہ ٹری کی فائنل اوٹ پٹ نظر آ جائے گی۔



لفٹ روٹیشن apply ہونے کے بعد دیکھا جاسکتا ہے کہ K1 کی جگہ change ہو گئی ہے۔ کیونکہ یہ لفٹ سائیڈ روٹیشن apply ہوئی ہے اس لئے K2 کے رائٹ میں جو C تھا وہ اپنی ہی جگہ پر قائم ہے جبکہ B شفٹ ہو کر نوڈ A کے رائٹ پر چلا گیا ہے، نوڈ K1 کے رائٹ میں جانے کی وجہ یہ ہے کہ نوڈ K1 کے لفٹ میں پہلے سے ہی نوڈ A موجود ہے۔ اب لفٹ روٹیشن کے بعد رائٹ روٹیشن apply کریں گے۔



رائٹ روٹیشن apply ہونے کے بعد ہمارے پاس ٹری کی فائنل اوٹ پٹ آگئی ہے۔  
جو ایک بیلنس ٹری ہے۔

## Deletion in AVL Tree

Deletion اور Insertion کے روٹین سٹینڈرڈ لائبریری میں پائے جاتے ہیں۔ ہم ان روٹین کو اپنے پروگرام میں استعمال کر سکتے ہیں۔ ہمیں ان روٹین کو لکھنے کی ضرورت نہیں ہے۔ ٹری میں کوئی بھی نوڈ add کرنے کیلئے ہم سنگل یا پھر ڈبل روٹیشن کا استعمال کرتے ہیں۔ ہم ٹری کے اس نوڈ پر روٹیشن کرتے ہیں جو AVL ٹری کی کنڈیشن کو پورا نہیں کرتی مطلب جس نوڈ کا بیلنس فیکٹر منفی ایک۔ زیر و یا پلس ایک نہ ہو اس پر روٹیشن کرتے ہیں۔ Deletion کے بدترین کیس میں ہم  $\log_2 N$

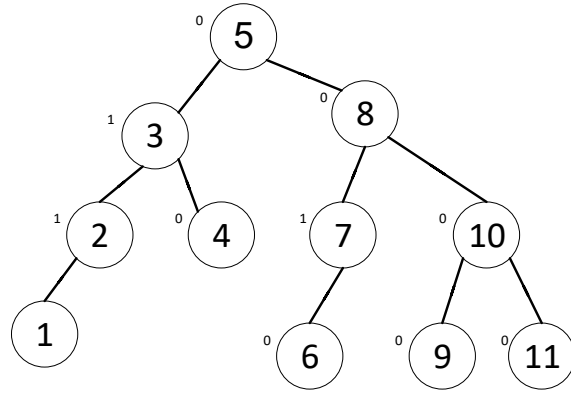
فارمولا کا استعمال کرتے ہیں۔  $\log_2 N$  ٹری کے لیول ہوتے ہوئے  $N$  نمبر اف نوڈز ہوتے ہیں۔ ٹری میں سے کوئی بھی نوڈ ڈیلیٹ کرنے کیلئے کچھ رولز ہوتے ہیں۔

$R_0 \rightarrow LL$   
 $R_1 \rightarrow LL$   
 $R_{-1} \rightarrow LR$   
 $L_0 \rightarrow RR$   
 $L_1 \rightarrow RL$   
 $L_{-1} \rightarrow RR$   
Check B.F. (prev)  
with siblings.

اوپر دیئے گئے رولز کو مد نظر رکھ ہم ٹری سے نوڈ ڈیلیٹ کر سکتے ہیں۔

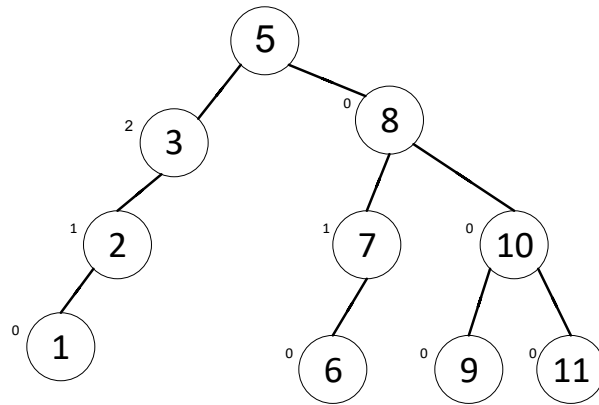
ہم ٹری سے لیف نوڈ کو بہت آسانی سے ڈیلیٹ کر سکتے ہیں اور اس کیلئے ہم ٹری کو پھر سے بیلنس نہیں کرنا پڑتا کیونکہ لیف نوڈ سے ٹری کا بیلنس فیکٹر change نہیں ہوتا۔

اگر ڈیلیٹ ہونے والے نوڈ کے رائٹ یا لفٹ پر کوئی نوڈ ہے تو پھر ہمیں اوپر دیئے گئے رولز کو مد نظر کر ٹری سے نوڈ کو ڈیلیٹ کرنا ہوگا۔

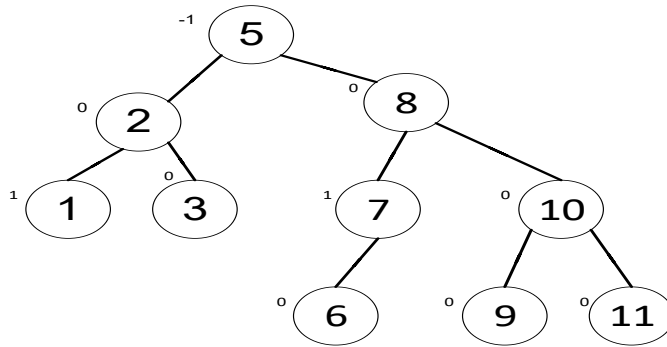


اس ٹری سے اب ہم کچھ نوڈ ڈیلیٹ کریں گے اور پھر دیکھیں گے کہ کونسا Rule لاگو ہوتا ہے۔ اب اگر سے نوڈ 4 کو ڈیلیٹ کرنا ہو تو یہ بہت آسان اور simple کیس ہے اس میں ٹری کا بیلنس فیکٹر change نہیں ہوگا۔ ابھی تک یہ ٹری بیلنس ٹری ہے۔

اب ہم اس ٹری سے نوڈ 4 کو ڈیلیٹ کریں گے اور پھر دیکھیں گے کہ اس ٹری کا بیلنس فیکٹر change ہوتا ہے یا نہیں اگر بیلنس فیکٹر change ہو تو یہ بھی دیکھنا ہوگا کہ کون سا rule لاگو ہوگا تاکہ ٹری کو پھر سے ایک بیلنس AVL ٹری بنایا جاسکے۔



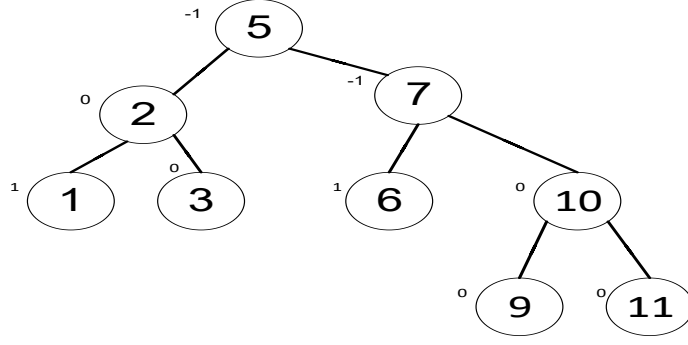
نوڈ 4 ڈیلیٹ ہونے کے بعد نوڈ 3 کا بیلنس فیکٹر change ہو کر 2 ہو گیا ہے جو AVL ٹری کے رولز کے مطابق نہیں ہے اب یہ ٹری ایک بیلنس AVL ٹری نہیں رہی اس کو بیلنس ٹری بنانے کیلئے اب ہمیں دیکھنا ہوگا کہ deletion کہا ہوئی ہے اور جہاں deletion ہوئی ہے اس کے سبلنگ کا بیلنس فیکٹر کی تھا۔ اوپر ہم دیکھ سکتے ہیں کہ جہاں بیلنس فیکٹر change ہوا ہے وہ نوڈ 3 ہے اور نوڈ 3 کے رائٹ سائیڈ میں deletion ہوئی ہے، اور نوڈ 3 کے سبلنگ جو کہ نوڈ 8 بنتا ہے اس کا بیلنس فیکٹر 0 ہے، اب ہم rule جو apply کرے گے وہ R0 ہوگا۔ اور R0 پر LL یعنی لفٹ ٹو لفٹ روٹیشن apply ہوتی ہے جیسے کہ اوپر رولز والے پیکچر میں دیکھا جاسکتا ہے۔ اب لفٹ ٹو لفٹ روٹیشن apply کرنے کے بعد ہم اس ٹری کو بیلنس ٹری کر کے دیکھتے ہیں۔



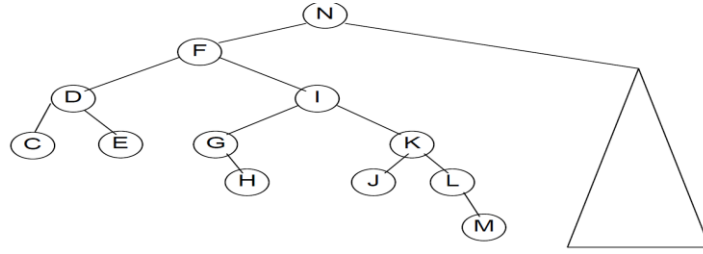
لفٹ ٹو لفٹ روٹیشن apply ہونے کے بعد ہماری ٹری ایک بار پھر سے ایک بیلنس AVL ٹری بن چکی ہے۔ آئیں کچھ اور نوڈ ڈیلیٹ کر کے اس کو سمجھنے کی کوشش کرتے ہیں۔

## Deleting node 8

اس سے اوپر والی مثال میں ہم نے ایک لیف نوڈ کو ڈیلیٹ کیا تھا اس لئے اس لئے آسانی سے فائنڈ اوٹ کر لئے تھا کہ کون سا rule لاگو ہوگا مگر اب نوڈ 8 کو ڈیلیٹ کرنے کیلئے ہم نے دیکھنا ہے کہ یہ لیف نوڈ ہے یا نہیں۔ ہمارے کیس میں نوڈ 8 لیف نوڈ نہیں ہے کیونکہ اس کے رائٹ اینڈ لفٹ پر سب ٹری موجود ہیں۔ جب کسی بھی نوڈ کے رائٹ اینڈ لفٹ پر سب ٹری یعنی نوڈز موجود ہوں گے تو پھر ہمیں کچھ پر اپرٹی کو چیک کرنا ہوگا اور rule کو apply کرنا ہوگا۔ اب نوڈ 8 کے رائٹ لفٹ نوڈ ہونے کی وجہ سے پہلے ہمیں inorder predecessor value کو replace کرنا ہوگا۔ اب اس rule کے مطابق ہمیں سب سے پہلے inorder کرنا ہوگا اس ٹری کو۔ تو یہ ٹری کچھ یوں بنے گی۔  
1، 2، 3، 5، 6، 7، 8، 9، 10، 11 اب ہم ان اپنی ٹری کو inorder کر لیا ہے۔ اب اگر نوڈ 8 کو ڈیلیٹ کرنا ہے تو نوڈ 8 کا inorder نوڈ 7 بنتا ہے تو ہم نوڈ 8 کو ڈیلیٹ کرنے کیلئے نوڈ 7 کو اس جگہ پر پلےس کر دیں گے۔

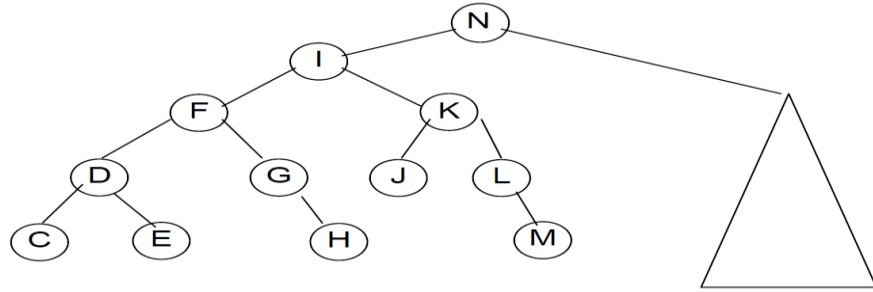


نوڈ 8 ڈیلیٹ ہونے کے بعد ٹری کا بیلنس فیکٹر change نہیں ہوا ہے اگر ٹری کا بیلنس فیکٹر change ہو جاتا تو ہم پھر سے دیکھتے deletion کہا ہوئی ہے اور جہاں deletion ہوئی ہے اس کے سبلنگ کا بیلنس فیکٹر کیا ہے۔



ہمارے ہینڈ آؤٹ میں جو ٹری دے گئی وہ اوپر دے گئی ہے۔ اب اس ٹری کی مطابق نوڈ F کا بیلنس فیکٹر change ہو کر 2- ہو گیا ہے جس کی وجہ سے اب یہ ٹری AVL ٹری نہیں رہی۔ اس کو ایک بیلنس ٹری بنانے کیلئے ہمیں روٹیشن apply کرنی ہوگی، اس کیس میں ہمیں یہ دیکھنا ہے کہ بیلنس فیکٹر کہاں change ہوا ہے، ہمارے کیس میں بیلنس فیکٹر نوڈ F کا change ہوا ہے، اس کیس میں بیلنس change ہونے سے پہلے رائٹ روٹیشن apply ہوئی تھی جس کی وجہ سے یہ بیلنس فیکٹر change ہوا ہے۔

اب اس کو بیلنس ٹری بنانے کیلئے ہم یہ دیکھیں گے کہ F کی ٹری سب سے چوٹی ہے تو ہم لفٹ روٹیشن apply کرے گے تاکہ ہماری ٹری بیلنس ٹری بن جائے۔ لفٹ apply کرنے کے بعد ہماری ٹری بیلنس ٹری بن چکی ہے اور جو نیچے دے گئی ہے۔



لفٹ روٹیشن apply ہونے کے بعد ہم دیکھ سکتے ہیں کچھ نوڈز کی جگہ change ہو گئی ہے۔ نوڈ F کے نیچے جو کچھ تھا وہ تو ویسے کا ویسا آگیا، نوڈ A اوپر چلا گیا اور نوڈ A کے لفٹ پر جو نوڈز G اور H تھے وہ ابھی شفٹ ہو کر نوڈ F کے رائٹ سب ٹری بن گئے ہیں اور نوڈ A کے رائٹ میں جو نوڈز تھے وہ ویسے کے ویسے رہیں گے۔

بائنری سرچ ٹری سے نوڈ ڈیلیٹ کرنے کے تین کیس ہوتے ہیں۔

کیس نمبر 1: لیف نوڈ کو بائنری سرچ ٹری دے ڈیلیٹ کرنا سب سے

simple کیس ہوتا ہے۔ جس نوڈ کو ڈیلیٹ کرنا ہوتا ہے پوائنٹر کو پائنٹ کر

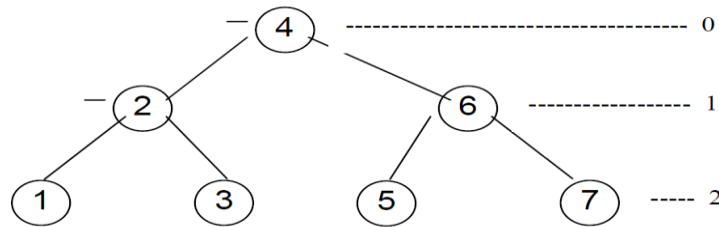
کے نل ریٹرن کر دیتے ہیں اور وہ نوڈ ڈیلیٹ ہو جاتا ہے۔

کیس نمبر 2: جو نوڈ ڈیلیٹ کرنا ہے اسکا کوئی بھی سب ٹری ہو چاہے وہ رائٹ ہو یا لفٹ سب ٹری ہو۔ اس کیس میں ہم inorder successor کو فائنڈ کر کے نوڈ کو bypass کروادیتے ہیں۔

کیس نمبر 3: جو نوڈ ڈیلیٹ کرنا ہے اسکا رائٹ اور لفٹ دونوں نوڈ موجود ہو، یہ کیس سب سے مشکل کیس ہے۔ اس کیس میں بھی ہم inorder successor کو فائنڈ کرتے ہیں۔ رائٹ سب ٹری کا لفٹ موست یعنی آخری نوڈ inorder successor ہوگا۔

## لیکچر نمبر 24

ایسی ٹری جس کا بیلنس فیکٹر 0 ہے perfectly balanced tree کہلاتی ہے۔



اس ٹری کو ہم کہہ سکتے ہیں کہ یہ پرفیکٹ بیلنس ٹری ہے۔

ٹری کا جو بھی سائز ہو سرچ traversing کے بعد پر فارم ہوتی ہے اور  $\log_2 N$  لیول تک زیادہ سے زیادہ جاتی ہے۔

Whatever is the size of the tree, the search is performed after traversing up to  $\log_2 n$  levels maximum.

AVL ٹری میں بدترین کیس یہ ہے کہ ہمیں  $1.44 \log_2 N$  لو لس تک سرچ کرنا پڑتا ہے۔

While searching in an AVL tree, in the worst case scenario we have to search  $1.44 \log_2 n$  levels

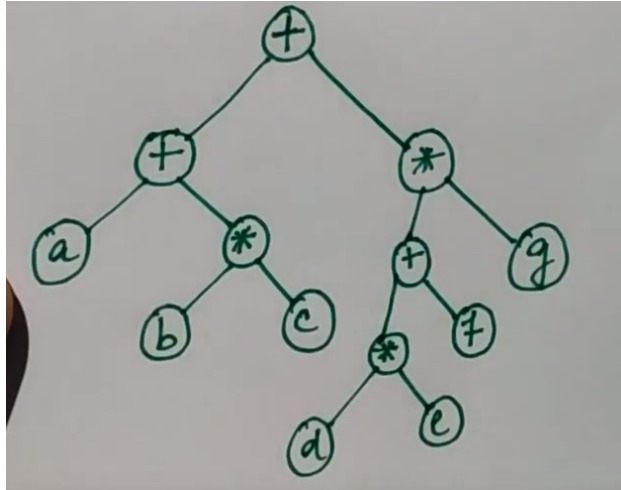
### Expression Trees:

$$(a+b*c) + ((d*e+f) *g)$$

اوپر ایک ایکسپریشن دیا گیا ہے اور اس سے ہم نے ابھی ایک ٹری بنانی ہے۔ یہ بائری سرچ ٹری جیسے کے آپ سب کو معلوم ہوگا کہ جمع، ضرب، تقسیم اور منفی بائری آپریٹرز ہیں۔ اب اس سے ہم کیسے پتا چلے گا کہ اس کا روٹ نوڈ کون سا ہوگا اور کون سے آپریٹرز رائٹ سب ٹری بنے گے اور کون سے لفٹ سب ٹری۔ اس ٹری میں کیونکہ parathence کا استعمال ہوا ہے

اس لئے ہم آسانی سے اس ٹری کاروٹ فائنڈ کر سکتے ہیں۔ اس ٹری کاروٹ  
 $(a+b*c) + ((d*e+f)*g)$  نوڈ + سائن بنے گا کیونکہ یہ بریکٹ سے  
 باہر ہے اور + آپریٹر کی priority بھی کم ہوتی ضرب اور تقسیم کے مقابلے  
 میں۔ آئیں دیکھتے ہیں کہ اس ایکسپریشن سے کیسے ہم ٹری بنا سکتے ہیں۔

$$(a+b*c) + ((d*e+f)*g)$$

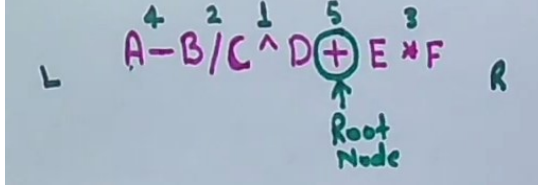


تو ہماری ایکسپریشن بائری ٹری کچھ یوں بنی گی۔

$$A-B/C^D+E*F$$

اگر بائری ٹری بنا بریکٹ کی دے جائے تو پھر روٹ نوڈ کو دیکھنا ہوگا کہ کون  
 سب سے کم priority والا ہے۔ جیسے کہ اوپر ایک ایکسپریشن بائری ٹری  
 دی گئی ہے اب اس میں اگر ہم دیکھیں تو ہائی Priority ^ سائن کی ہے،  
 دوسرے نمبر پر / سائن ہے جس کی Priority ہائی ہے۔ + کی Priority

کیونکہ ایک جیسی ہے اس لئے اس کو ہم لفٹ ٹورائٹ کاؤنٹ کرکٹ نمبر دے گے۔



تو یہ ایکسپریشن ٹری کچھ یوں بنے گی۔

نمبر 1: ^

نمبر 2: /

نمبر 3: \*

نمبر 4: -

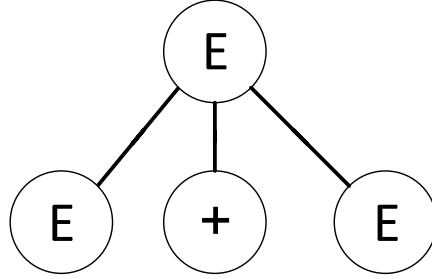
نمبر 5: +

اب اس ٹری کو آپ لوگ خود حال کرنے کی کوشش کریں اور اگر سمجھ نہ آئے تو آپ مجھ سے رابطہ کر سکتے ہیں۔

## Parse Tree

Parse ٹری ٹرمنل اور نان ٹرمنل کے درمیان میں بنتی ہے، ایک سنٹیکس ٹری بھی ہوتی ہے جو کہ Operand اور آپریٹر کے درمیان میں بنتی ہے۔

Parse میں ہمیں CFG رولز کی ضرورت پڑتی ہے۔ سنٹیکس ٹری میں ہمیں CFG رولز کی ضرورت نہیں ہوتی ہے۔ Parse ٹری میں ہم آپریٹر کو ہمیشہ انٹرئل لیف نوڈ میں Move کرتے ہیں۔ اب انٹرئل لیف کیا ہوتا ہے۔ اس کی ایک مثال دیکھتے ہیں تاکہ آسانی سے سمجھ میں آسکے۔



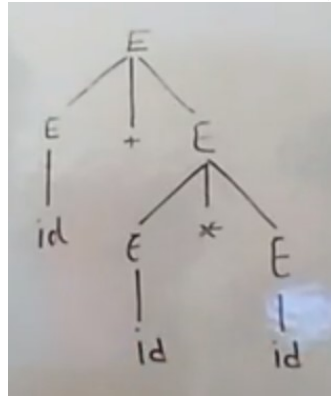
اس میں + کا سائن لیف نوڈ نہیں بلکہ سینٹر میں آ رہا ہے اس لئے اس کو انٹرئل لیف کہتے ہیں۔ CFG گرامر کارولز چارٹ

$E \rightarrow E + E$

$E \rightarrow E * E$

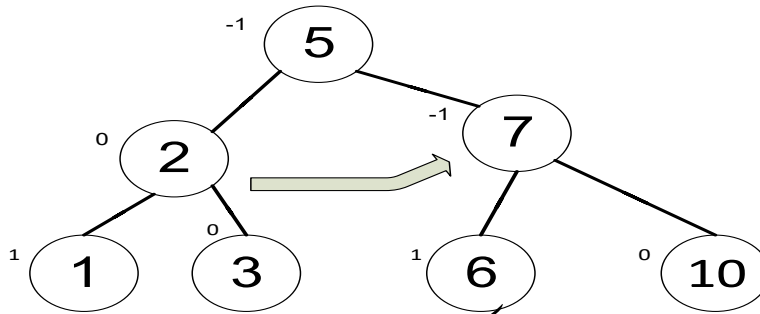
$E \rightarrow ID$

Parse ٹری کی مثال نیچے دی گئی ہے



The parse trees are used in query processing.

کمپائلر کا وہ حصہ جو اس قسم کے ایکسپریشن ٹری کو ہینڈل کرتا ہے اس کو Optimizer کہتے ہیں۔ اگر کوئی ایسی ٹری ہو جس میں ایک نوڈ کو ایکسیس کرنے کیلئے ایک سے زیادہ way ہو تو ٹری نہیں بلکہ گراف کہلاتا ہے۔  
مثال کے طور پر



نوڈ 2 نوڈ 7 کو دو راستوں سے ایکسیس کر رہا ہے تو اس قسم کی ٹری جب بھی نظر آئے یہ غلط ہے بلکہ یہ گراف ہے ٹری نہیں ہے۔

Optimizer ایکسپریشن ٹریز کو استعمال کر کے انکو گراف میں کنورٹ کرتا ہے۔ لیف نوڈز وہ نوڈز ہوتے ہیں جنکے رائٹ لفٹ ٹریز Null ہو مطلب کوئی بھی نوڈ نہ ہو۔ لیف نوڈز Inner نوڈز سے کنیکٹ ہوتے ہیں۔

## Lecture No. 25

a b + c d e + \* \*

اوپر ایکسپریشن دی گئی ہے اب اس سے آپ خود پریکٹس کے طور پر حل کرنے کی کوشش کریں۔

Huffman Encoding:

ڈیٹا کو destination تک تیزی سے بھیجنے کیلئے ہم یا تو لنک کی سپیڈ بڑھا دیتے ہیں یا پھر data کو compress کرتے ہیں تاکہ data تیزی سے اپنے destination تک پہنچ سکے۔ عام طور پر ہم data کو compress کرتے ہیں۔ کمپریشن کی مدد سے ہم 70% data کو compress کر سکتے ہیں۔ مثال کے طور پر اگر ہماری فائل ایک MB کی ہے تو Compress ہونے کے بعد وہ 300KB کی ہو جاتی ہے۔ کمپریشن Methods ٹیکسٹ، وائس، امیجز اور دوسرے data ٹائپ کیلئے استعمال کی جاتی ہے۔ ہم انٹرنیٹ کو استعمال کرنے کیلئے موڈیم کو انٹرنیٹ سے کنکٹ کرتے ہیں اور موڈیم لائیو کمپریشن کرتا ہے۔ Huffman کوڈ کمپریشن کے سٹینڈرڈ ٹیکسٹ documents کا میٹھڈ ہے۔

Huffman code is method for the compression of standard text documents.

Huffman کوڈ بائریٹری کا استعمال کرتا ہے تاکہ میسج میں جو original لیٹر استعمال ہوئے ہیں انکے لئے کوڈ بنا سکے۔ Huffman کوڈ JPEG امیج کے کمپریشن کا بھی پارٹ

ہے۔ Huffman کوڈ کو 1952 میں David Huffman نے متعارف کروایا۔  
انگلش Alphabets کیلئے ہم ASCII کوڈ کا استعمال کرتے ہیں، ASCII کوڈ عام  
طور پر آٹھ بٹز پر مشتمل ہوتا ہے۔ ہم ان آٹھ بٹز سے 256 قسم کے مختلف Patterns  
بنا سکتے ہیں۔ انگلش میں ہمارے پاس 26 لوئر کیس اور 26 ہیپر کیس  
الفبیٹس Alphabets ہیں۔ ASCII ٹیبل میں کچھ گراف کے کیریکٹر بھی ہوتے ہیں۔

"Traversing threaded binary trees"

اوپر دیئے گئے Sentence میں ہم نے Huffman کوڈ بنا ہے۔ اس کیلئے ضروری ہے  
کہ ہم سب سے پہلے کاؤنٹ کر لیں کہ کتنے کیریکٹر اور Spaces ہیں۔ ہمارے  
پاس 29 انگلش کے Alphabets ہیں اور 3 بار ان کے درمیان سپیس آئی ہے۔ اور  
آخر میں ایک نیا لائن ہوتی ہے۔ تو ٹوٹل ہمارے پاس 33 کیریکٹر بنتے ہیں۔ ASCII جو  
سپیس کیلئے ہے وہ 32 ہے اور new لائن کیلئے کیریکٹر کوڈ 10 ہے۔

# ASCII Table

Dec = Decimal Value

Char = Character

'5' has the int value 53

if we write '5'-'0' it evaluates to 53-48, or the int 5

if we write char c = 'B'+32; then c stores 'b'

| Dec | Char                        | Dec | Char  | Dec | Char | Dec | Char |
|-----|-----------------------------|-----|-------|-----|------|-----|------|
| 0   | NUL (null)                  | 32  | SPACE | 64  | @    | 96  | `    |
| 1   | SOH (start of heading)      | 33  | !     | 65  | A    | 97  | a    |
| 2   | STX (start of text)         | 34  | "     | 66  | B    | 98  | b    |
| 3   | ETX (end of text)           | 35  | #     | 67  | C    | 99  | c    |
| 4   | EOT (end of transmission)   | 36  | \$    | 68  | D    | 100 | d    |
| 5   | ENQ (enquiry)               | 37  | %     | 69  | E    | 101 | e    |
| 6   | ACK (acknowledge)           | 38  | &     | 70  | F    | 102 | f    |
| 7   | BEL (bell)                  | 39  | '     | 71  | G    | 103 | g    |
| 8   | BS (backspace)              | 40  | (     | 72  | H    | 104 | h    |
| 9   | TAB (horizontal tab)        | 41  | )     | 73  | I    | 105 | i    |
| 10  | LF (NL line feed, new line) | 42  | *     | 74  | J    | 106 | j    |
| 11  | VT (vertical tab)           | 43  | +     | 75  | K    | 107 | k    |
| 12  | FF (NP form feed, new page) | 44  | ,     | 76  | L    | 108 | l    |
| 13  | CR (carriage return)        | 45  | -     | 77  | M    | 109 | m    |
| 14  | SO (shift out)              | 46  | .     | 78  | N    | 110 | n    |
| 15  | SI (shift in)               | 47  | /     | 79  | O    | 111 | o    |
| 16  | DLE (data link escape)      | 48  | 0     | 80  | P    | 112 | p    |
| 17  | DC1 (device control 1)      | 49  | 1     | 81  | Q    | 113 | q    |
| 18  | DC2 (device control 2)      | 50  | 2     | 82  | R    | 114 | r    |
| 19  | DC3 (device control 3)      | 51  | 3     | 83  | S    | 115 | s    |
| 20  | DC4 (device control 4)      | 52  | 4     | 84  | T    | 116 | t    |
| 21  | NAK (negative acknowledge)  | 53  | 5     | 85  | U    | 117 | u    |
| 22  | SYN (synchronous idle)      | 54  | 6     | 86  | V    | 118 | v    |
| 23  | ETB (end of trans. block)   | 55  | 7     | 87  | W    | 119 | w    |
| 24  | CAN (cancel)                | 56  | 8     | 88  | X    | 120 | x    |
| 25  | EM (end of medium)          | 57  | 9     | 89  | Y    | 121 | y    |
| 26  | SUB (substitute)            | 58  | :     | 90  | Z    | 122 | z    |
| 27  | ESC (escape)                | 59  | ;     | 91  | [    | 123 | {    |
| 28  | FS (file separator)         | 60  | <     | 92  | \    | 124 |      |
| 29  | GS (group separator)        | 61  | =     | 93  | ]    | 125 | }    |
| 30  | RS (record separator)       | 62  | >     | 94  | ^    | 126 | ~    |
| 31  | US (unit separator)         | 63  | ?     | 95  | _    | 127 | DEL  |

سپیس کا ASCII کوڈ 32 ہے جو کہ اوپر دیئے گئے ٹیبل میں دیکھا جاسکتا ہے۔ نیولائن کا کوڈ 10 ہے۔ a کی ویلیو 97 ہے اور A کی ویلیو ASCII کوڈ میں 65 ہے۔ جیسے کہ ہمارے پاس 33 کریکٹر ہیں اور ہر کریکٹر آٹھ بٹ ہے تو  $33 \times 8$  ہوگا جو کہ 264 بنتا ہے۔ مگر Huffman کی مدد سے یہی میسج ہم صرف 116 بٹز میں سنڈ کر سکتے ہیں۔

ہم کیسے Huffman کوڈ کا استعمال کر سکتے ہیں؟

سب سے پہلے تمام کریکٹر کو کاؤنٹ کریں اور سپیس بھی include کرے پھر میسج میں جو بھی انکی فریکوئنسی ہے وہ بھی دیکھیں۔ چلیں اس کو ایک مثال سے سمجھنے کی کوشش کرتے ہیں۔

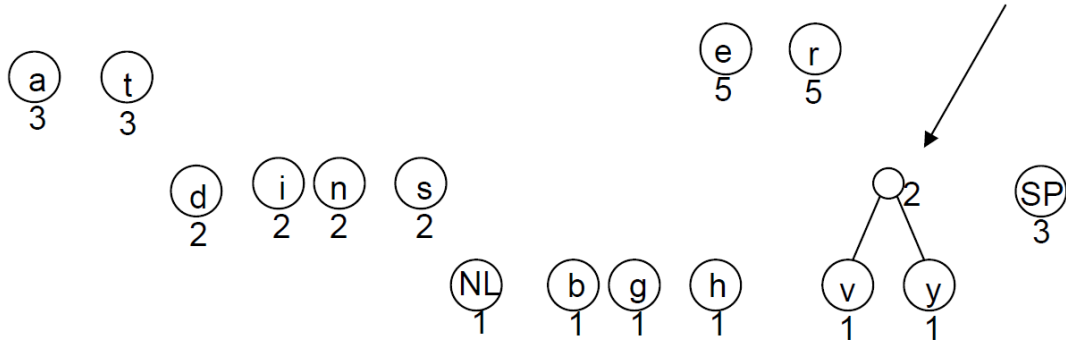
"Traversing threaded binary trees"

Total characters 33 included 3 Spaces and NL

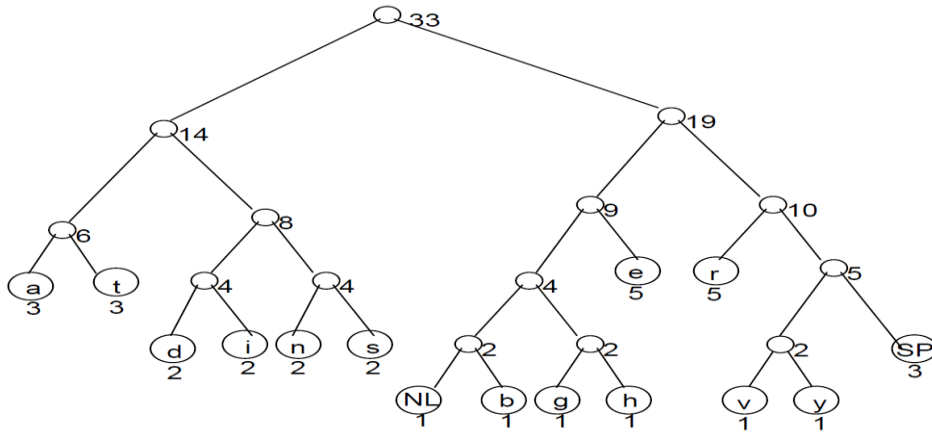
| Letters       | Freequency |
|---------------|------------|
| (NL) New Line | 1          |
| SP (Space)    | 3          |
| a             | 3          |
| b             | 1          |

|   |   |
|---|---|
| d | 2 |
| e | 5 |
| g | 1 |
| h | 1 |
| t | 3 |
| v | 1 |
| r | 5 |
| s | 2 |
| i | 2 |
| n | 2 |
| y | 1 |

جو کریکٹر جتنی بار آیا ہے وہی اسکی فریکوئنسی بنتی ہے۔ کریکٹر کاؤنٹ کرنے اور انکی فریکوئنسی معلوم ہو جانے کے بعد ہم ٹری بنائے گے۔ جو سب سے کم value والا نمبر ہوگا اسکو سب سے نیچے لکھیں اور باقی کو توڑا اوپر کر کے لکھیں اور ٹری بنالیں۔



ہم value کے مطابق نمبر لکھ لئے ہیں۔ اب ان کو آپس میں جوڑ کر ٹری بنائے گے۔



اور اس طرح ہم ایک بائنری ٹری بنالے گے۔

## Lecture Number: 26

اس لیچر میں بھی Huffman Alogrithm پر بات ہوئی ہے، اس لئے جیسے ہینڈ آؤٹ میں لیچر ہیں میں بھی اسی حساب سے اردو میں Translate کرنے کی کوشش کر رہا

ہوں۔ میں ہینڈ آؤٹ سے ایک پیرا گراف کاپی کر رہا ہوں یہاں کیونکہ اس پیرا گراف سے MCQ آنے کا چانس ہے۔

Huffman code is also a part of the JPEG image

Compression scheme. David Huffman introduced this algorithm in the year 1952 as Part of a course assignment at MIT.

In the previous lecture, we had started discussing a simple example to understand Huffman encoding. In that example, we were encoding the 32-character phrase:

"Traversing threaded binary trees". If this phrase were sent as a message in a network

Using standard 8-bit ASCII codes, we would have to send  $8 \times 32 = 256$  bits. However,

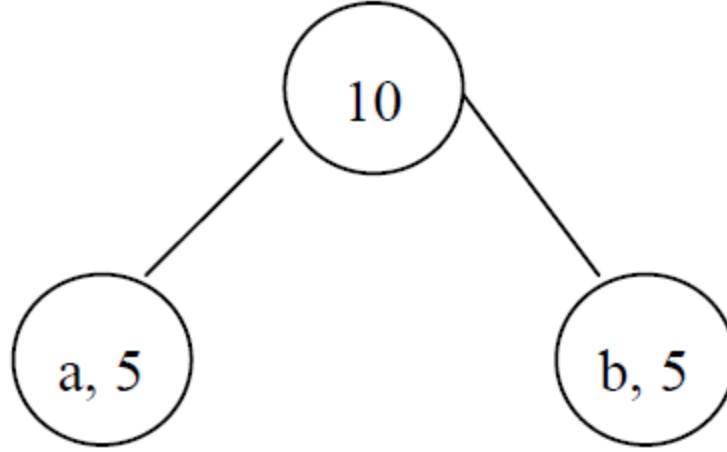
The Huffman algorithm can help cut down the size of the message to 116 bits.

In the Huffman encoding, following steps are involved:

جو بھی آپکو Huffman کوڈ میں Sentance دیا جیسے اس کے سارے لٹرز کاؤنٹ کریں، اس میں Space بھی کاؤنٹ کریں۔ پھر کریکٹر کی فریکوئنسی کو نکالیں، مثال کے طور پر اگر A چار بار آیا ہے تو A کی فریکوئنسی 4 ہوگی۔

جس کریکٹر کی فریکوئنسی سب سے کم ہوگی اسکو سب سے نیچے لکھیں اور پھر اس سے زیادہ فریکوئنسی والے کو اوپر لکھتے جائے۔

پھر دو فریکوئنسی کو ایسے ملا کر ایک نیا نوڈ بنادیں کہ جب نیا فریکوئنسی کو جمع کریں تو وہ ان دو نوڈ کی فریکوئنسی کے برابر ہو جن سے نیا نوڈ یا نئی فریکوئنسی بنی ہے۔ اسی طرح فریکوئنسی کو جمع کرتے رہیں یہاں تک کہ آپکی ترے بن نہ جائے۔ اگر A کی فریکوئنسی 5 ہو اور B کی بھی 5 تو نیا نوڈ یا نئی فریکوئنسی یوں بنی گی۔



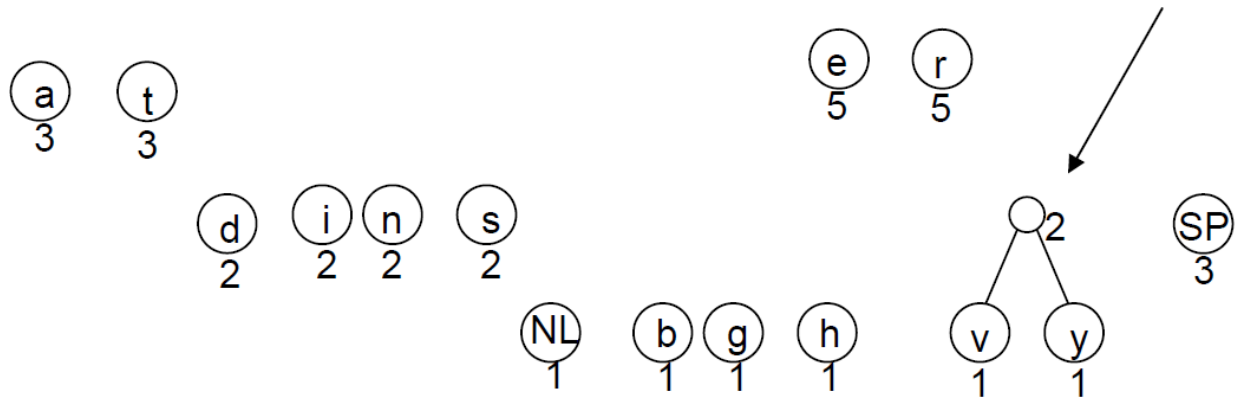
امید ہے کہ اب آپکو فریکوئنسی معلوم کرنا اور فریکوئنسی کو جمع کر کے نیا نوڈ بنانے کی سمجھ آگئی ہوگی۔ مثال کے طور پر آپکو پیپر میں یہ Sentence دیا ہے۔

### Traversing threaded binary trees

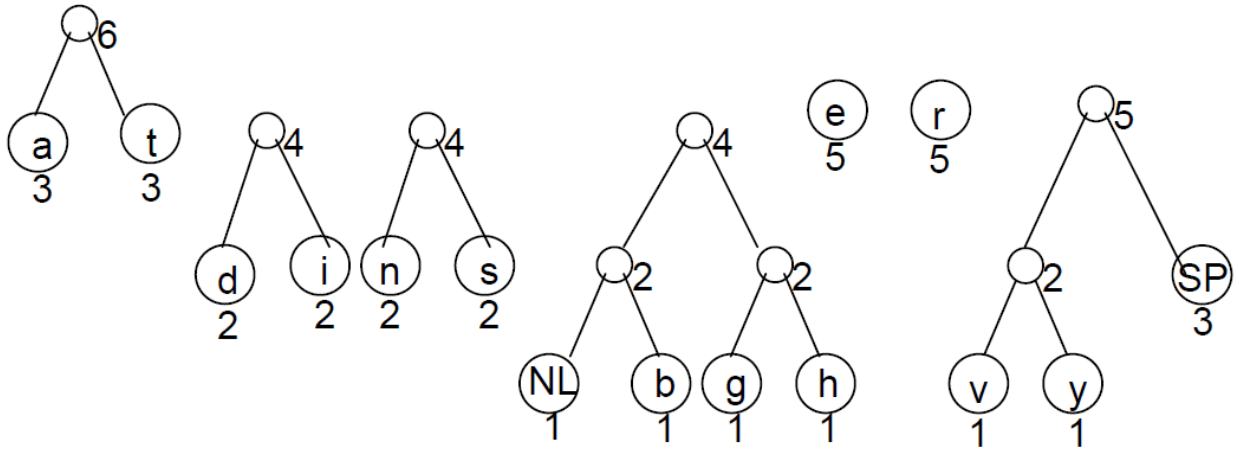
اب سب سے پہلے اس کے کریکٹر کاؤنٹ کریں، ایک بار پھر بتاتا چلوں کے کریکٹر کاؤنٹ کرتے وقت Space کو کاؤنٹ کرنا بھولنا نہیں۔ تو اس Sentence میں ہمارے پاس کتنے کریکٹر بنتے ہیں اور سپیس کتنی بار آئی ہے وہ کاؤنٹ کرنے کے بعد ہم فریکوئنسی کو معلوم کرنے کیلئے ایک ٹیبل بنائیں گے۔ جو کہ کچھ یوں بنے گا

| Character | frequency | character | frequency |
|-----------|-----------|-----------|-----------|
| NL        | 1         | I         | 2         |
| SP        | 3         | n         | 2         |
| A         | 3         | r         | 5         |
| B         | 1         | s         | 2         |
| D         | 2         | t         | 3         |
| E         | 5         | v         | 3         |
| G         | 1         | y         | 1         |
| H         | 1         |           |           |

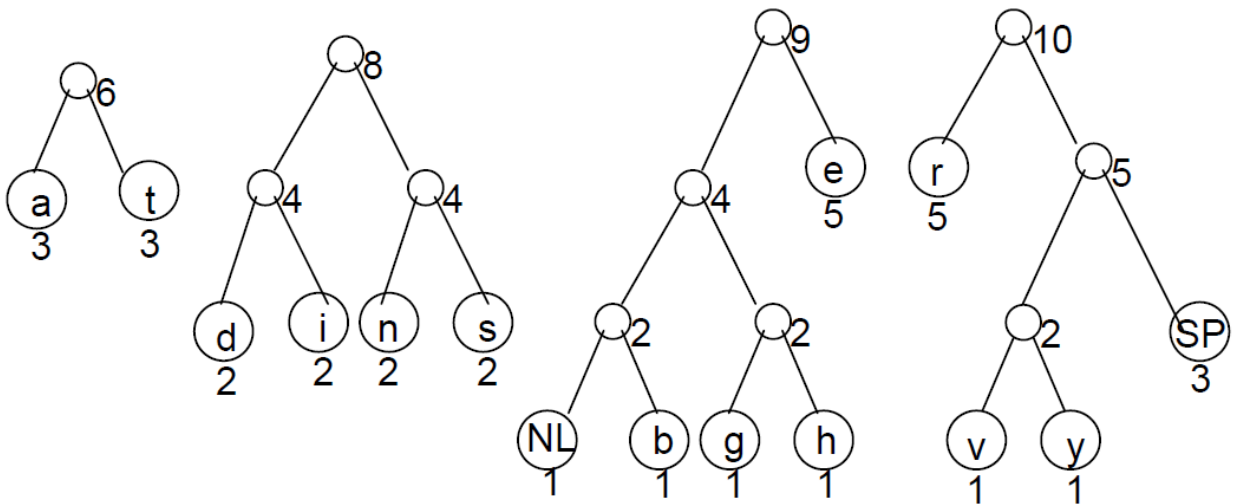
اب اس ٹیبل میں جو سب سے کم والے والے کریکٹر ہیں انکو سب سے نیچے لکھیں اور پھر اسے زیادہ نمبر والے کو اوپر لکھ کر ٹری بنانا شروع کریں۔ جیسے کہ



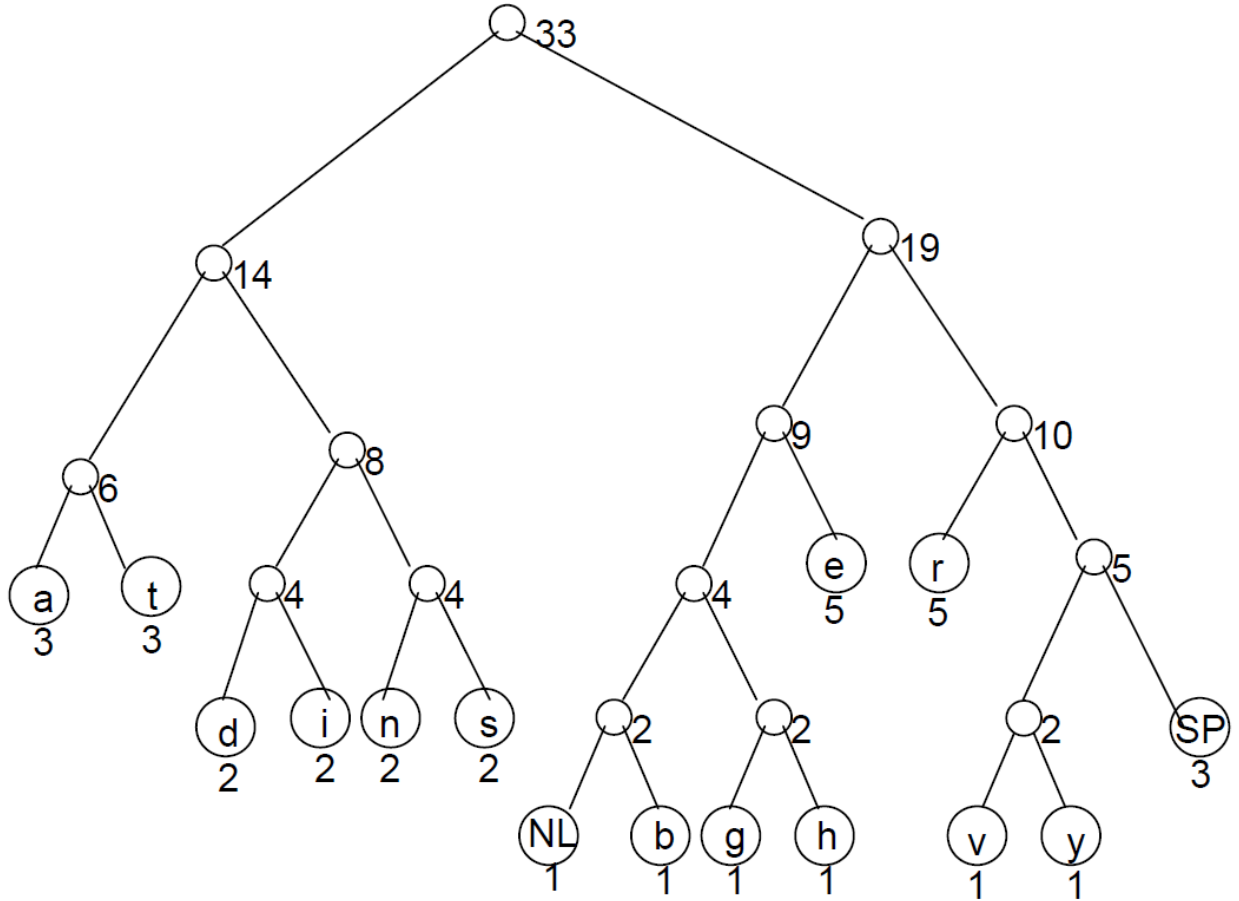
ہم نے سب سے پہلے کم والے کو لکھا اور پھر جمع کیا،



اور پھر دوسرے نمبر پر کم والے کو لکھا اور جمع کیا۔ یہاں ایک بات کا خیال رکھیں کہ جب آپ والے کو جمع کریں تو نئی ویلیو جمع ہونے والے ویلیوز کے برابر ہونہ زیادہ ہو نہ کم ہو۔ جیسے کہ آپ لوگ دیکھ سکتے ہیں کہ ہم نے جب 1+1 کیا تو جواب 2 آیا ہے۔ پھر اسی طرح فریکوئنسی کو جمع کرتے ہوئے ہم اپنی ٹری بنالے گے۔



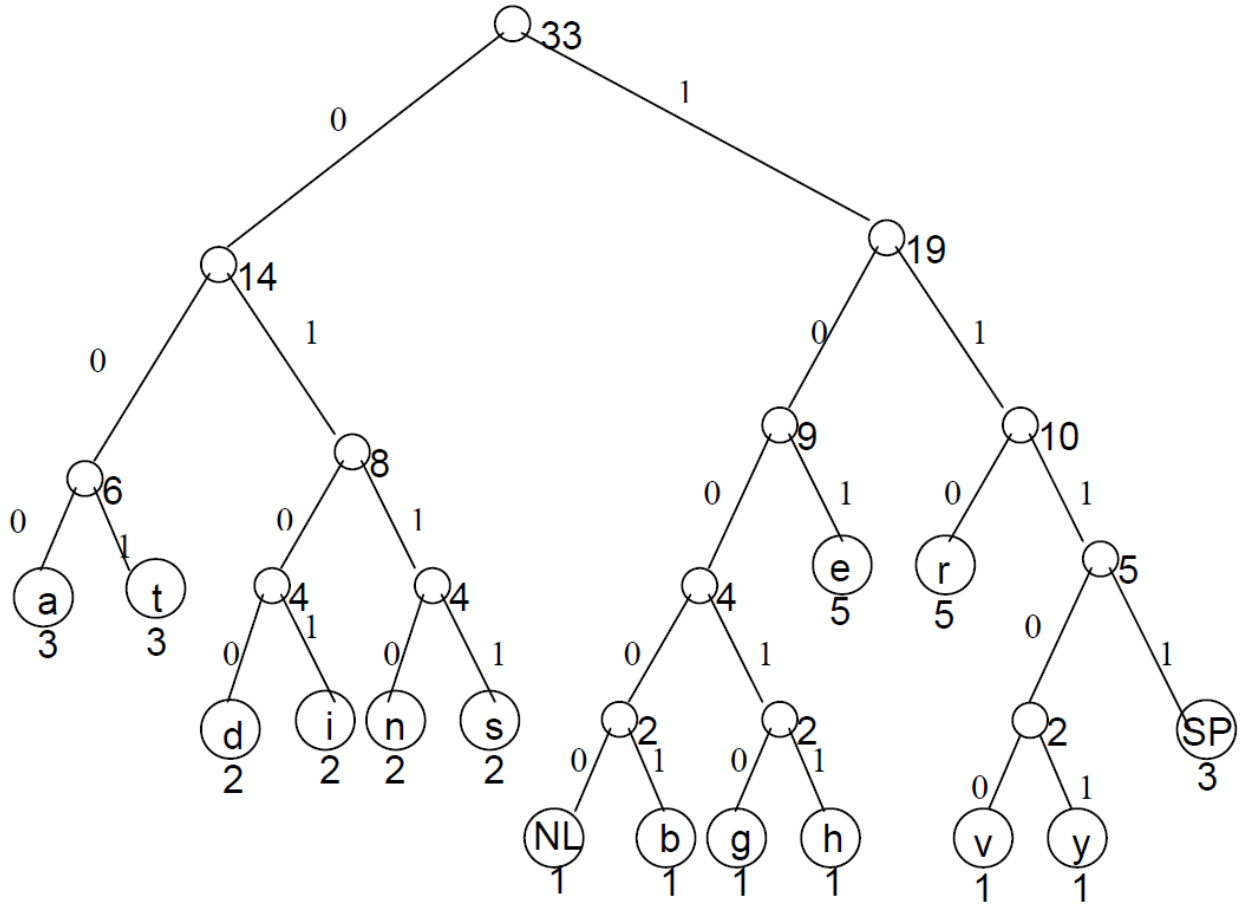
ایک اہم بات وہ یہ ہے کہ جب آپ کی ٹری کمپلیٹ ہو جائے گی تو جو ٹری کاروٹ نوڈ ہوگا وہ آپکو دیئے گئے Sentence کے ٹوٹل کریکٹر کے برابر ہونا چاہئے۔ جیسے کہ ہمارے پاس ٹوٹل 33 کریکٹر بنتے ہیں اور ہماری ٹری کاروٹ بھی نوڈ 33 ہوگا۔



روٹ نوڈ کو دیکھا جاسکتا ہے کہ 33 آیا ہے۔ امید ہے کہ اس میں کوئی مشکل پیش نہیں آئی گی آپ سب کو۔

اب اس ٹری سے ہم Huffman کے کریکٹر انکوڈنگ اسکیم بنائے گے۔

آپ روٹ سے شروع کریں اور لفٹ سائیڈ پر 0 لکھیں اور ریت سائیڈ سب ٹری پر 1  
یہی پراسیس رپیٹ کرتے ہوئے تمام ٹری پر Apply کریں۔ چلیں آپ کو ایک مثال سے  
سمجھا دیتا ہوں۔



میں روٹ 33 کے لفٹ سائیڈ پر 0 لکھا اور رائٹ سائیڈ پر 1، اسی طرح 14 کے لفٹ  
سائیڈ پر 0 اور رائٹ 1 لکھا، نوڈ 14 کے لفٹ رائٹ برانچز ہیں اس لئے نوڈ 14 ان

کیلئے روٹ ہے، اور روٹ کے لفٹ سائیڈ پر 0 آتا ہے اور رائٹ سائیڈ پر 1۔ اسی طرح باقی سب پر بھی ہم یہی میتھڈ Apply کریں گے۔ یہ میتھڈ Apply کرنے دے بعد ہمیں آخر میں Huffman کریکٹر کوڈ مل جائے گا۔ جو کے کچھ یوں ہوگا۔

| character | code  | character | code |
|-----------|-------|-----------|------|
| NL        | 10000 | i         | 0101 |
| SP        | 1111  | n         | 0110 |
| a         | 000   | r         | 110  |

|   |       |   |       |
|---|-------|---|-------|
| b | 10001 | s | 0111  |
| d | 0100  | t | 001   |
| e | 101   | v | 11100 |
| g | 10010 | y | 11101 |
| h | 10011 |   |       |

اور یوں ہمارا یہ ٹاسک کمپلیٹ ہو جائے گا۔

یاد رکھیں کہ کمپیوٹر میں ہر کریکٹر بائری فاریٹ میں سٹور ہوتا ہے۔ اور ہر کریکٹر کا ایک کوڈ ہوتا ہے جو کہ ASCII کوڈ کہلاتا ہے، اور یہ ASCII کوڈ 1 اور 0 پر مشتمل ہوتا ہے۔ ASCII کے آٹھ بٹس ہوتے ہیں۔ اگر آپ سے یہ پوچھا جائے کہ آپ کے Sentence میں 33 کریکٹر ہیں تو اس کی Length کتنی بنتی ہے تو آپ 33\*8 کر کے

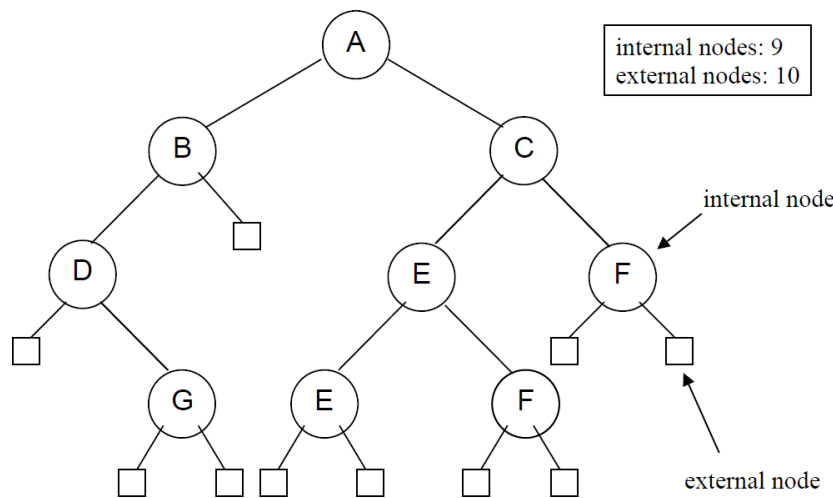
اپنا جواب لکھ سکتے ہیں۔ کیونکہ ہمیں پتا ہے کہ ASCII کوڈ کے 8 بٹس ہوتے ہیں اور ہمارے پاس کریکٹر 33 ہیں اس لئے دونوں کو ضرب کر کے جواب لکھ دیں۔

تولیف اور انر نوڈ کیا ہوتے ہیں؟

The leaf node is a node whose left and right links is NULL. An inner node has a left or right or both children. If the bit is zero, it will go to the left of the root. If the bit is one, it will go to the right side of the root before reaching to the child node.

There are some mathematical properties of binary trees, which are actually theorems

باسٹری ٹری میں انٹرئل نوڈ N ہوتا ہے مطلب کہ باسٹری ٹری میں نمبروف نوڈز ہوتے ہیں۔ اور ہر انٹرئل نوڈ کا  $N+1$  ہو سکتا ہے۔ ہم اس کو ایک مثال سے دیکھتے ہیں



اس ٹری میں ہم دیکھ سکتے ہیں کہ ہمارے پاس 9 انٹرئل نوڈز ہیں اور 10 ایکسٹرئل نوڈز ہیں۔ اس ٹری کو دیکھتے ہوئے ہم دیکھ سکتے ہیں کہ جو لیف نوڈز ہیں جن کے رائٹ لفٹ Null ہے مطلب کوئی نوڈز نہیں ہے وہ ایکسٹرئل نوڈز ہے۔ اسی طرح انٹرئل نوڈز کے رائٹ اور لفٹ پر ہم سب ٹری یعنی نوڈز دیکھ سکتے ہیں۔

### Lecture Number: 27

Properties of Binary Tree:

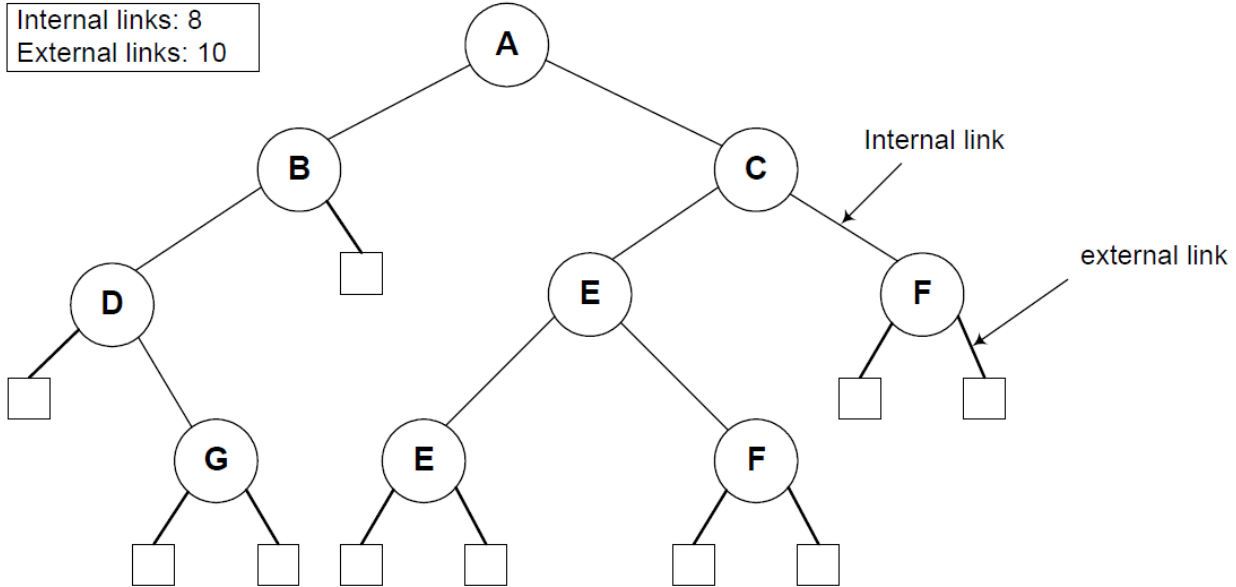
بائنری ٹری کی پراپرٹی کے بارے میں ہم نے اس سے پچھلے لیچر میں بھی تھوڑا سے دیکھا تھا کہ اگر بائنری میں انٹرئل نوڈز  $N$  ہوں گے تو ایکسٹرئل نوڈز  $N + 1$  ہوں گے۔ بائنری پراپرٹی کی یہ تعریف ایک بار دیکھ لیں ہو سکتے ہیں یہ پیپر میں آجائے۔

Property

A binary tree with  $N$  internal nodes has  $2N$  links,  $N-1$  links to internal nodes and  $N+1$  links to external nodes.

اس تعریف کے مطابق بائنری ٹری جو کہ انٹرئل نوڈز  $N$  رکھتی ہے اس کے دو لنکس ہو سکتے ہیں، انٹرئل نوڈز کو  $N-1$  لنکس ہو سکتے ہیں اور ایکسٹرئل کو  $N+1$  ہو سکتے ہیں۔ بائنری ٹری میں لنک سے مطالب وہ لائن ہوتی ہے جو ہم لوگ نوڈز کے درمیان لگتے ہیں نوڈز کو ایک دوسرے سے کنیکٹ کرنے کیلئے۔ بائنری ٹری میں کچھ لنکس انٹرئل

لنک ہوتے ہیں اور کچھ کو ہم ایکسٹرئل لنکس کہتے ہیں۔ آئیں ایک مثال سے سمجھنے کی کوشش کرتے ہیں کہ انٹرئل اور ایکسٹرئل لنکس کون سے ہوتے ہیں۔



اس بائنری ٹری میں ہوں دیکھ سکتے ہیں کہ کون سے انٹرئل لنکس ہیں اور کون سے ایکسٹرئل لنکس ہیں۔ اس ٹری کو دیکھنے کے بعد آپ کو باخوبی اندازہ ہو گیا ہوگا کہ انٹرئل اور ایکسٹرئل لنکس کون سے ہوتے ہیں۔

بائنری ٹری میں روٹ نوڈ کے علاوہ ہر نوڈ کا ایک یونیک parent ہوتا ہے۔ ہر لنک نوڈ کو اس کے parent نوڈ سے کنیکٹ کرتا ہے اس لئے انٹرئل لنک کو  $N-1$  لنکس کنیکٹ کرتے ہیں۔ اسی طرح انٹرئل نوڈز کو  $N+1$  فارمولا سے لکھا جاتا ہے۔ اس

$N-1 + N + 1 = 2N$  links کہتے ہیں۔

بائری ٹری کے چار ٹراور سنگ میتھوڈس ہوتے ہیں۔

Preorder.1

Inorder.2

Postorder.3

Levelorder.4

پہلے تین Order کو Recursion کا استعمال کر کے Implement کیا جاتا ہے۔ یہ

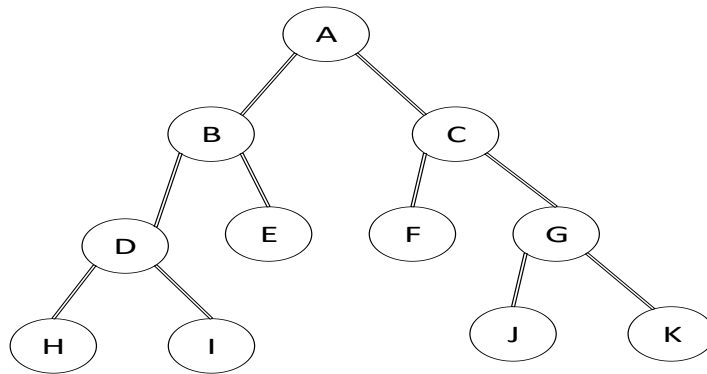
Recursion روٹین بہت چھوٹے ہوتے ہیں 3 یا 4 لائن کے کوڈ ہوتے ہیں۔

Recursive روٹین میں اسٹیک کا استعمال کرتے ہیں۔

بائری ٹری کو اسٹیک فری ٹری بنانے اور اس کی سپیڈ کو تیز کرنے کیلئے Threaded

بائری ٹری کا استعمال کیا جاتا ہے۔ آئیں اب ان سب ٹراور سل کو ایک ایک کر کے

سمجھنے کی کوشش کرتے ہیں۔

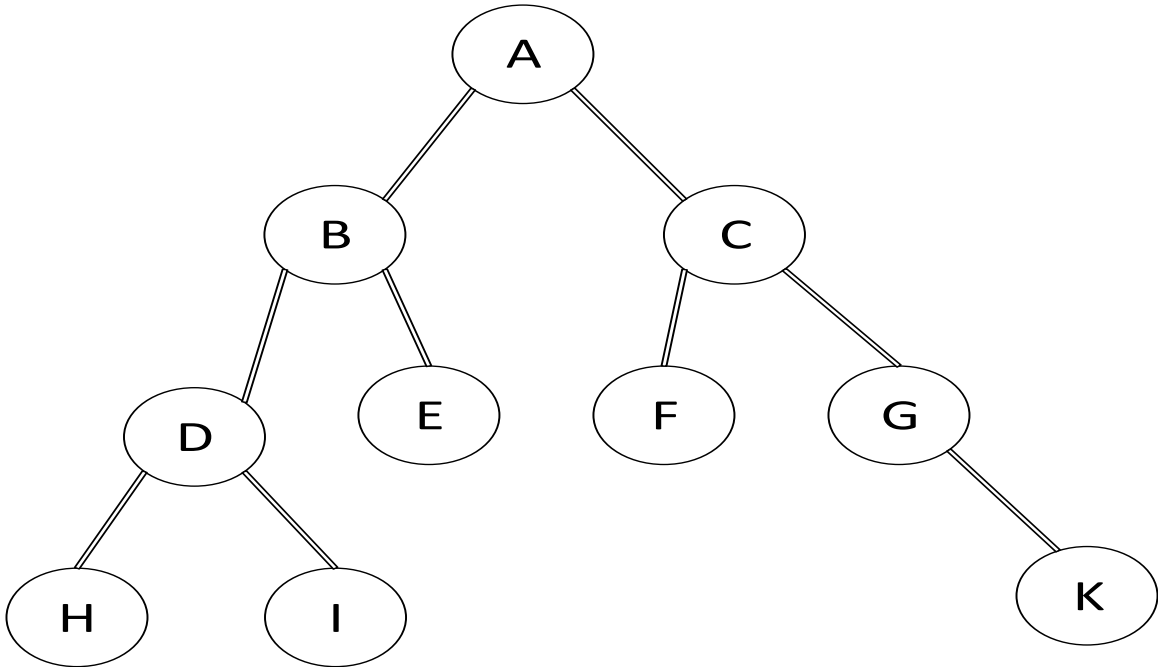


## پری آرڈر ٹراورسل

اس ٹراورسل میں ہم سب سے پہلے روٹ کو ٹراورس کرتے ہیں پھر لفٹ سب ٹری اور آخر میں رائٹ سب ٹری کو ٹراورس کرتے ہیں۔ Pre مطلب پہلے روٹ کو کرنا ہے۔ ان آرڈر ٹراورسل مطلب روٹ کو درمیان میں ٹراورس کرنا ہے اور پوسٹ آرڈر ٹراورسل مطلب روٹ کو آخر میں ٹراورس کرنا ہے۔ اب ہم ایک ایک کر کے ان سب ٹراورسل آرڈرز کو سمجھنے کی کوشش کرتے ہیں۔

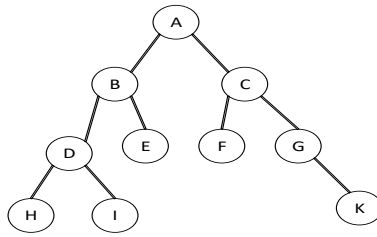
پہلا: پری آرڈر ٹراورسل

اس کو سمجھنے کیلئے سب سے پہلے ہم ایک ٹری Draw کرتے ہیں تاکہ آسانی سے سمجھ سکے۔



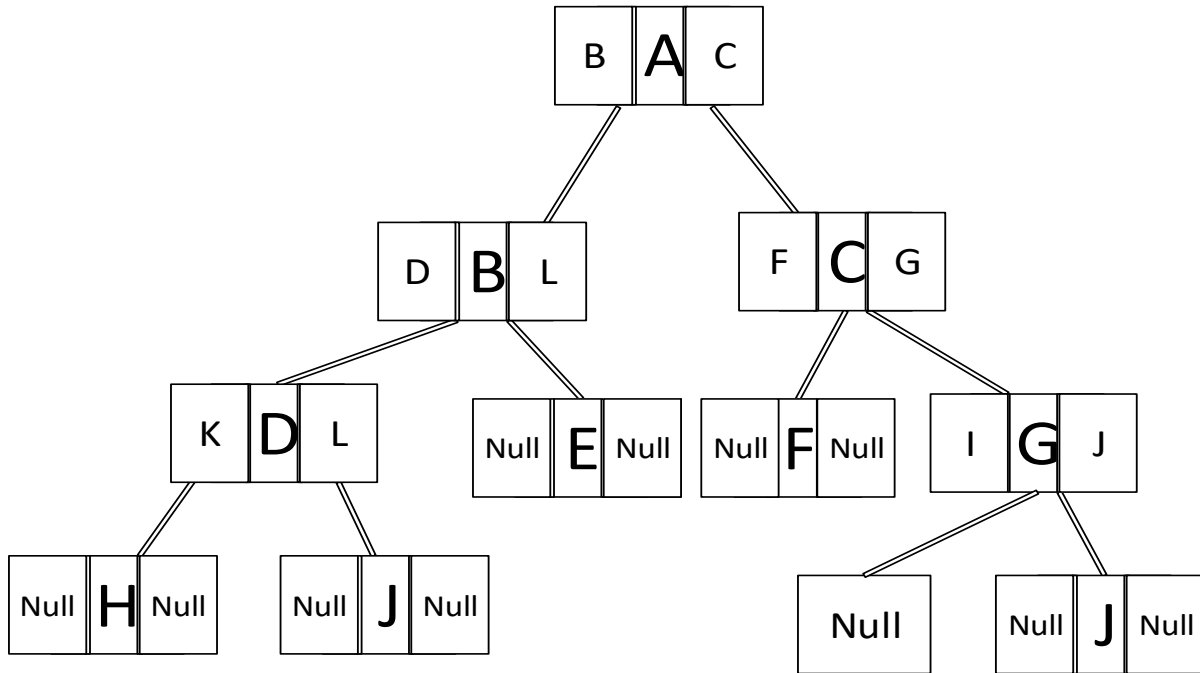
اسی ٹری کو استعمال کرتے ہوئے ہم چاروں ٹراور سلز کو سمجھنے کی کوشش کریں گے۔  
 توپری آرڈر ٹراور سل کے Algorithm کے مطابق سب سے پہلے ہم نے روٹ کو  
 پراسیس کریں گے، پھر روٹ کے لفٹ سائیڈ پر جو سب ٹری ہوگا اس پر پری آرڈر  
 ٹراور سل Apply کریں گے، اور آخر میں روٹ کے رائٹ سائیڈ کی SubTree  
 ٹراور سل کریں گے۔

اب اس ٹری پر ہم پری آرڈر ٹراور سل کا Algorithm لگا کر ٹراور سل کریں گے۔  
 تو سب سے پہلے ہمارا روٹ A پرنٹ ہوگا، پھر روٹ کے لفٹ سائیڈ پر جو سب ٹری ہے  
 اسکو دیکھیں گے۔ لفٹ سائیڈ کے بعد پھر رائٹ سائیڈ پر جائے گے اور پھر رائٹ کے  
 لفٹ سائیڈ کو پرنٹ کروا کر پری آرڈر پروسس کمپلیٹ کرے گے۔ چلے ہم اسی ٹری کا  
 پری آرڈر پروسس دیکھ لیتے ہیں۔



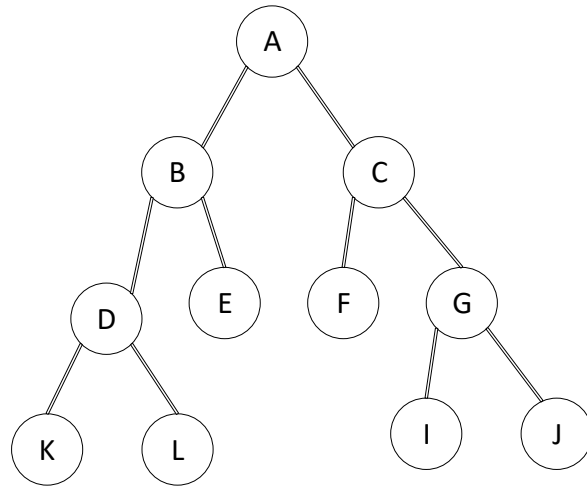
ADH I B E C F G K تو یہ ہماری ٹری کا پری آرڈر ٹراور سل ہو گیا ہے۔  
 پری آرڈر پروسس میں روٹ سب سے پہلے پرنٹ ہوتا ہے پھر روٹ کے لفٹ نوڈ اور  
 پھر رائٹ نوڈ پرنٹ ہوتے ہیں۔

پری آرڈر ٹر اور سل کے بعد ہم تھریڈڈ بائری ٹری بنائیں گے جس میں لیف نوڈ نل نہیں ہوتے بلکہ ان کو پوائنٹر سے دوسرے نوڈ کی طرف پوائنٹ کر دیا جاتا ہے۔ اب میں ٹری کو ایک اور شکل میں لکھ دیتا ہوں تاکہ آپ سب کو آسانی سے پتا چل سکے کہ ہماری اس ٹری میں کون کون سے نوڈز نل ہیں اور کہاں کہاں ہم threaded بائری ٹری استعمال کر کے پوائنٹرز استعمال کر کے نل نوڈز کو ٹر اور س کریں گے۔



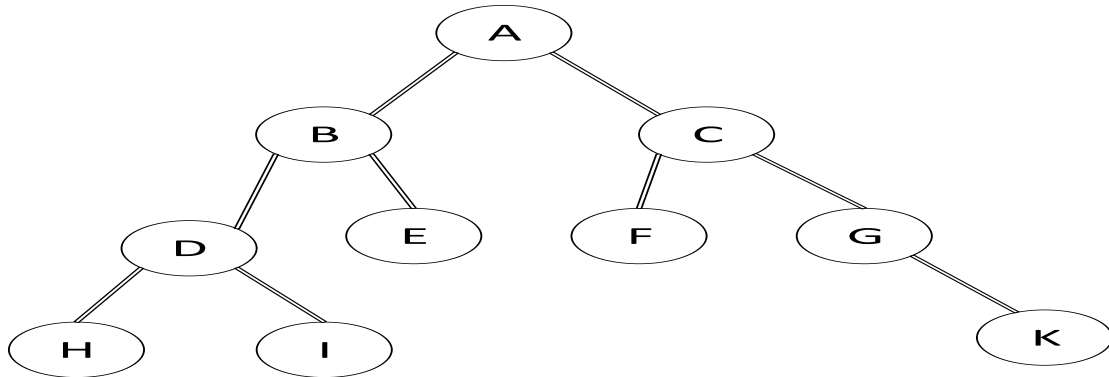
یہ ہماری وہی ٹری ہے اس میں چینج صرف یہ کیا ہے کہ ہر بائری ٹری کے ساتھ جو دو پوائنٹرز ہوتے ہیں وہ شو کروا دیئے ہیں۔ اگر اس ٹری کو دیکھا جائے تو ہمیں پتا چل جائے گا کہ کون کون سے نوڈز نل ہیں اب نل نوڈز پر ہم Threaded ٹری کا استعمال کر کے پوائنٹرز کی مدد سے ان نل نوڈز کو ٹر اور س کرے گے۔ ہم دیکھ سکتے ہیں کہ ہماری ٹری

میں 7 نوڈز ایسے ہیں جن کے پوینٹرز نل ہیں، ان نل پوائنٹرز میں UseFul انفارمیشن رکھنے کیلئے Threaded بائری ٹری کا کانسیپٹ عمل میں لیا گیا ہے۔ Threaded بائری ٹری کی دو قسم ہوتی ہے۔  
 پہلا: سنگل تھریڈڈ بائری ٹری  
 دوسرا: ڈبل تھریڈڈ بائری ٹری۔  
 سنگل threaded بائری ٹری میں ہم رائٹ سائیڈ نل پوائنٹر کو ان آرڈر Successor کی طرف پوائنٹ کر دیتے ہیں اگر ان آرڈر سکسر ہو تو۔ اب ان آرڈر سکسر ہوتا کیا ہے؟ ان آرڈر سکسر کیسے نکلتے ہیں ہم ایک مثال سے سمجھنے کی کوشش کرتے ہیں۔



Left Pointer= Inorder Predecessor  
 Right Pointer= Inorder Successor

ہم نے یہ ٹری بنائی تھی اور اب پہلے اس پر ہم ان آرڈر ٹراورسنگ کا Algorithm لگا کر چیک کریں گے کہ ان آرڈر سکسر کون سا بنتا ہے ہر نل نوڈ کیلئے۔ تو ان آرڈر ٹراورسنگ میں سب سے پہلے لفٹ سب ٹری کو ٹراورس کرتے ہیں پھر روٹ نوڈ کو پروس کرتے ہیں پھر رائٹ سب ٹری کو ٹراورس کرتے ہیں۔ ہم اپنی اس ٹری کا پہلے پری آرڈر ٹراورسل کرتے ہیں پھر ان آرڈر اینڈ آخر میں پوسٹ آرڈر ٹراورسل کر کے اس کو اچھی طرح سے سمجھنے کی کوشش کریں گے۔ پری آرڈر ٹراورسل میں پہلے روٹ پروس ہوتا ہے پھر لفٹ سب ٹری اور پھر رائٹ سب ٹراورس ہوتی ہے۔



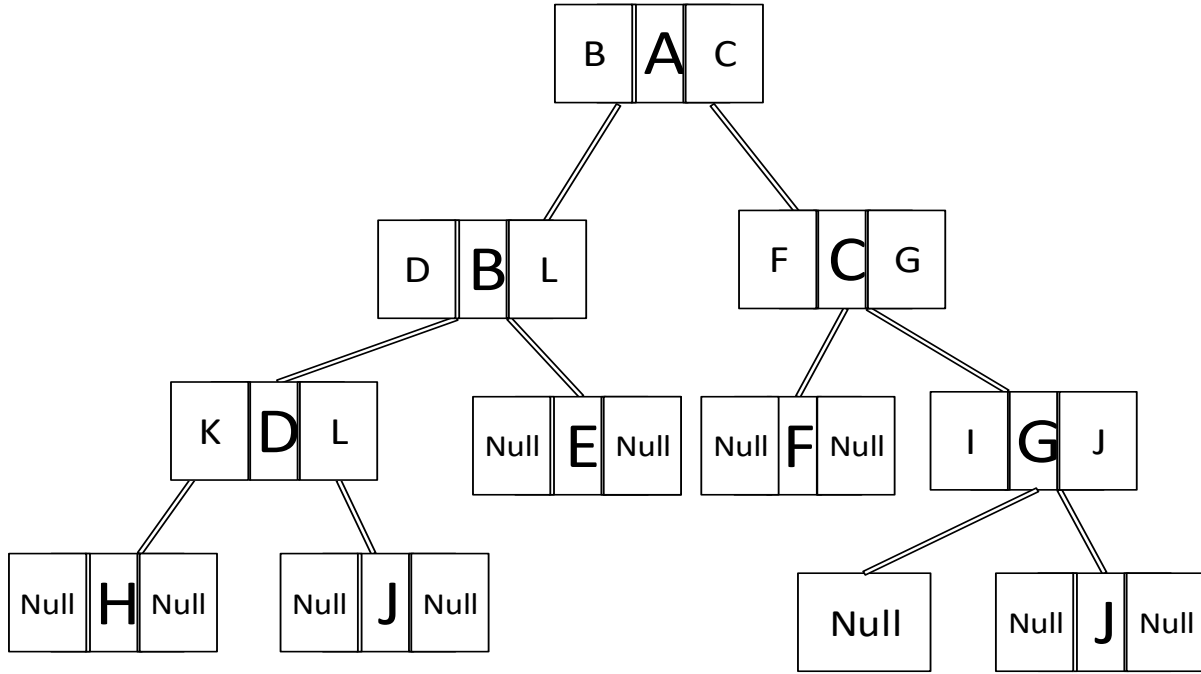
ABDHI E C F G k

تو یہ ہم نے پری آرڈر ٹراورسل کر کے ٹری کی اوٹ پٹ نکل لی ہے۔ اب ہم

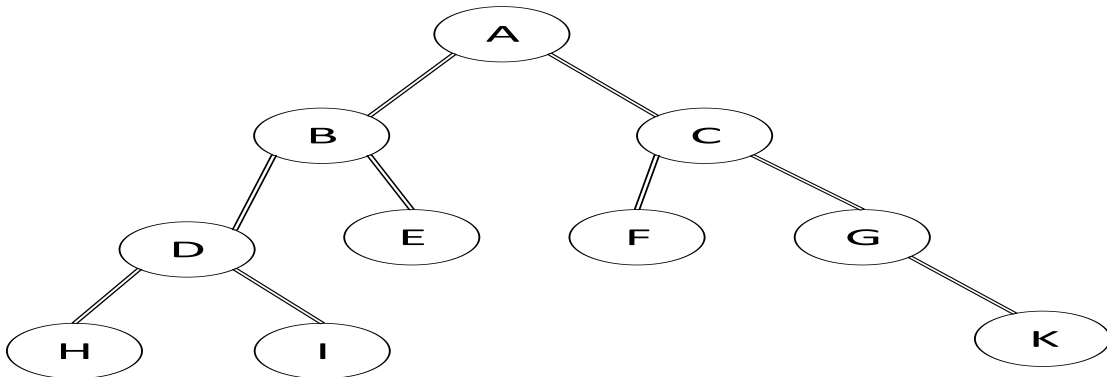
Left Pointer= Inorder Predecessor

Right Pointer= Inorder Successor

اس algorithm کے تحت تھریڈڈ بائری آسانی سے بنا سکتے اور نل نوڈز کو پوانٹ کر سکتے ہیں۔ اس میں اب ان آرڈر سکسر کو دیکھنے کیلئے ہم یہ دیکھیں گے کہ A سے پہلے کوئی نوڈ ہے یا نہیں تو ہم دیکھ سکتے ہیں کہ A سے پہلے کوئی نوڈ نہیں اس لئے A کا ان آرڈر سکسر نہیں ہے مگر B کیلئے ان آرڈر سکسر A ہے۔



اب میں اسی ٹری کا ان آرڈر ٹراورسل نکل کر دوں گا۔

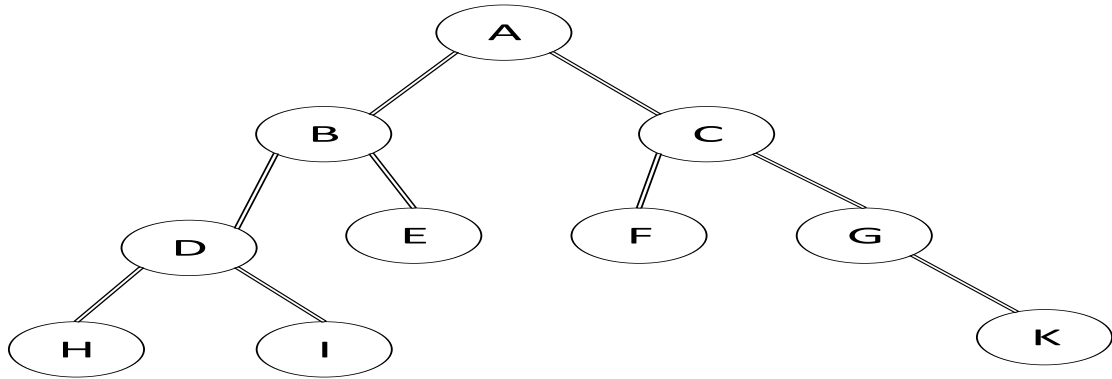


ان آرڈر ٹراورسل میں سب سے پہلے لفٹ سب ٹری کو ٹراورس کرتے ہیں پھر روٹ نوڈ کو اور پھر رائٹ سب ٹری کو ٹراورس کرتے ہیں۔ تو اس ٹری کا ان آرڈر ٹراورسل کی آؤٹ پٹ کچھ یوں ہوگی۔

HDIBEAFCGK

اس آؤٹ پٹ کو دیکھتے ہوئے اب آپ لوگ پریکٹس کیلئے ان کا ان آرڈر سکس اور Predecessor نکلنے کی کوشش کریں۔

اب ہم اپنی اسی ٹری کو استعمال کرتے ہو پوسٹ آرڈر ٹراورسل کر کے دیکھیں گے



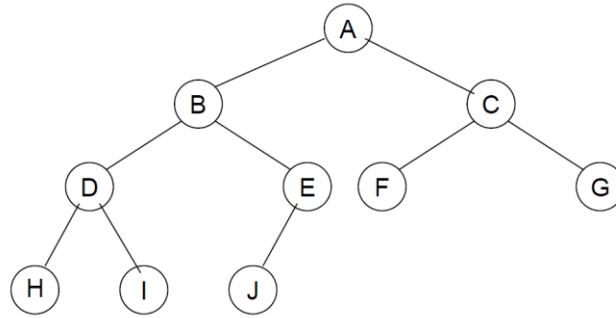
پوسٹ آرڈر ٹراورسل میں جیسے کہ میں نے بتایا تھا کہ پہلے لفٹ سب ٹری کو ٹراورس ہوتی ہے پھر رائٹ سب ٹری اور آخر میں روٹ کو پراسیس کرتے ہیں۔ اب پوسٹ آرڈر ٹراورسل کو میں آپ لوگوں کیلئے پریکٹس کیلئے چھوڑ رہا ہوں۔ اگر سمجھ نہ آئے تو نیچے ای میل آئی ڈی اور فیس بک آئی ڈی پر مجھ سے رابطہ کر سکتے ہیں۔

## لیکچر نمبر 28

لیکچر نمبر 28 میں بائری ٹری کی ٹراور سل بتائی گئی ہے جو ہم پہلے ہے دیکھ چکے ہیں، اس میں ایک نوڈ ہے جس کو ڈمی نوڈ کا نام دیا گیا ہے۔ اب یہ ڈمی نوڈ کیا ہوتا اور کیوں ہم اس کا استعمال کرتے ہیں اس کو سمجھنے کیلئے آپ لوگ خود ہینڈ آؤٹ کو دیکھ۔  
ڈمی نوڈ کے علاوہ اس لیکچر میں کمپلیٹ بائری ٹری کا بتایا گیا ہے، کمپلیٹ بائری ٹری کیا ہوتی ہے اس کی تعریف ہمارے ہینڈ آؤٹ میں کچھ یوں لکھی گئی ہے۔

A complete binary tree is a tree that is completely filled, with the possible exception of the bottom level. The bottom level is filled from left to right.

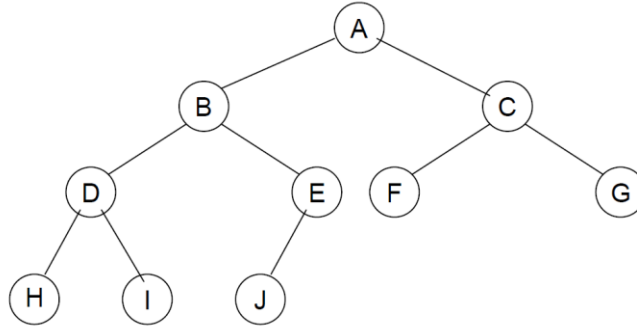
اب ہم یہ دیکھتے ہیں کہ بائری ٹری کو ہم Array کے ساتھ کیسے سٹور کر سکتے ہیں۔ اس کیلئے ہم ایک بائری ٹری لی ہیں۔



تو یہ ہماری بائری ٹری ہو گئی ہے جس کو اب ہم ارے کے ساتھ سٹور کرنے کی کوشش کرے گے۔ ٹری کو ارے کے سٹور کرنے کیلئے کچھ رولز ہیں۔

For any array element at position  $i$ , the left child is at  $2i$ , the right child is at  $(2i+1)$  and the parent is at floor  $(i/2)$ .

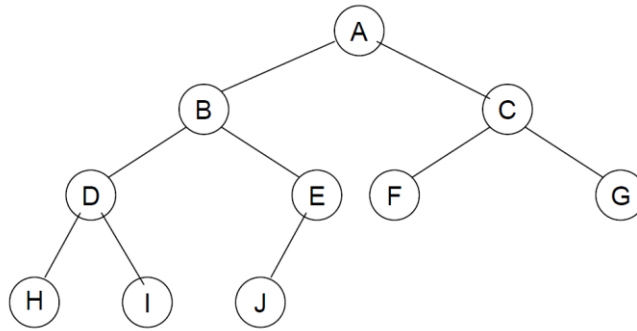
ہمارے پاس ٹری میں کچھ نوڈز ہیں اور انکے لفٹ رائٹ نوڈز یعنی سب ٹریز بھی ہیں۔  
اب ہم اس فارمولا کا استعمال کر کے اس اررے کو سٹور کرے گے۔



اس ٹری میں اگر ہم دیکھیں تو ہمارے پاس نوڈ A اور اس کے لفٹ پر سب ٹری B اور رائٹ پر سب ٹری C، پھر اسی طرح B کے لفٹ رائٹ پر بھی سب ٹریز ہیں۔ اب ہم اپنے فارمولے کے مطابق اس ٹری کو اررے میں سٹور کریں گے۔ یاد رکھیں کہ اررے میں ایلیمنٹس 0 سے شروع ہوتے ہیں، مگر ہم اس ٹری کو 0 ایلیمنٹ پر سٹور نہیں کرے گے بلکہ روٹ A کو اررے کے ایلیمنٹ 1 پر سٹور کر کے اپنے اس فارمولا کا استعمال کر کے باقی کے نوڈز کو سٹور کرے گے۔

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
|   | A |   |   |   |   |   |   |   |   |    |    |    |    |    |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

اب ہم نے ایلیمینٹ 1 پر رکھ دیا ہے اب اگر ہم اپنے فارمولا کو دیکھیں جس میں کہا گیا ہے کہ لفٹ نوڈ  $2i$  ہوگا اور رائٹ نوڈ  $2i+1$  ہوگا۔ اس فارمولا کو یاد رکھیں تاکہ پیپر میں جب تری کو ارے میں سٹور کرنے کا کہا جائے تو آپ نوڈز کو سہی جگہ پر سٹور کر سکیں۔



|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
|   | A |   |   |   |   |   |   |   |   |    |    |    |    |    |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

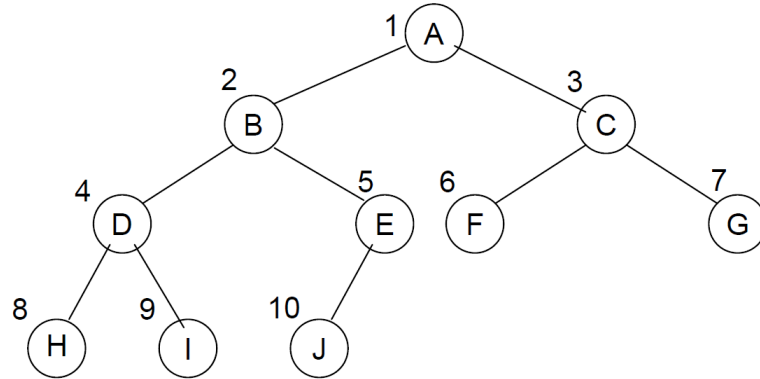
اب اگر ہم دیکھیں تو ہم نے روٹ کو ارے کے Element نمبر ایک پر سٹور کیا ہے اب A کے لفٹ نوڈ کو 2 پر اور رائٹ کے نوڈ کو 3 سٹور ہونا چاہئے۔ تو اپنا فارمولا دیکھیں کے لفٹ ٹری  $2i$  ہوگا اور رائٹ ٹری  $2i+1$  ہوگا۔  $2*1$  تو B نمبر پر سٹور ہوگا اور رائٹ نوڈ جو کہ C ہے وہ  $2i+1$  ہوگا تو یہ بن جائے گا  $2+1$ ، تو اس طرح سے C نمبر تین پر سٹور ہوگا۔ اب ہم دیکھتے ہیں کہ B کا لفٹ سب ٹری اور رائٹ سب ٹری کہاں سٹور ہونا چاہئے۔ پھر سے اپنا فارمولا لگائیں گے۔ ہمارے پاس اب  $2i$  پہلے سے 2 ہے تو ہم  $2*2$

کر کے D کو سٹور کرے گے جو کہ 4 والے بنتی ہے، اب B کا رائٹ سب ٹری  $2i + 1$  ہوگا تو D ہمارے پاس  $4 + 1$  ہو کر E ارے کے الیمینٹ نمبر 5 پر

سٹور ہوگا۔ اسی طرح فار مولا کا استعمال کر کہ ہم اس ٹری کو ارے میں سٹور کر سکتے ہیں۔ تمام ویلیو سٹور کرنے کے ہمار پاس output کچھ یوں آنی چاہئے۔

|   | A | B | C | D | E | F | G | H | I | J  |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |

ہماری ٹری کی فائنل output کے بعد ہم ٹری پر نمبرنگ کر کہ اور اور فار مولا سے اس ٹری کو ٹھیک ہونے کو ثابت کرتے ہیں۔



اب ہم یہ دیکھتے ہیں کہ چلڈ نوڈ کا جو فار مولا ہے کیا اس کے مطابق ہماری ٹری ٹھیک ہے بھی یا نہیں۔ اگر کوئی نوڈ آئی پوزیشن پر ہو تو اس کا parent کا فار مولا ہوگا،

*floor(i/2)*

اب فار مولا کے مطابق سب سے پہلے نوڈ 1 کو دیکھتے ہیں کہ کیا اس کا parent نمبر پانچ پر آتا ہے یا نہیں۔ تو فار مولا ہوگا (10/2) floor اور جواب آ یا گا 5۔ تو ٹری میں دیکھا جا سکتے ہیں کہ نوڈ 1 کا parent نمبر پانچ پر ہے جو کہ سہی ہے۔ اب اگر ہم نوڈ 2 کا parent دیکھیں تو فار مولا ہوگا (9/2) floor جو کہ 4.5 بنتا ہے مگر یہاں ہم راؤنڈ down کر کے 4 لیں گے اور 2 کا parent بھی سہی جگا ہے۔ اب نوڈ 3 کو اگر فار مولا لگا کر دیکھیں تو وہ بھی سہی بنتا ہے۔ باقی جو رہ گئے ہیں وہ آپ لوگ خود فار مولا لگا کر دیکھیں گے سہی آتے ہیں یا نہیں۔

یہ جو فار مولا ہم نے لگا کر parent نوڈز چیک کئے ہیں اس کو ہی لیول آرڈر نمبر کہا جاتا ہے یا لیول آرڈر ٹر اور سنگ کہا جاتا ہے۔ ہینڈ آؤٹ کا یہ پیرا گراف دیکھ لیں۔

Now let's talk about the usage of pointers and array. We have read that while implementing data structures, the use of array makes it easy and fast to add and remove data from arrays. In an array, we can directly locate a required position with the help of a single index, where we want to add or remove data. Array is so important that it is a part of the language. Whereas the data structures like tree, stack and queue are not the part of C or C++ language as a language construct. However we can write our classes for these data structures. As these data structures are not a part of the language, a programmer can not declare them directly. We can not declare a tree or a stack in a program. Whereas we can declare an array directly as `int x [ ]`; The array data type is so efficient and is of so common use that most of the languages support it. The compiler of a language handles the array and the programmer has to do nothing for declaring and using an array.

اگر آپ کے مائنڈ میں سوال ہو کہ ہم ٹری کے ساتھ اررے کا استعمال کیوں کرتے ہیں تو اس پیراگراف میں آپ کو سوال کا جواب مل جائے اور پیپر میں بھی یہاں سے کچھ پوچھا جاسکتا ہے۔

ڈیٹا اسٹرکچر کو Implement کرتے ہوئے اگر اررے کا استعمال کیا جائے تو اررے ڈیٹا اسٹرکچر کو آسان اور فاسٹ بنادیتی ہے، اور اررے ڈیٹا کو Add اور Remove کرنے کو بھی آسان اور فاسٹ کردیتی ہے۔ اررے میں ہمیں جس پوزیشن کی ضرورت ہوتی ہے اس کو ہم سنگل انڈکس کی مدد سے directly locate کر لیتے ہیں، جہاں ہم ڈیٹا Add یا Remove کرنا چاہتے ہیں۔ اررے اس لئے بھی بہت اہم ہے کہ یہ لینگویج کا پارٹ ہے جبکہ ڈیٹا اسٹرکچر جیسے کہ ٹری، سٹیک اور Queue سی یا سی پلس پلس لینگویج کا پارٹ نہیں ہیں، یہ لینگویج کے Construct کہلاتے ہیں۔ جیسے کہ میں بتا چکا ہوں کہ ٹری، اسٹیک اور Queue ڈیٹا اسٹرکچر سی یا سی ++ کا پارٹ نہیں ہیں اس لئے پروگرامر ان کو Directly declare نہیں کر سکتا ہے۔ ہم ٹری یا اسٹیک کو پروگرام میں Declare نہیں کر سکتے ہیں، تاہم ہم اس اس ڈیٹا اسٹرکچر کے لئے کلاسز لکھ سکتے ہیں۔

اس کے بارعکس ہم ارے کو directly declare کر سکتے ہیں۔ کیونکہ ارے سی اور سی ++ کا پارٹ ہیں۔

ہم نے پوینٹرز کی مدد سے بھی ٹری built کی ہے، پوائنٹر کا استعمال میموری میں کچھ ٹائم لیتا ہے۔ جو Executable کوڈ ہوتا ہے وہ میموری میں نہیں آتا ہے اس کیلئے ایک ٹرم پیجنگ یا ورچوئل میموری ہے۔ جب پروگرام Execute ہوتا ہے تو اس کا کچھ حصہ میموری میں آتا ہے۔ اگر ہم نے پوینٹرز کا استعمال کر کے پروگرام کے مختلف حصوں میں جانا ہو تو پروگرام کے کوڈ کا کچھ حصہ لوڈنگ میں آیا گا اور کچھ حصہ ہو سکتا ہے کہ remove یا unloading میں آئے۔ پروگرام کے لوڈنگ اور unloading کوڈ کو Execute کرنے کیلئے ایک میکانزم ہوتا ہے جس کو پیجنگ کہا جاتا ہے۔

ونڈوز آپریٹنگ سسٹم میں ورچوئل میموری یعنی پیجنگ میکانزم کو فائل استعمال کرتے ہیں جس کو فائل پیجنگ کہا جاتا ہے۔ پوینٹرز کے استعمال سے ہو سکتا ہے کہ پیجنگ کا پروس بڑھ جائے۔ جس سے ہو سکتا ہے کہ پروگرام سلوی Execute ہو۔

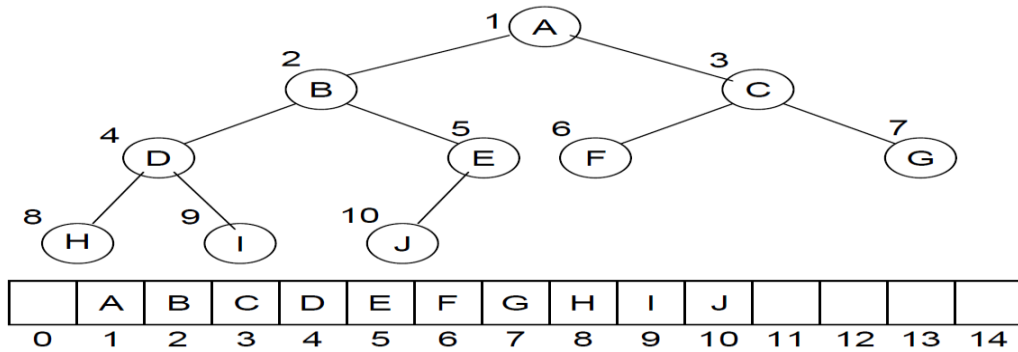
اس لئے ہمیں ارے کا استعمال کرنا چاہئے جہاں جہاں ارے سے ہماری ضرورت پوری ہو سکتی ہے۔ ارے بہت فاسٹ اور Efficient ڈیٹا سٹرکچر ہے اور کمپائلر بھی اس کو سپورٹ کرتا ہے۔ مگر ہمارے پاس کچھ Situations ایسی بھی ہیں جہاں

پوینٹرز کے استعمال سے زیادہ فائدہ حاصل ہوتا ہے، AVL ٹری کا بیلنس کرنا پوینٹرز کے استعمال کا ایک بڑا فائدہ ہے۔ اگر ہم ٹری کو بیلنس کرنے کیلئے اررے کا استعمال کرے تو ہم بہت زیادہ ڈیٹا دھرا دھرا کرنا پڑے گا۔ اس لئے AVL کو بیلنس کرنے کیلئے پوینٹرز کا استعمال زیادہ فائدہ مند ثابت ہوتا ہے۔

نوٹ: لیول آرڈر ٹراورسل کو ہم Queue کے استعمال سے بھی کر سکتے ہیں اور جبکہ ان، پری اور پوسٹ آرڈر ٹراورسل ہم نے Recursion کے ساتھ کی ہے اسٹیک کا استعمال کرے کے۔

## لیکچر نمبر 29

کمپلیٹ بائنری ٹری کو سٹور کرنے کا فارمولا یاد رکھیں، جو کہ ہے  $2i$  اور  $2i+1$ ۔



اس ٹری میں ہم نے روٹ نوڈ A نمبر ایک پر سٹور کیا تھا اور پھر فارمولا لگا کر باقی کے

نوڈسٹور کے تھے۔

## Heap

ہیب ایک ڈیٹا سٹرکچر ہے اور اس کے استعمال کے بہت فائدے ہیں۔ ہیب کا استعمال Priority Queue میں کیا جاتا ہے۔ جیسے کے آپ سب بنک جاتے ہوں گے تو وہاں پر ہمیشہ Queue ہوتی ہے، جس میں لوگ لائن میں کھڑے ہو کر اپنے کام کرواتے ہیں، Queue میں جو سب سے پہلے کھڑا ہوتا ہے وہ اپنا کام کروا کر Queue سے نکل جاتا ہے اور پھر دوسرا شخص Queue میں آ کر اپنا کام کروا لیتا ہے۔ اس کا مطلب ہے کہ Queue میں فرنٹ اینڈ سے لوگ نکلتے ہیں اور بیک اینڈ سے دخل ہوتے ہیں۔ بنک میں جو Queue استعمال ہوتی ہے اس کو ایونٹ بیس Queue کہتے ہیں۔ مگر جب ہم لوگ Priority Queue کا استعمال کرتے ہیں تو پھر ہماری Queue میں فسٹ ان فسٹ آؤٹ والا رول Apply نہیں ہوتا ہے۔ ہم اررے کے ساتھ Priority Queue استعمال کر چکے ہیں جس سے ہماری ضرورت بھی پوری ہو گئی تھی مگر اررے کے ساتھ Priority Queue کے استعمال سے مثلاً یہ ہوتا ہے کہ جب اررے میں ڈیٹا

remove یا add کرنا ہو تو کافی ٹائم لگتا ہے۔ اس اس مسئلے کو ختم کرنے کیلئے ہم اررے کی بجائے ہیپ کا استعمال کرے گے۔

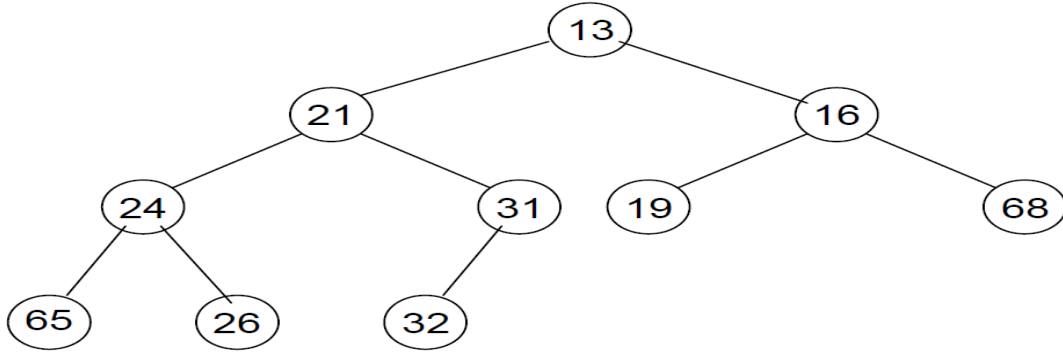
Priority Queue کا استعمال آپریٹنگ سسٹم میں بھی کیا جاتا ہے اور نیٹ ورک ڈیوائس جیسے کہ روٹرز میں بھی Priority Queue کا استعمال کیا جاتا ہے۔

”The definition of heap is that it is a complete binary tree that conforms to the heaporder”.

ہیپ کی تعریف میں ایک ٹرم استعمال ہوئی ہے جس کو ہیپ آرڈر کہا گیا ہے۔ ہیپ آرڈر ایک پراپرٹی ہے جو کہتی ہے کہ ہر نوڈ کو کم سے کم کتنی ہیپ چاہئے۔ جو Key parent میں ہوتی ہے وہ چھوٹی یا پھر برابر ہوتی ہے X کی Key کے۔ دوسرے الفاظ میں ہم یہ کہہ سکتے ہیں کہ parent نوڈ کی Key اس کے چلڈرن سے چھوٹی یا پھر اس کے برابر ہوتی ہے۔ ہیپ کو ہم کمپلیٹ بائنری ٹری کے ساتھ Implement کرتے ہیں۔ ہیپ کو دوسرے Methods کے ساتھ بھی استعمال کیا جاسکتا ہے مگر عام طور پر ہم ہیپ کو کمپلیٹ بائنری ٹری کے ساتھ استعمال کرتے ہیں۔

ہیپ کی دو قسمیں ہیں ایک MIN Heap اور دوسرا Max Heap۔ ان دونوں میں فرق یہ ہے کہ Min ہیپ میں روٹ کی ویلیو اس کے رائٹ لفٹ چلڈ سے کم ہوگی اور

Max ہیپ میں روٹ نوڈ یا parent نوڈ کی ویلیو اس کے لفٹ رائٹ چلڈ سے زیادہ ہوگی۔

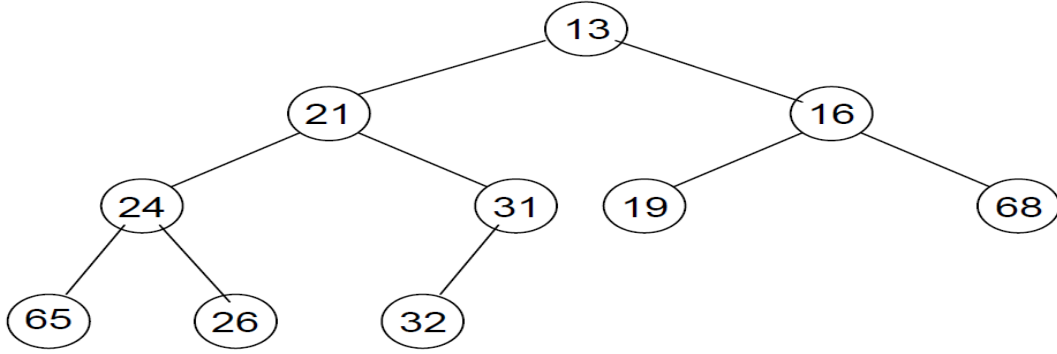


یہ Min ہیپ کی ایک مثال ہے اس میں ہم دیکھ سکتے ہیں کہ parent نوڈ کی ویلیو 13 ہے جو کہ اس کے لفٹ رائٹ چلڈ سے کم ہے، اور اسی طرح 24 اور 31 کا جو parent نوڈ ہے وہ 21 اور اس کی بھی ویلیو اس کے چلڈ سے کم ہے۔ اسی طرح نوڈ 65

اور نوڈ 26 کا parent نوڈ 24 ہے اور اسکی ویلیو بھی اپنے رائٹ لفٹ چلڈ سے کم ہے۔ اگر ہم روٹ نوڈ کے رائٹ سائیڈ میں دیکھیں تو ہمیں ایک اور سب ٹری نظر آئی گی۔ اور اس سب ٹری میں روٹ نوڈ 16 ہے جس کی ویلیو اس کے چلڈ سے کم ہے۔

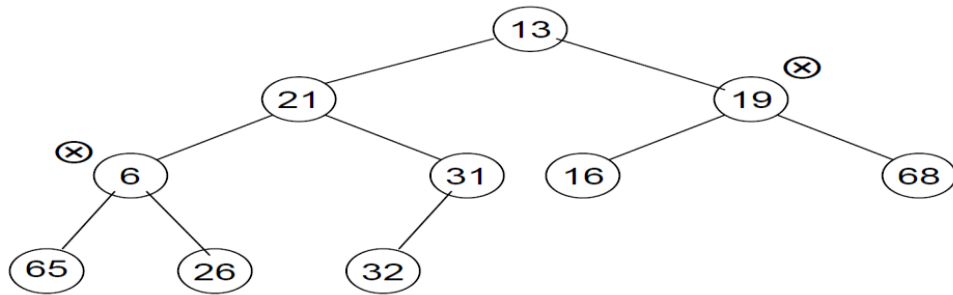
اس کو ہم Min ہیپ کہتے ہیں اور Max ہیپ میں parent نوڈ کی ویلیو زیادہ ہوتی ہے۔ اب ٹری سے تین چیزیں ثابت ہو گئی ہیں، پہلی کہ یہ ایک بائنری ٹری ہے، دوسری چیز کہ یہ ہے کہ یہ ایک کمپلیٹ بائنری ٹری ہے اور تیسری یہ کہ یہ ٹری ہیپ کی آرڈر

پراپرٹی کے رول کو Fullfill کرتی ہے . Min ہیپ میں ٹری کا روٹ وہ نوڈ بنے گا جس کی ویلیو سب سے کم ہوگی .



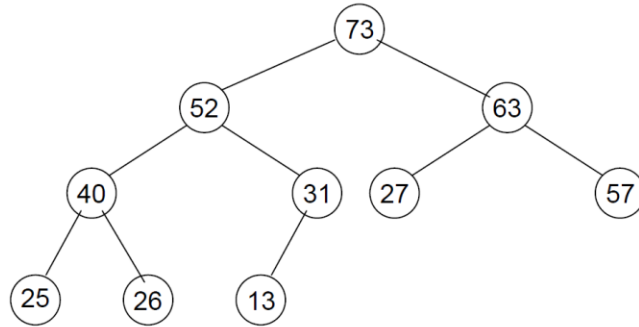
ہم اس Min ہیپ میں دیکھ سکتے ہیں کہ نوڈ 13 کی ویلیو سب سے کم ہے، اس لئے نوڈ 13 اس ٹری کا روٹ نوڈ بن گیا ہے .

اب ایک ٹری میں آپ لوگوں کو پریکٹس کیلئے دے رہا ہوں جس میں آپ لوگوں نے یہ بتانا ہے کہ کیا یہ بائنری ٹری ہے؟ کیا یہ کمپلیٹ بائنری ٹری ہے؟ کیا یہ Min ہیپ کی آرڈر پراپرٹی کے رول کو پورا کرتی ہے؟ ٹری دیکھ کر ان تین سوالوں کے جواب نکالیں .



# MaxHeap

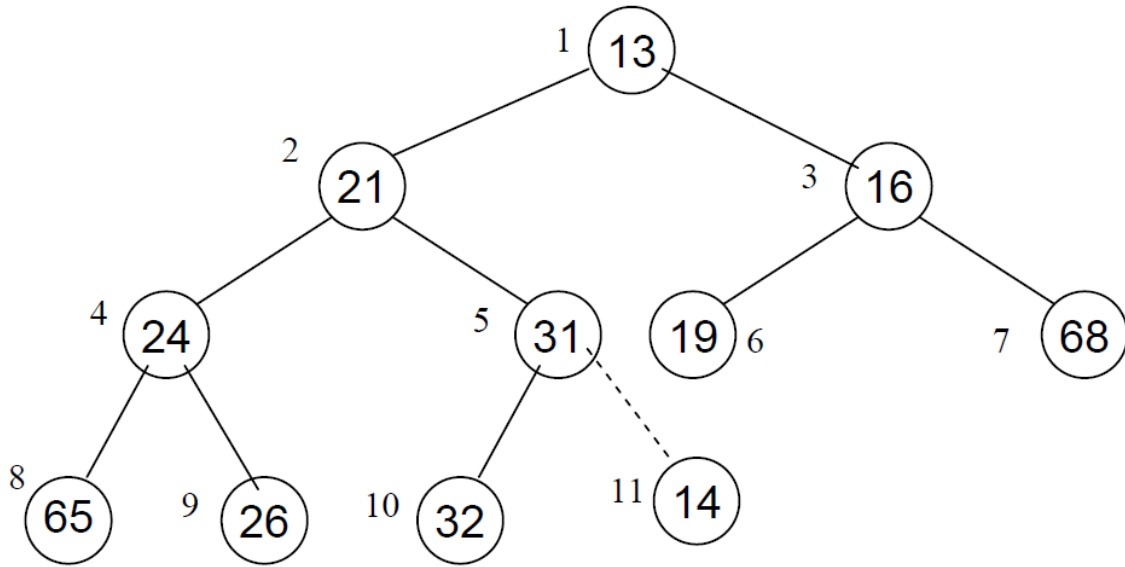
جیسے کہ میں اس سے پہلے بھی بتا چکا ہوں کہ میکس ہیپ میں روٹ کی ویلیو اس کے لفٹ اور رائٹ چلڈ سے بڑی ہوتی ہے۔ روٹ نوڈ کی ویلیو ٹری میں سب سے بڑی ویلیو ہوگی اور سب سے نیچے چھوٹی ویلیو آئی گی۔ آئیں میکس ہیپ کو ایک ٹری کی مثال سے سمجھنے کی کوشش کرتے ہیں۔



یہ میکس ہیپ ٹری ہے اس میں ہم دیکھ سکتے ہیں کہ روٹ کی ویلیو سب سے بڑی ہے جو کہ 73 ہے اور اس کے لفٹ اور رائٹ سب ٹری کی ویلیوز parent نوڈ سے کم ہیں۔ اور اسی طرح اگر ہم اس ٹری کے لفٹ پر دیکھیں تو ایک اور سب ٹری ہے جس کا parent نوڈ 52 ہے اور اس کے لفٹ رائٹ چلڈ کے ویلیوز parent سے کم ہیں۔ اگر ہم نوڈ 40 کو دیکھیں تو اس کی ویلیو اس کے لفٹ رائٹ چلڈ سے بڑی ہے۔ اس ٹری سے بھی تین چیزیں پروف ہوئی ہیں۔ پہلی: یہ ایک بائری ٹری ہے۔ دوسری: یہ ایک

کمپلیٹ بائنری ٹری ہے۔ تیسری: یہ ایک میکس ہیپ ٹری ہے کیونکہ یہ میکس ہیپ کے پروپٹی آرڈر کو فل Fill کرتی ہے۔

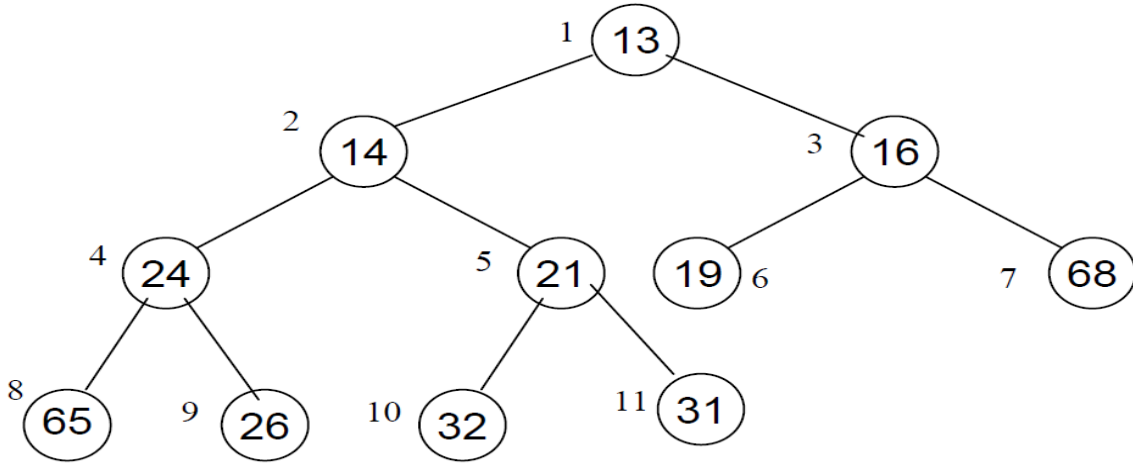
ایک اور ٹری پریکٹس کیلئے دے رہا ہوں تاکہ آپ لوگ کو اچھی طرح سے Min اینڈ میکس ہیپ یاد ہو سکے۔ اس ٹری میں آپ لوگوں نے غلطی بتانی ہے اور پھر اسے ٹھیک کر کے اس ٹری کو ہیپ پر اپنی آرڈر کے رول کے مطابق سہی کرنا ہے۔



# لیکچر نمبر 30

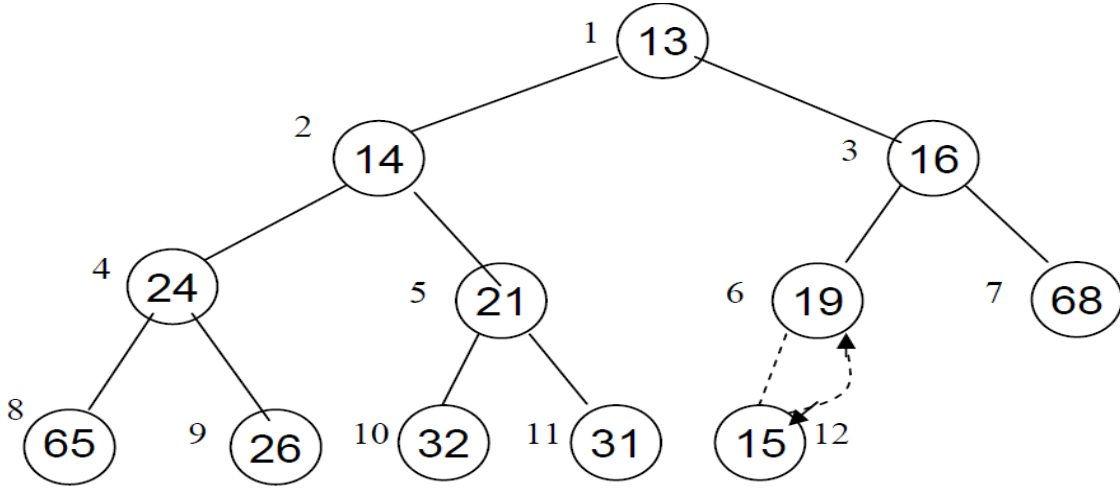
Min ہیپ اور میکس ہیپ میں نوڈ انسرت کرنا: Inserting into a Min-Heap

سب سے پہلے ہم Min ہیپ میں نوڈ انسرت کرے گے اور پھر دیکھیں گے کہ کیا نیا نوڈ انسرت ہونے کے بعد ٹری ہیپ ٹری کے پراپرٹی رول پر پورا اترتی ہے یا نہیں۔

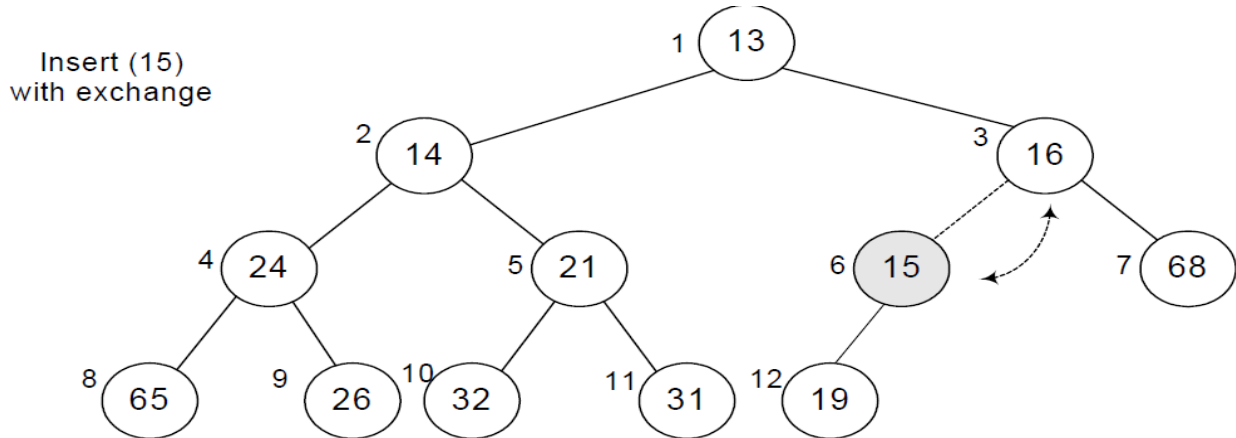


تو ہمارے پاس یہ ایک کمپلیٹ بائنری ٹری ہے یہ Min ہیپ ٹری بھی ہے کیونکہ اس کے parent نوڈز اپنے چلڈ نوڈز سے چھوٹے ہیں۔ اب اگر ہم اس میں ایک اور ویلیو 15 کو add کرنا چاہیں تو کہاں add کریں گے؟ تو اس کا جواب ہے کہ 19 کے لفٹ میں اب نیا نوڈ 15 ہوگا وہ اس لئے کہ روٹ یعنی parent کی لفٹ ٹری پہلے ہی

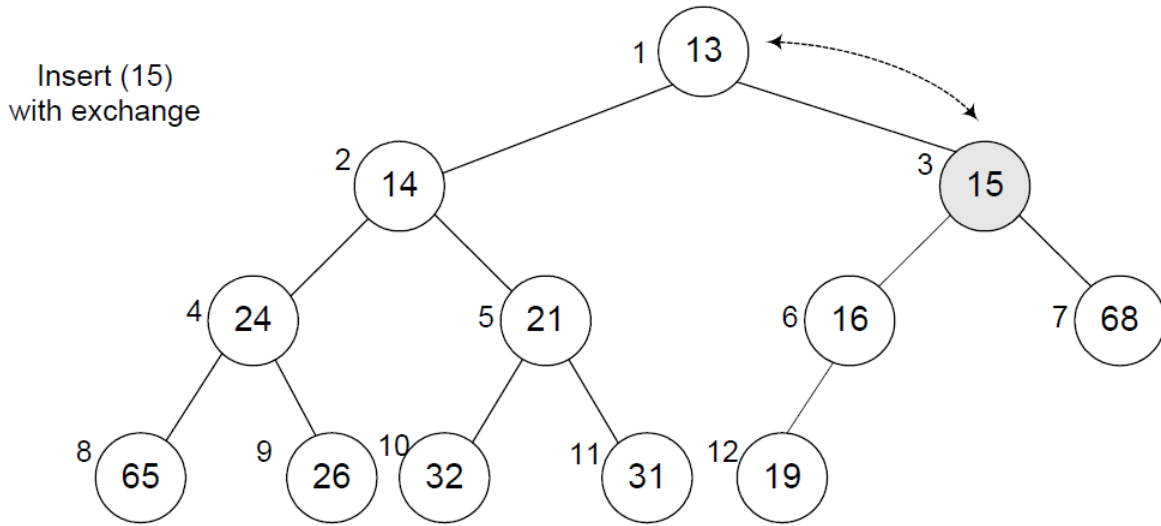
کمپلیٹ ہو چکی ہے اس لئے اب ہم نوڈ یعنی چلڈ 15 کو 19 کے لفٹ میں add کر کے دیکھیں گے کہ کیا یہ ٹری Min ہیپ کے پراپرٹی رول کو فل fill کرتی ہے یا نہیں۔



تو نوڈ 15 add ہونے کے بعد اگر اس ٹری کو دیکھا جائے تو یہ اب Min ہیپ ٹری نہیں ہے کیونکہ Min ٹری کی کنڈیشن یہ تھی کہ parent نوڈ کی ویلیو اپنے چلڈ کی ویلیو سے چھوٹی ہوگی، اب نوڈ 19 جو کے parent ہے اس کی ویلیو نوڈ 15 سے بڑی ہے اس لئے اب ہم اس نوڈ کو swap کر کے اس کو پھر سے Min ٹری بنا دے گے۔



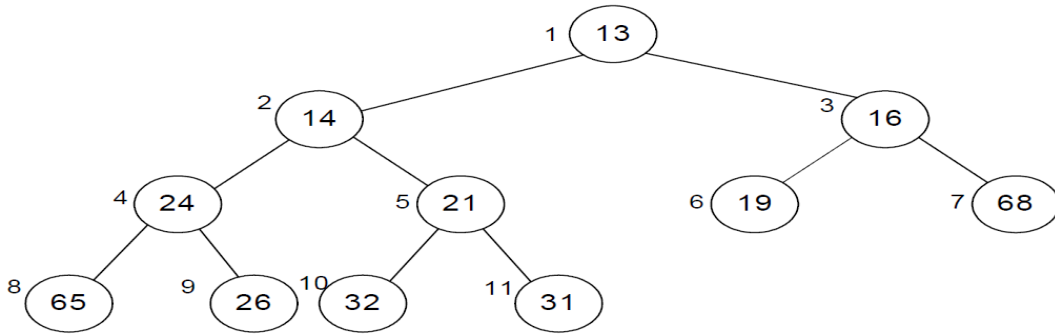
اب 15 جو کے parent نوڈ ہے 19 کا اس کی ویلیو اپنے چلڈ سے چھوٹی ہو گئی ہے مگر ابھی بھی یہ ٹری Min ہیپ کی کنڈیشن کو پورا نہیں کر رہی ہے وہ اس لئے کہ نوڈ 15 کا جو parent نوڈ ہے وہ 16 اور 16 کی ویلیو اپنے چلڈ سے چھوٹی ہونے کی بجائے بڑی ہے اس لئے اب یہاں بھی ہم نوڈ کو Swap کرے گے۔



اب ہمارے پاس Min ہیپ کی ٹری آگئی ہے کیونکہ یہ ٹری Min ہیپ کی کنڈیشن کو پورا کرتی ہے۔ ایک بات کا خیال رکھیں جیسے کے آپ جانتے ہیں کہ ہم ہیپ کو ارے کے ساتھ سٹور کرتے ہیں تو جب نوڈ Swap ہوتے ہیں تو ارے میں انڈکس میں انکی جگہ بھی تبدیل ہوگی۔

## Deleting from a Min-Heap (deleteMin):

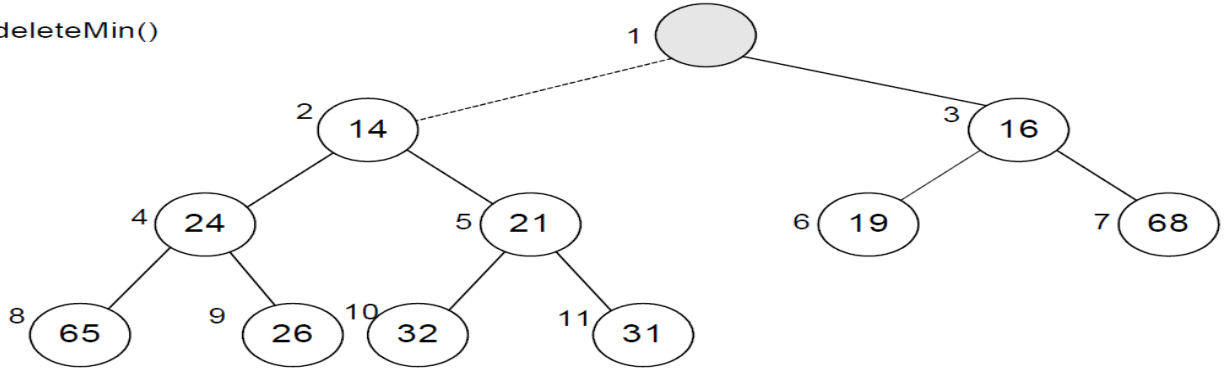
اب ہم Min ہیپ کا استعمال کر کے نوڈ کو اس سے ڈیلیٹ کرنے کے میٹھڈ کو سمجھنے کی کوشش کرتے ہیں۔ Min ہیپ سے نوڈ ڈیلیٹ کرنے کیلئے ہم ایک فنکشن (DeleteMin) لکھیں گے۔ یہ فنکشن Min ہیپ میں سب سے چھوٹا نمبر remove کرے گا۔ Min ہیپ میں سب سے چھوٹا نمبر تلاش کرنا بہت آسان ہے، کیونکہ سب سے چھوٹا نمبر Min ہیپ کے سب سے ٹاپ پر ہوتا ہے۔



اب اس ٹری سے ہم MIN یعنی سب سے چھوٹا نوڈ ڈیلیٹ کریں گے، اس ٹری کا سب سے چھوٹا نوڈ اسی ٹری کا روٹ نوڈ نمبر 13 ہے۔ Min ہیپ میں نوڈ Add کرنے یا remove کرنے سے ہول کا مسئلہ پیدا جس کو نل کرنا ضروری ہے۔

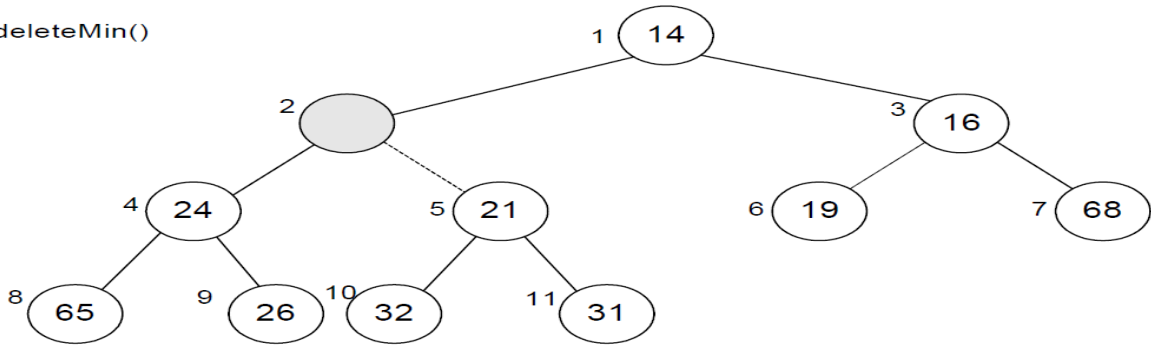
Deletion (or removal) causes a hole which needs to be filled.

deleteMin()



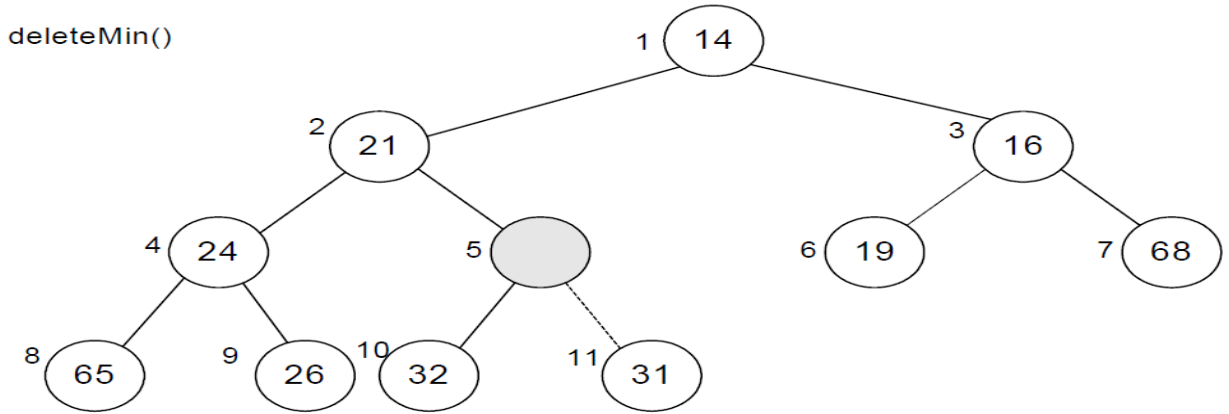
سب سے چھوٹا نمبر ڈیلیٹ ہونے کے بعد اب اس ٹری میں ایک ہول بن گیا ہے جس کو اب فل کرنا ضروری ہے۔ اب اس ہول کو فل کرنے کیلئے ہم اس ٹری میں سب سے چھوٹا نمبر دیکھیں گے اور MIN ہیپ کی definition کے مطابق اس کو اس ٹری کا روٹ نوڈ یعنی parent بنادے گے۔ اس ٹری کا اگلا سب سے چھوٹا نمبر 14 ہے، ایک بات کا خاص خیال رکھیں کہ نوڈ 14 کو روٹ نوڈ بنانے سے ایک اور ہول پیدا ہو جائے گا اب ایسا نہیں کہ آپ لوگ ایک ہول فل کر کر دوسرا چھوڑ دیں، اب ہمیں نوڈ 14 کی جگہ پیدا ہونے والے ہول کو بھی فل کرنا ہوگا۔

deleteMin()

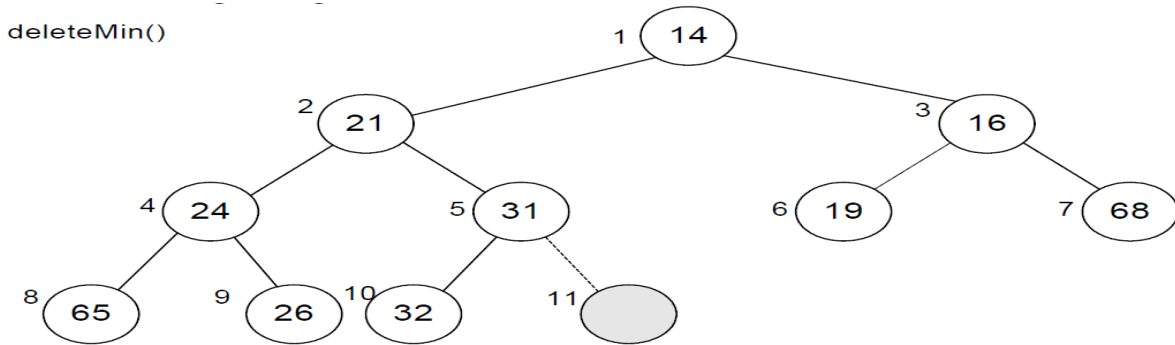


اب نمبر 14 کے رائٹ لفٹ سب ٹری کو ہم Compare کرے گے اور جو سب سے چھوٹا نوڈ ہوگا اس کو روٹ نوڈ بنادے گے۔ ہمارے کیس میں نوڈ یعنی چلڈ 21 سب سے

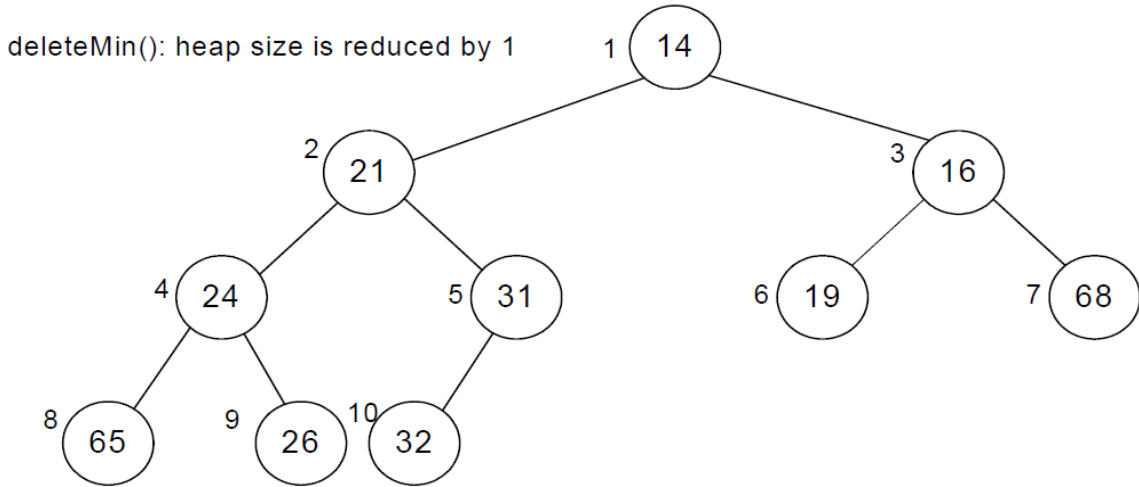
چھوٹا نوڈ ہے اس لئے اب یہ نوڈ 14 کی جگہ کو فل کر دے گا۔ پھر سے ایک اور ہول بن جائے گا کیونکہ ہم نے نوڈ 21 کو اوپر Move کروادیا ہے۔ اب پھر سے اس ہول کو فل کرنا ہوگا۔



اب اس ہول کو فل کرنے کیلئے ہم پھر سے رائٹ لفٹ سب ٹری میں سب سے چھوٹا نوڈ دیکھنا ہوگا۔ اب 31 اور 32 میں سب سے چھوٹا نوڈ 31 بنتا ہے، اس لئے اس کو ہم اب اوپر Move کروادیں گے۔ اب جو ہول سب سے اوپر بنا تھا وہ سب سے نیچے آ چکا ہے۔



اب سوال یہ پیدا ہوتا ہے کہ اب اس ہول کا کیا جائے؟ اب یہ ایک ایسی جگہ پہنچ چکا ہے جہاں سے اس کو آسانی سے ڈیلیٹ کیا جاسکتا ہے۔ اس ہول کو ڈیلیٹ کرنے بعد ہماری ٹری کا فائنل آؤٹ پٹ کچھ یوں آئے گا۔



## Building a Heap (buildHeap)

اب ہم دیکھتے ہیں کہ ہم ہیپ کو کیسے اور کن شرائط پر Built کر سکتے ہیں۔ ویسے تو ہم نے Min ہیپ اور Max ہیپ بنانا سیکھ لیا ہے مگر ہینڈ آؤٹ میں یہ بھی ایک topic ہے تو سوچا اس کو بھی ایک بار دیکھا جائے۔ پیپر میں ہیپ بنانے کیلئے نمبر یا پھر الفابیٹ دیئے جائے گے اور یہ بھی کہا جائے گا کہ Min ہیپ یا MAX ہیپ کی کنڈیشن Apply کر کے ٹری بنائے۔ Min میں سب سے چھوٹا اور میکس ہیپ میں سب سے بڑا نمبر روٹ یعنی parent نوڈ بنے۔ MIN یا میکس ہیپ سے عام طور پر ڈیٹا سورت کرنے کا کہا جاسکتا۔ تو اس لئے confuse نہیں ہونا۔

The worst time for building a heap of N keys could be

$N \log_2 N$ .

پیپر میں ہو سکتا ہے کے آپکو کچھ اس قسم کا ڈیٹا دیا جائے اور کہا جائے کہ MIN یا میکس ہیپ سے اس ڈیٹا کو سورٹ کر کے دیں۔

|   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|   | 65 | 31 | 32 | 26 | 21 | 19 | 68 | 13 | 24 | 15 | 14 | 16 | 5  | 70 | 12 |
| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |

اب اس کو سورٹ کر کے ٹری بنانی ہوگی۔ اس کو آپ لوگ خود حل کریں اگر سمجھ میں نہ آئے تو ہینڈ آؤٹ یا مجھ سے رابطہ کریں۔

## Lecture No. 31

MIN یا میکس ہیپ میں کوئی بھی نوڈ ڈیلیٹ کرنے کیلئے *percolateDown* کا میٹھڈ بہت مدد دے سکتا ہے۔ *percolateDown* کے کچھ رولز ہوتے ہیں جن کو مائنڈ میں رکھنا بھی ضروری ہے۔ Min ہیپ میں *PercolateUp* ہوتا ہے جبکہ میکس ہیپ میں *PercolateDown* ہوتا ہے۔

- The general algorithm is to place the N keys in an array and consider it to be an unordered binary tree.
- The following algorithm will build a heap out of N keys.

```
for ( i = N/2; i > 0; i-- )
    percolateDown(i);
```

### ***Decrease Key(p, delta)***

یہ میٹھڈ پوائنٹر لیتا ہے اس نوڈ تک جو آررے پوزیشن سٹور ہوتا ہے۔ User نوڈ کی ویلیو کو Decrease کرنا چاہتا ہے ڈیٹا کے ساتھ۔ اگر ہمارے پاس ایک نوڈ 17 ہو اور ہم اسکی ویلیو کو کم کر کے دس کرنا چاہتے ہیں تو اس اس کی نئی ویلیو 10 ہوگی۔ اب نوڈ کی ویلیو Decrease کرنے سے ہیپ کا آرڈر Violate ہو جائے گا۔ اگر ہیپ کا آرڈر خراب ہو جائے تو ہمیں ہیپ کا آرڈر ٹھیک کرنا ہوگا۔ ہیپ آرڈر کو maintain کرنے کیلئے ہو سکتا ہے کہ ہمیں پوری ٹری پر کام کرنا پڑے۔ اس لئے ہیپ آرڈر کو Maintain رکھنے کیلئے ہم PercolateUP اور PercolateDown کے میٹھڈ کا استعمال کرتے ہیں۔ ہیپ کا استعمال ہم زیادہ تر Priority Queue میں کرتے ہیں۔ Priority Queue میں LIFO اور FIFO کا رول لاگو نہیں ہوتا ہے۔

Task manager can be activated by pressing Ctrl + Alt + Del.

## ***IncreaseKey (p, delta)***

یہ میتھڈ DecreaseKey کے الٹ ہے۔ یہ ایلیمینٹ کی ویلیو کو ڈیلٹا کے ساتھ Increase کر دے گا۔ یہ میتھڈ بہت مفید ہوتے ہیں جب Priority Queue کو ہیپ کے ساتھ استعمال کیا جائے۔

## ***Remove(p)***

یہ میتھڈ ہیپ سے نوڈ کو جو پوزیشن P یعنی Parent پر ہوتا ہے اسکو Remove کر دیتا ہے۔ یہ میتھڈ سب سے پہلے  $decreaseKey(p, \infty)$  اور پھر  $DeleteMin()$  پر فارم کرے گا۔ سب سے پہلے نوڈ کی ویلیو کو ڈیلٹا کے ساتھ Decrease کرے گا اور پھر  $DeleteMin$  کے میتھڈ کو کال کرے گا۔  $DeleteMin$  کا میتھڈ ٹری کی سب سے کم ویلیو جو کہ روٹ ہوتی ہے اس کو ڈیلیٹ کر دے گا۔ اگر Min ہیپ کا استعمال ہو رہا ہے تو Next سب سے کم ویلیو والا نوڈ روٹ نوڈ بن جائے گا۔ نوڈ ڈیلیٹ کرنے کے بعد ہم PercolateDown کا میتھڈ استعمال کر کے ہیپ آرڈر کوریسٹور کرے گے۔ ہم ٹری سے کوئی بھی نوڈ کو ڈیلیٹ کر سکتے ہیں، ہم اس کام کیلئے خاص Prcedure بھی لکھ سکتے ہیں۔ اور پہلے سے موجود میتھڈ کا بھی استعمال کر کے نوڈ کو ڈیلیٹ کر سکتے ہیں۔ سب سے پہلے DecreaseKey کا میتھڈ کال ہوگا اور نوڈ کی ویلیو ڈیلٹا سے Decrease ہو جائے گی۔ اس میتھڈ سے نوڈ کی ویلیو سب سے کم ہو جائے گی۔ اور کم ویلیو والا نوڈ روٹ نوڈ بن جائے گا۔ پھر ہم DeleteMIN کے میتھڈ کو کال کر کے نوڈ کو ڈیلیٹ کر دے گے۔

ڈیٹا ایک Mathematical نوٹیشن ہے اور یہ C++ میں دستیات نہیں ہے۔ ہم ہیپ میں ہر قسم کا ڈیٹا سٹور کر سکتے ہیں جیسے کے Integers، سٹرنگز، فلوٹنگ پوائنٹ نمبر اور دوسرے ٹائپ کا ڈیٹا۔ اس مقصد کیلئے ہم ٹیمپلیٹ کا استعمال کرتے ہیں۔ ٹیمپلیٹ کی مدد سے ہم کسی بھی قسم کا آبجیکٹ ہیپ میں سٹور کر سکتے ہیں۔

### **GetSize ()**

کیا ہیپ فل ہو سکتا ہے؟ جیسے کے آپ سب جانتے ہیں کہ ہیپ کا استعمال ہم ارے ڈیٹا ٹائپ کے ساتھ کرتے ہیں جو کے فکسڈ ڈیٹا سائز ہوتی ہے اسلئے ہیپ بھی فل ہو سکتا ہے۔ اب کیسے پتا چلے کہ ہیپ فل ہے یا نہیں؟ ہم () getSize کے میتھڈ کو کال کرے گے تاکہ معلوم کر سکے کہ ہیپ کا سائز کتنا ہے۔ کیا ہیپ فل ہو گیا یا نہیں۔

### **Insert Method**

جب ہیپ کا آرڈر maintain کرنا ہو تو انسرت میتھڈ ہیپ میں ایٹم انسرت کرنے کے کام آتا ہے۔ ڈپلیکٹ بھی Allow ہوتے ہیں۔ انسرت میتھڈ کے اندر ہم () isFull کے میتھڈ کو کال کرے گے۔ اگر ہیپ فل ہو تو میسج ڈسپلے ہو گا اور 0 ریٹرن ہو جائیگا اور اگر ہیپ فل نہ ہو تو ایلیمینٹ ہیپ میں انسرت ہو جائے گا اور variable ہول لے گا اور currentSize پلس ون ویلیو assign ہو جائے گی۔

اب اگلا مرحلہ deleteMin کے میتھڈ کو کال کرنا ہے، سب سے پہلے یہ isEmpty میتھڈ کو کال کرے گا یہ دیکھنے کیلئے ہیپ خالی ہے یا نہیں۔ اگر ہیپ خالی ہو تو میسج ڈسپلے کرے گا اور ریٹرن آجائے گا۔ اگر ہیپ خالی نہیں ہو تو یہ نوڈ کو ڈیلیٹ کر دے گا۔

# Lecture No. 32

## PercolateDown Method

یہ میتھڈ اررے انڈکس کو آرگمنٹ کے طور پر لیتا ہے اور اپنی Functionality کو شروع کرتا ہے۔ ہم نوڈ میں اررے انڈکس کو ہول کا نام دیتے ہیں۔  
2i (or i \* 2) formula کا فارمولا انڈکس کے پوزیشن کو چلڈ کے لفٹ سائیڈ پر کر دیتی ہے۔

## getMin Method

اس میتھڈ سے ہیپ میں سب سے کم ویلیو Determin ہوتی ہے۔ یہ میتھڈ ہمیں ہیپ میں سب سے چھوٹی ویلیو بھی فراہم کرتا ہے جو کہ ہیپ سے ڈیلیٹ نہیں کرنی ہوتی ہے۔ اس میتھڈ کو لکھنے کیلئے نوڈ کو ایک بار دیکھ لیں۔

```
template <class eType>
const eType& Heap<eType>::getMin( )
{
    if( !isEmpty( ) )
        return array[ 1 ];
}
```

# BuildHeap Method

یہ میتھڈ ارے کو اس کے سائز کے ساتھ آرگمنٹ کے طور پر لیتا ہے اور پھر ہیپ کو بلڈ کرتا ہے۔ بلڈ ہیپ Linear ٹائم Algorithm ہے جبکہ  $\log_2 N$  لینتیر نیچر کا نہیں ہے۔

## BuildHeap in Linear Time

بلڈ ہیپ ارے سے ہیپ کو بلڈ کرتا ہے۔ بلڈ ہیپ Algorithm زیادہ اچھا کام کرتا ہے  $N \log_2 N$  کی بجائے۔ بلڈ ہیپ Linear ٹائم Algorithm ہے جبکہ  $\log_2 N$  لینتیر نیچر کا نہیں ہے۔ لینتیر Algorithm ایک ایسی چیز ہے کہ اگر ہمارا گراف ڈرا کریں تو یہ ایک سیدھی لائن ہوگی جو سلوپ کے ساتھ ہوگی۔ جبکہ نان لینتیر Algorithm کا گراف Curve کی طرح ہوتا ہے۔

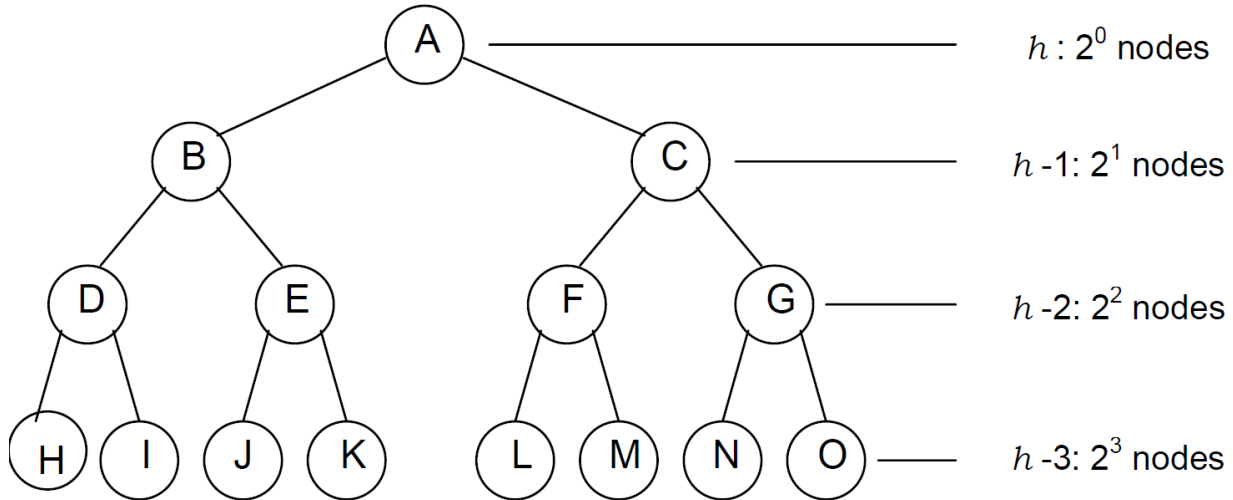
## Theorem:

تھیورم کے مطابق "پرفیکٹ بائنری ٹری کی ہائٹ  $2^{h+1} - 1$  نوڈز ہوتے ہیں۔  $2^{h+1} - 1 - (h + 1)$ , or  $N - h - 1$  Sum کا  $2^{h+1} - 1$  ہوگا۔ یہ تھیورم ٹری کی ہائٹ کے بارے میں ہے۔ ہم پہلے بھی ٹری کی ہائٹ کو سمجھ چکے ہیں مگر ایک بار پھر سے review کر لیتے ہیں۔ اگر روٹ نوڈ سے شروع کرے اور ٹری کے سب سے نچلے لیف نوڈ تک جائے تو جو لنکس ہم نے روٹ نوڈ سے لیکر نچلے لیف نوڈ تک پاس کیے ہوتے

ہیں اس کو ٹری کی ہائٹ کہا جاتا ہے۔ ایک پرفیکٹ بائنری ٹری میں ہائٹ ایچ کے ٹوٹل

$$\text{نوڈز} = 2^{h+1} - 1$$

آئیں پرفیکٹ ٹری کی ایک مثال دیکھتے ہیں۔



یہ پرفیکٹ ٹری ہے اس کے لفٹ رائٹ سب ٹری کمپلیٹ ہیں۔ ایسی ٹری کو ہم پرفیکٹ بائنری ٹری کہتے ہیں۔

# Lecture No. 33

## Priority Queue Using Heap:

ہم عام طور بلڈ ہیپ کا استعمال کرتے ہیں ہیپ کو Construct کرنے کیلئے، اگر ہمارے پاس Required ڈیٹا ہو تو ہم انسرت کا استعمال نہیں کرتے بلکہ بلڈ ہیپ کا استعمال کرتے ہیں۔ بلڈ ہیپ زیادہ Optimized ہے انسرت میٹھڈ کے مقابلے میں۔

## Disjoint Set ADT:

ADT کا مطلب ہے Abstract ڈیٹا ٹائپ۔ اے ڈی ٹی کیا چیز ہے اور یہ کام کیسے کرتی ہے؟ مثال کے طور پر ہمارے پاس لوگوں کا ڈیٹا بیس ہے، اور ہم یہ دیکھنا چاہتے ہیں کہ اس ڈیٹا بیس میں لوگوں کا ایک دوسرے سے تعلق کیا ہے، ہمارے پاس صرف لوگوں کی لسٹ ہے اور ان کے relation کی ڈیٹیل ہے جو فارم سے حاصل کی گئی ہے۔ مثلاً حارث کا تعلق سعد سے ہے، اب ان دونوں کا تعلق واضح نہیں ہے۔ جیسے جیسے ہمیں لوگوں کے relation کا معلوم ہوتا جائے گا ڈائریکٹ اور ان ڈائریکٹ relationship اسٹبلش کی جاسکے گی۔ اس کو ہم اس طرح سے بھی دیکھ سکتے ہیں کہ حارث سعد سے ریلاٹڈ ہے اور سعد احمد سے ریلاٹڈ ہے۔ جب ہمارے پاس relation کی انفارمیشن

ہوگی تو پھر اس قسم کے سوالوں کے جواب دینا بھی آسان ہو جائے گے، کیا حارث ریلاٹڈ احمد ہے؟؟

اس قسم کی Queries کو Answer کرنے کیلئے اور پروگرام کو انٹیلیجنٹ بنانے کیلئے ڈس جانٹ سیٹ اے ڈی ٹی استعمال ہوتی ہے۔

اے ڈی ٹی کی دوسری ایپلیکیشن امیج انالیسس ہے، اس پر اہلم کو Blob بلوب کالارنگ کے نام سے جانا جاتا ہے۔

## ***Blob Coloring***

یہ ایک ویل known لولیول کمپیوٹر ویژن پر اہلم ہے جو کہ بلیک اینڈ وائٹ امیجز کیلئے ہوتا ہے۔ یہ تصویر کے تمام ایلیمینٹ پکسل کو ایک ساتھ پٹ کر دیتا ہے جو ایک ہی بلوب سے تعلق رکھتے ہیں، اور ہر پکسل کو ہر ایک مختلف بلوب میں ایک identical لیبل دے دیتا ہے۔

کمپیوٹر ویژن کے ساتھ جو ایک اور پر اہلم ریلاٹڈ ہے وہ ہے امیج Segmentation پور اہلم کہلاتی ہے۔ CAT سکین یا سی ٹی سکین سے انسانی باڈی کے اندر کے تمام پارٹس کو دیکھا جاتا ہے، سی ٹی سکین انسانی باڈی کے امیجز گرے سکیل Organs پیش کرتی ہیں

کے۔ ڈس جائنٹ سیٹ اے ڈی ٹی کے اپلیکیشن میں ایم آر آئی، CAT سکین یا سی ٹی سکین آتے ہیں۔

## Lecture No. 34

یہ لیکچر مجھے کچھ خاص امپورٹنٹ نہیں لگا اس لئے اس کو اردو میں کنورٹ نہیں کیا ہے۔ مگر آپ ایک بار ہینڈ آؤٹ سے ضرور دیکھ لینا اس کو۔

## Lecture No. 35

ہم نے اس سے پہلے ڈس جائنٹ سیٹ کی بات کی تھی ایک بات کا خیال رکھیں ڈس جائنٹ سیٹ کی implementation کی بات کی جائے تو اس کا مطلب ہے کہ بولین میٹر کس کی بات ہو رہی ہے۔ بولین میٹر کس ٹو dimensional اسٹرکچر ہے۔ یونین کا استعمال اس وقت کیا جاتا ہے جب دو ایٹم آپس میں ریلیٹڈ ہوں۔ یونین clearly کانسٹنٹ ٹائم آپریشن ہے۔

سنگل Dimensional ارے VS ٹو Dimensional ارے :

اگر ہمارے پاس ہزار سیٹ ممبرز مطلب کے ہزار ایلیمینٹ ہوں یا لوگوں کے نام ہوں تو اگر ہم بولین میٹر کس ایک ہزار ایٹم کی بنائیں تو ٹو Dimensional ارے میں اسکا

سائز ایک ہزار ضرب ایک ہزار ہو جائے گا، اس لئے ہمیں دس لاکھ لوکیشن چاہئے ہوں گے بولین ویلیو کیلئے، جبکہ ارے کی صورت میں ہمیں صرف ایک ہزار لوکیشن ہے چاہئے ہوں گی۔ اس لئے ارے بہتر ہے تو Dimensional ارے سے کیونکہ اس سے Space کم استعمال ہوتی ہے۔ ارے کی صورت میں ہم پونینٹرز کا بھی استعمال نہیں کرتے ہیں۔

## Lecture No. 36

### Union by Size:

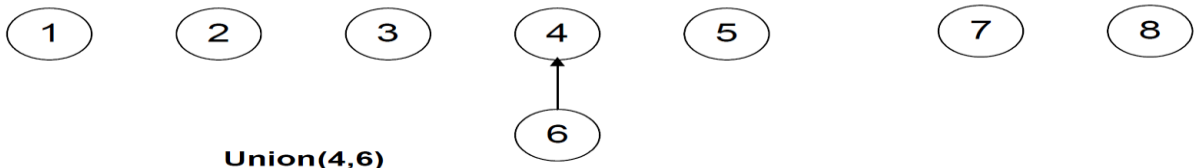
اس کے کچھ نمایاں خصوصیات ہیں !  
یہ ٹری کے تمام نوڈز کا سائز یونین کے دوران Maintain کرتا ہے۔  
یہ چھوٹی ٹری بناتا ہے، بڑی ٹری کی سب ٹری کی۔  
اس کو یونین By weight بھی کہا جاتا ہے۔  
نن نیگیٹو ویلیو کے لئے لوپ کو چیک کرنے کی کنڈیشن ہوتی ہے۔ کسی بھی پوانٹ پر جب parent کی ویلیو نیگیٹو ہو جائے گی لوپ Terminate ہو جائے گا۔ کنڈیشن کوئی خاص نیگیٹو نمبر کو Specify نہیں کرتا ہے کہ وہ نمبر آئے گا تو لوپ Terminate ہوگا۔ اس کا مطلب ہوا کہ اگر کسی ٹری میں دس نوڈز ہوئے تو نمبر دس نوڈ 10 - ہوگا

اور اسی پر لوپ Terminate ہوگا۔ جب دو ٹریز کو یونین آپریشن میں کنبائن کیا جائے گا تو چھوٹی ٹری بڑی سائز کی ٹری کا پارٹ بن جائے گی۔ اس سے ٹری کا سائز کم ہوگا اسی لئے اس کو Union-By-Size یا یونین-by-weight کہا جاتا ہے۔



|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |

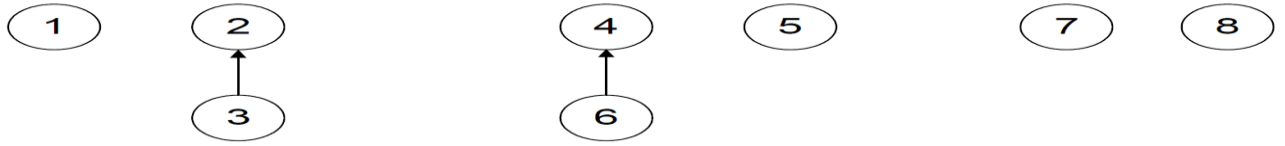
ہم اس سیٹ پر یونین کر کے دیکھتے ہیں کہ یونین کے بعد کیا تبدیلی آتی ہے اور ہمارے پاس ٹری کس قسم کی بن کے آتی ہے۔ سب سے پہلے ہم نوڈ 4 اور 6 کا یونین کرتے ہیں یونین کرتے ہی نوڈ 6 نوڈ 4 کے نیچے آ جائے گا یعنی نوڈ 6 نوڈ 4 کا چلڈ بن جائے گا اور جو ارے لسٹ ہے ہمارے پاس اس میں 6 کی انڈکس میں اب 4 لکھا جائے اور 4 کے انڈکس میں جہاں 1 تھا اب وہاں 2 ہو جائے۔



Union(4,6)

|    |    |    |    |    |   |    |    |
|----|----|----|----|----|---|----|----|
| -1 | -1 | -1 | -2 | -1 | 4 | -1 | -1 |
| 1  | 2  | 3  | 4  | 5  | 6 | 7  | 8  |

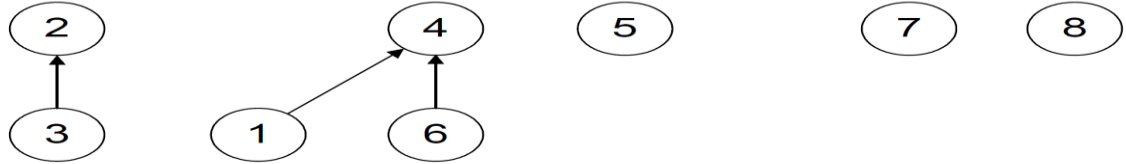
اب ہم نوڈ 2 اور 3 کا یونین کرتے ہیں۔ ان دونوں کا یونین کرتے ہی result وہی آئے گا جو نوڈ 4 اور 6 کا یونین کر کے آیا تھا۔



After union(2,3)

|    |    |   |    |    |   |    |    |
|----|----|---|----|----|---|----|----|
| -1 | -2 | 2 | -2 | -1 | 4 | -1 | -1 |
| 1  | 2  | 3 | 4  | 5  | 6 | 7  | 8  |

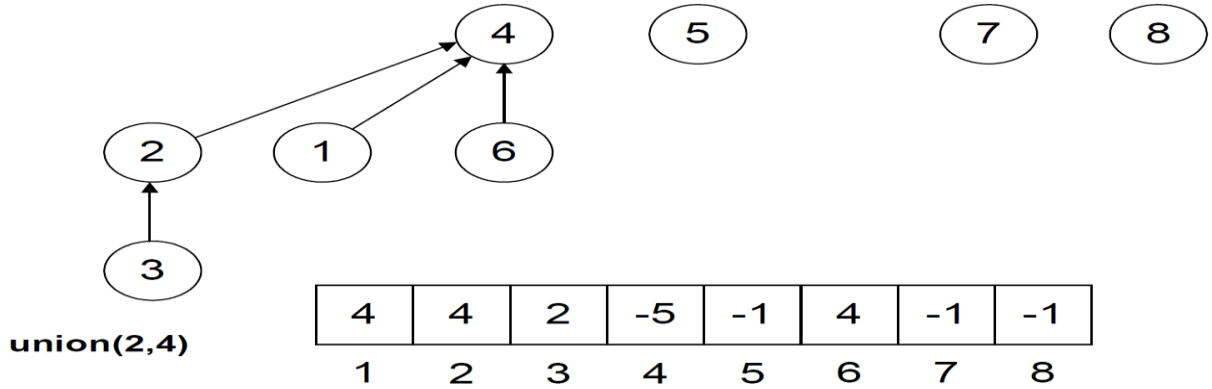
نوڈ 3 اب نوڈ 2 کا چلڈ نوڈ بن گیا ہے اور ارے انڈ کس میں جہاں 3 تھا اب وہاں 2 لکھا ہوا اور 2 کے انڈ کس میں جہاں 1 تھا اب وہاں -2 ہو گیا ہے۔ اب نوڈ ایک اور 4 پر یونین آپریشن کرے گے۔ نوڈ ایک سنگل نوڈ ہے جبکہ نوڈ 4 روٹ نوڈ ہے نوڈ 6 کا۔ جیسے کہ ہمیں پتا ہے کہ چھوٹی ٹری بڑی ٹری کی سب ٹری بنتی ہے یونین آپریشن میں اس لئے نوڈ 1 اب نوڈ 4 کا چلڈ نوڈ بن جائے گا۔



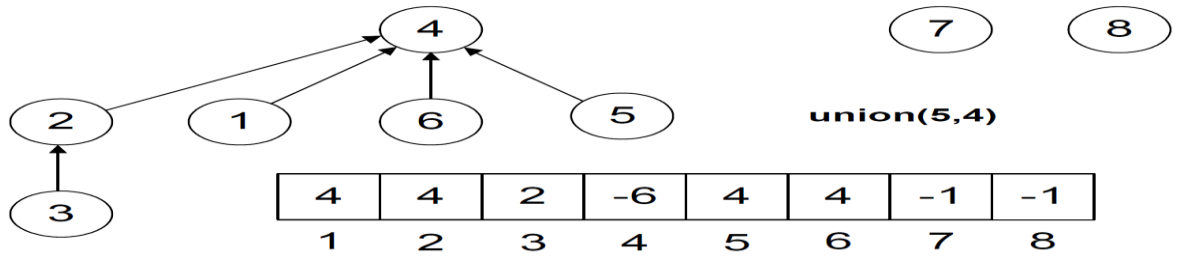
union(1,4)

|   |    |   |    |    |   |    |    |
|---|----|---|----|----|---|----|----|
| 4 | -2 | 2 | -3 | -1 | 4 | -1 | -1 |
| 1 | 2  | 3 | 4  | 5  | 6 | 7  | 8  |

اب نوڈ 2 اور 4 پر یونین آپریشن ہوگا، نوڈ 2 بھی چھوٹا ہے اس لئے یونین آپریشن جو کہ یونین by weight ہے اس کے مطابق چھوٹی یعنی سماں ٹری بڑی ٹری کا پارٹ بن جاتی ہے۔ اب نوڈ 2 کی ٹری چھوٹی ہے اس لئے وہ بڑی ٹری کی پارٹ بن جائے گی۔



یہاں ہم دیکھ سکتے ہیں کہ جیسے جیسے چھوٹی ٹریز اس بڑی ٹری 4 میں جوائن ہو رہی ہیں ارے میں نوڈ 4 کا انڈکس بھی بڑھ رہا ہے۔ اب ہم نوڈ 5 پر یونین by سائز کا رول apply کرے گے۔ یونین by سائز کے مطابق چھوٹی ٹری بڑی ٹری کا پارٹ بنتی ہے، مگر یہاں نوڈ 5 اکیلا ہے اس لئے یہ ٹری 4 کا پارٹ بن جائے گا۔



ارے انڈکس بھی ساتھ ساتھ چینیج ہو رہا ہے اور یہ چینیج اس میں یونین by سائز کی وجہ سے آرہی ہے۔ اس رول کی وجہ سے ہم نے سائز کو کافی کم کر دیا ہے۔ یاد رکھیں کہ ٹری کی ہائیٹ کو کم کرنے کا فائدہ ٹری کی پرفارمنس کو بڑھانا ہے جب ایلیمینٹ کو ٹری کے اندر سرچ کیا جاتا ہے۔

If unions are done by weight (size), the depth of any element is never greater than  $\log_2 n$ .

یونین by سائز ہمیں ٹری کے زیادہ سے زیادہ 20 لیول کی depth دیتا ہے۔

## Union by Height:

یہ یونین by سائز کا الٹ ہے، یہ ٹری کی ہائیٹ کو Maintain کرتا ہے۔ یونین کے دوران چھوٹی ہائیٹ کی ٹری کو دوسری ٹری کی سب ٹری بنا دیتا ہے۔ چھوٹی ہائیٹ کی ٹری بڑی ہائیٹ کی ٹری کو جوائن کر لیتی ہے۔

## Sprucing up Find:

ابھی تک ہم نے یونین کو Optimize کیا ہے، کیا ہم فائنڈ کو بھی Optimize کر سکتے ہیں؟ تو اس کا جواب ہے جی ہاں کر سکتے ہیں، ہم فائنڈ کو پاتھ Compression کے ساتھ آپٹمائز کر سکتے ہیں۔ فائنڈ کے دوران ہم پاتھ کو آئی سے روٹ تک ٹراورس کرتے ہیں۔ تمام نوڈز کی parent انٹریز اپ ڈیٹ ہوں گی روٹ تک۔ اس سے تمام نوڈز کی ہائیٹ Reduce ہوگی۔

Pay now, and reap the benefits later

# Lecture No. 37

- Image Segmentation
- Maze Example
- Pseudo Code of the Maze Generation

## Image Segmentation:

ایمجز Segmentation میں ایمجز کو مختلف پارٹ میں divide کیا جاتا ہے۔ ایمجز کو پکسل کی Intensity کی بنیاد پر بھی سیگمنٹ کیا جاسکتا ہے۔ ہو سکتا ہے کہ ہمارے پاس پکسل کا گروپ ہو جن کی Intensity ہائی ہو اور کچھ کی لو بھی ہو سکتی ہے۔ ان پکسل کو ان کی ویلیو کے Threshold کی بنیاد پر Divide کیا جاتا ہے۔ گرے لیول کے پکسل جو 50 سے کم ہوں کو ایک گروپ میں مہائن کیا سکتا ہے جو کے گرے لیول کے گروپ جو 50 سے کم ہوگا اسکو فالو کرے گے۔ پکسل کے 8 Neighbors ہوتے ہیں جو کے درج ذیل ہیں۔ TOP، Bottom، Left، Right، Top-Left، Top-Right، Bottom-Left، Bottom-Right۔ اب ہم پکسل کے Threshold اور اس کے Neighbors میں Difference دیکھتے ہیں۔ مختلف ویلیو کی بنیاد پر ہم پکسل کو گروپنگ کرے گے۔ پکسل کو Texture کی بنیاد پر گروپ کر سکتے ہیں، پکسل کو Divide ہم Threshold کی بنیاد پر کرتے ہیں۔



اس تصویر کو اگرا دیکھا جائے تو اس میں وائٹ، گرے اور بلیک portions نظر آرہے ہیں۔ یہ چھوٹے چھوٹے پورشن پکسل یا ہم کہہ سکتے ہیں کہ تصویر کے ایلیمینٹ کوری پرسنٹ کر رہے ہیں۔ اس میں پانچ Rows اور پانچ ہی کالم ہیں۔ اس کو  $5 \times 5$  میج بھی کہتے ہیں۔ اب ہم پانچ Rows اور پانچ کالم کا میٹر کس لیں گے۔ ہم میٹر کس میں ایلیمینٹس کو ویلیو Assign کرے گے جو کہ ان کے کلر پر Depend کرے گی۔ وائٹ کلر کیلئے 0 استعمال ہوا ہے، بلیک کیلئے 4 اور گرے کیلئے 2 استعمال ہوا ہے۔

|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 4 | 4 |
| 1 | 2 | 0 | 4 | 4 | 0 |
| 2 | 4 | 2 | 2 | 4 | 4 |
| 3 | 4 | 4 | 0 | 4 | 4 |
| 4 | 0 | 2 | 2 | 4 | 0 |

جب ہم ڈیجیٹل ڈیوائس سے میج کو حاصل کرتے ہیں تو یہ نمبرز میں سٹور ہوتے ہیں۔ اوپر کے میٹر کس میں ہمارے پاس 0 سے لیکر 4 تک نمبرز ہیں۔ یہ نمبرز 0 سے لیکر

255 کے درمیان ہو سکتے ہیں یا پھر ڈیجیٹل ڈیوائس سے لے گئی تصویر پر منحصر ہو سکتے ہیں۔ اب ایک اور امیج بھی ہے جس کو Raw ڈیجیٹل امیج کہتے ہیں۔ مثلاً ہمارے پاس ایک پکسل ہے جس کی ویلیو 4 یا 4 سے زیادہ ہے اور پکسلز کی ویلیو 4 سے کم ہے۔ جب ایسا پکسل فائنڈ ہو جائے تو ہم ایک پٹ کرے گے ان پکسلز کیلئے جس کی ویلیو 4 سے زیادہ ہے اور 0 پٹ کرے گے ان پکسلز کیلئے جن کی ویلیو 4 سے کم ہے۔ ہم بائری اررے بنانا چاہتے ہیں Threshold کو Apply کر کے۔

|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 2 | 1 | 0 | 0 | 1 | 1 |
| 3 | 1 | 1 | 0 | 1 | 1 |
| 4 | 0 | 0 | 0 | 1 | 0 |

ہم نے Threshold کو apply کیا ہے جو 4 کے برابر ہے۔ ہم نے تمام ڈجٹ جو کہ 4 ہیں انکو 1 سے replace کیا ہے اور جن کی ویلیو 4 سے کم ہے وہ 0 کے ساتھ Replace کیا ہے۔ اب امیج کو بائری فارم میں کنورٹ کر دیا گیا ہے۔ عام طور پر ہمارے پاس 25 سیٹس ہوتے ہیں جو اررے میں سٹور ہوتے ہیں۔

# Maze Example:

مزے Maze ایک Puzzle گیم ہے جس میں User ایک سائیڈ سے داخل ہوتا ہے اور پھر اسے باہر جانے کیلئے پاتھ (راستہ) تلاش کرنا پڑتا ہے۔ زیادہ تر پاتھ بلا سٹڈ Alley میں جاتے ہیں جو User کو فورس کرتے ہیں کہ واپس اسکوائر ون میں جائے۔ یہ روبو ٹکس میں بہت مفید ہو سکتا ہے۔ Maze گیم کو یونین- فاسٹڈ کے Algorithm کے ڈس جائنٹ سیٹس سے بنایا جاتا ہے۔ اس Algorithm سے بہت آسانی سے مزے گیم کو سمجھا جاسکتا ہے۔

ہم 5\*5 کا گرڈ لیتے ہیں اور 5\*5 مزے Generate کرتے ہیں۔ Random مزے جنریٹر یونین- فاسٹڈ کا Algorithm استعمال کر سکتی ہے۔ رینڈم مطلب نیا مزے ہر بار Generate ہونا چاہئے۔

Consider a 5x5 maze:

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 |
| 20 | 21 | 22 | 23 | 24 |

اب ہمارے پاس 25 Cells ہیں اور ہر سیل دوسرے سیل سے ایک دیواری کی صورت میں الگ کی گئی ہے۔ ہمارے پاس یہ 0 سے لیکر 25 تک نہیں بلکہ 24 تک ہیں اگر

25 تک ہو جاتے تو ٹوٹل نمبر 26 ہونے تھے جو کہ  $5 \times 5$  سے زیادہ بنتے ہیں۔ تمام ویلیوز کے گرد لائنز کی ایک دیوار ہے جو ان ویلیوز کو Move نہیں ہونے دے رہی ہے۔ اب ہمیں سب سے پہلے Entrance اور Exit کو Decide کرنا ہوگا۔ ہماری مثال میں 0 سے دخل ہوں گے اور 24 سے باہر جائے گے۔

→

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 |
| 20 | 21 | 22 | 23 | 24 |

→

اب مزے گیم کو کیسے create کر سکتے ہیں، اس کے کچھ رولز ہیں۔

- Randomly remove walls until the entrance and exit cells are in the same set.
- Removal of the wall is the same as doing a union operation.
- Do not remove a randomly chosen wall if the cells it separates are already in the same set.

## Lecture No. 38

- Table and Dictionaries
- Operations on Table ADT
- Implementation of Table
  - Unsorted Sequential Array
  - Sorted Sequential Array
- Binary Search

ٹیبل ایک Abstract ڈیٹا ٹائپ ہے جو کہ Rows اور کالمز کا کلیکشن ہوتا ہے۔ ان Rows اور کالمز میں انفارمیشن ہوتی ہے۔ اب Rows اور کالمز سے ہمیں یہ پتا چل گیا ہے کہ یہ ڈیٹا Dimensional ہے۔ مگر ضروری نہیں ہے کہ یہ ہمیشہ ایک جیسے ڈیٹا ٹائپ کا ڈیٹا Dimensional ہے ہو۔ ٹیبل کالمز میں انفارمیشن مختلف ہو سکتی ہے۔ ہو سکتا ہے کہ پہلے کالم میں ڈیٹا ٹائپ Integer ہو اور دوسرے کالم میں ڈیٹا ٹائپ سٹرنگ ہو۔ اور اسی طرح دوسرے ٹیبل میں بھی ڈیٹا ٹائپ مختلف ہو سکتا ہے۔ اس لئے ڈیٹا Dimensional ہے جو کہ ٹیبل کیلئے استعمال کی جاتی ہے اس کا ڈیٹا ٹائپ مختلف ہو سکتا ہے۔ ٹیبل میں کئی کالمز ہوتے ہیں جو کہ فیلڈز کے طور پر جانے جاتے ہیں۔ ان فیلڈز میں کچھ انفارمیشن ہوتی ہے۔ جیسے کہ ٹیلیفون ڈائریکٹری میں نام۔ ایڈریس اور فون نمبر ہوتا ہے۔ اس طرح کمپیوٹر میں User اکاؤنٹ میں User-ID، پاس ورڈ اور ہوم فولڈر ہوتے ہیں۔

| Name          | Address                               | Phone    |
|---------------|---------------------------------------|----------|
| Sohail Aslam  | 50 Zahoor Elahi Rd, Gulberg-4, Lahore | 567-4567 |
| Imran Ahmad   | 30-T Phase-IV, LCCHS, Lahore          | 517-2349 |
| Salman Akhtar | 131-D Model Town, Lahore              | 756-5678 |

ٹیبل کے ہر Row میں ہمیں کچھ انفارمیشن نظر آرہی ہے۔ یہ Rows ایک دوسرے سے لنک ہیں۔ ٹیبل کے ان Rows کو ریکارڈ کہا جاتا ہے۔ ہمارے پاس اوپر ٹیبل میں تین ریکارڈ ہیں۔ پروگرامر کیلئے ڈکشنری ایک ڈیٹا ٹائپ ہوتی ہے۔ ٹیبل کا سب سے زیادہ استعمال ہم ڈیٹابیس میں کرتے ہیں جہاں ہم ٹیبل بنا کر انفارمیشن کو سٹور کرتے ہیں۔ اگر ہمیں کسی کمپنی کیلئے پے رول کا سافٹ ویئر بنانا ہو تو اس صورت میں کمپنی کے ایمپلوائی کا ٹیبل بنانا پڑے گا۔ ایمپلوائی ٹیبل میں نام، آئی ڈی، Designation، تنخواہ کی انفارمیشن سٹور ہوگی۔

ٹیبل کی ایک اور آپلیکیشن کمپلیر ہے، اس میں Symbol ٹیبلز استعمال ہوتے ہیں۔ ٹیبل میں کسی بھی ریکارڈ کو تلاش کرنے کیلئے ہمیں اس ریکارڈ کی ایک key پتا ہونی چاہئے۔ اگر User کو تلاش کرنا ہے تو اس کی Key اس کی User-ID ہوگی۔ Key ہمیشہ یونیک ہونی چاہئے تاکہ ریکارڈ کو تلاش کرنے میں آسانی ہو۔ ڈیٹابیس میں اس کو پرائمری key کہا جاتا ہے۔

## Operations on Table ADT:

جیسے کہ ہم جانتے ہیں کہ ٹیبل ایک Abstract ڈیٹا ٹائپ ہے، اس ڈیٹا ٹائپ کے آپریشن کے کچھ میتھڈ بھی ہوتے ہیں۔ یہ Mehtods انسرت، فائنڈ اور Remove ہیں

انسرت:

اس کے نام سے پتا چل جانا چاہئے کہ اس میتھڈ سے ڈیٹا ٹیبل میں ریکارڈ انسرت ہوگا۔ اس کے Execution کیلئے فیلڈ کو ایک Key سے Designate کیا جاتا ہے۔ ٹیبل میں ریکارڈ کو انٹر کرنے کیلئے Key اور اینٹری کو جاننا کی ضرورت ہے۔ انسرت کا میتھڈ key کو دوسرے Related فیلڈز میں پٹ کر دیتا ہے۔ مطلب کہ انسرت میتھڈ key کو پٹ کرتا ہے۔ اس طرح ہم ریکارڈ کو ٹیبل میں Add کرتے ہیں۔ فائنڈ:

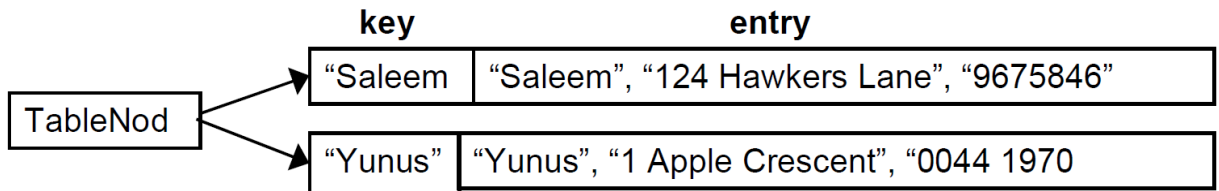
ہمارے پاس ٹیبل میں کچھ ڈیٹا ہوگا اور ہم اس میں سے کچھ خاص انفارمیشن کو تلاش کرنا چاہیں، تو فائنڈ میتھڈ اس ڈیٹا کو ایک Key دے دیتا ہے جس سے ہم ڈیٹا کو سرچ کر سکتے ہیں۔ تو اسکا مطلب ہوا کہ فائنڈ میتھڈ سے ہم ڈیٹا بیس میں کچھ خاص انفارمیشن کو سرچ کرتے ہیں۔

:Remove

اس میٹھڈ سے ہم ڈیٹا بیس میں سے اگر کچھ خاص ڈیٹا ڈیلیٹ کرنا چاہیں تو اس میٹھڈ کو استعمال کر کے سکتے ہیں۔ مگر ڈیلیٹ سے پہلے فائنڈ میٹھڈ پر فارم ہوگا اور پھر جب ڈیلیٹ ہونے والا ایلیمنٹ فائنڈ یعنی مل جائے تو تب ڈیلیٹ کا میٹھڈ پر فارم ہوگا۔

## Implementation of Table:

ٹیبل میں ڈیٹا سٹور کرنے اور اس کو سرچ کرنے کیلئے ضروری ہے کہ ہم اس کو ایک Key بھی اسائن کر دے۔ اب فرض کر لیں کہ ہمارے پاس کچھ ڈیٹا ہے جس میں نام اور ایڈریس اور فون نمبر ہے، یہ دو ٹیبل میں ہوگا ایک مکمل اور دوسرے میں ایک خاص ڈیٹا جو Key کے طور پر استعمال ہوگا تاکہ اس سے ڈیٹا کو سرچ کرنا آسان ہو جائے۔ ہمارے پاس جو Key ہوگی اس سے ہم ڈیٹا کو سرچ اور ڈیلیٹ کر سکیں گے۔ Key ہمیشہ ہر ریکارڈ کی یونیک ہونی چاہئے۔ Key کی مدد سے ہم Row تک پہنچ جاتے ہیں اور پھر دوسرے فیلڈز تک بھی اسی Key کی مدد سے پہنچ جاتے ہیں۔ ہم ٹیبل کی Row ٹیبل نوڈ بھی کہتے ہیں۔



ایک جگہ ہمارے پاس Key ہے اور دوسری جگہ پورا ڈیٹا ہے۔ اب اس Key کی مدد سے ہم اس ڈیٹا کو سرچ اور ڈیلیٹ کر سکتے ہیں۔ دونوں جگہ Key یونیک ہے اگر Key

یونیک نہ ہو تو بہت سارے ریکارڈ ایک جیسی Key سے سٹور ہو جائے جس سے سرچ اور ڈیلیٹ کرنے میں مشکل پیش آئی گی۔

## Unsorted Sequential Array

اس Implementation میں ہم ٹیبل کا ڈیٹا ارے میں سٹور کرتے ہیں جس سے ٹیبل نوڈ کسی بھی آرڈر میں سٹور ہو جاتے ہیں۔ Row کے ہر ایلیمنٹس کی ایک Key ہوگی اور ریکارڈ کی انٹری ہوگی۔ ہم ٹیبل کا ڈیٹا کیسے ارے میں سٹور کر سکتے ہیں؟ مثال کے طور پر ہمارے پاس ٹیلی فون ڈائرکٹری ہے جس میں کچھ فیلڈز ہوتی ہیں۔ اب اس میں نام کا فیلڈ ہوگا جو کہ سٹرنگ ڈیٹا ٹائپ ہوگا، پھر فون نمبر کی فیلڈ ہوگی جس کا ڈیٹا ٹائپ سٹرنگ یا انٹیجر ہوگا، پھر ایڈریس کی فیلڈ ہوگی جو سٹرنگ یا انٹیجر ڈیٹا ٹائپ ہوگی۔ فون ڈائرکٹری میں یہ تین فیلڈز ہو سکتی ہیں اور ان کا ڈیٹا ٹائپ بھی مختلف ہو سکتا ہے۔ اب ہم اس انفارمیشن کو ارے میں کیسے سٹور کر سکتے ہیں؟ اب ہم یہاں پر کلاس سے کام لے گے، ہم ایک کلاس بنائیں گے جس کا نام فرض کریں ہم ایمپلائی رکھ دے گے، اب اس کلاس کے آبجیکٹ میں نام، ایڈریس، Designation، ایمپلائی کی تنخواہ اور کچھ دوسری انفارمیشن ہو سکتی ہے۔ اسی طرح ہم فون ڈائرکٹری کی بھی کلاس بنا سکتے ہیں۔ اب ہم ان آبجیکٹس کا ارے بنائیں گے، اب ارے میں ہر آبجیکٹ کی ایک ٹیبل انٹری ہوگی۔ اب نیا آبجیکٹ ایڈ کرنے کیلئے ہم

اررے کے بیک پر اس کو ایڈ کرے گے۔ اررے میں یہ انسرٹ آپریشن فاسٹ ہوتا ہے بجائے اس کے ہم ڈیٹا کو آخری Available پوزیشن میں پٹ کر دے۔ دوسرے الفاظ میں ہم کہہ سکتے ہیں کہ ہمارے پاس ڈیٹا کر کوئی آرڈر نہیں ہے، اب اگر ہمارے پاس نمبرز اف انٹریز ہوں تو فائنڈ آپریشن جو Key سے ڈیٹا سرچ کرتا ہے زیادہ ٹائم لے گا۔ اس میٹھڈ میں ڈیٹا انسرٹ کرنا فاسٹ ہوتا ہے مگر فائنڈ آپریشن Slow ہوتا ہے اور سرچنگ زیادہ ٹائم لیتی ہے۔ اب سوال پیدا ہوتا ہے کیا ہمارے پاس کوئی ایسا طریقہ ہے جس سے سرچ آپریشن آسان اور فاسٹ ہو سکے؟ تو اس کا جواب ہے سورٹڈ Sequential اررے۔

## Sorted Sequential Array

ہم نے ٹری سیکشن میں سسٹمی کی تھی کہ بائرنری سرچ ٹری کا استعمال کر کے انفارمیشن کو تلاش کیا جاتا ہے۔ اسی طرح سورٹڈ Sequential اررے کے بارے میں یہ کہنا سچ ہوگا کہ اس سے ڈیٹا سرچ کرنا آسان اور فاسٹ ہوتا ہے۔ مگر اس کیس میں ہم نے ڈیٹا ٹری میں نہیں بلکہ اررے میں ڈالنا ہے، اررے میں ڈیٹا سرچ کرنا بہت فاسٹ ہوتا ہے۔ اس فاسٹ سرچ کے مقصد کے حصول کیلئے ہم ڈیٹا کو اررے میں سورٹڈ فارم میں رکھے گے۔ فرض کریں کہ ہم ٹیلی فون ڈائریکٹری ٹیبیل کی انفارمیشن اررے میں سورٹڈ فارم میں ایک خاص آرڈر میں رکھ رہے ہیں، مثلاً ہم نے ڈیٹا Alphabetically

سورٹ کر کے انسرت کیا ہے، اب جس کا نام Alphabet کے پہلے لفظ یعنی A سے شروع ہوگا اس کا نام ٹیبیل میں سب سے پہلے آئے گا، اور جس کا نام B سے شروع ہوگا اس کا ڈیٹا A ٹیبیل کے بعد آئے گا۔ اس آرڈر کو ہم سورٹڈ Sequential اررے کہتے ہیں، اس میں ڈیٹا آرڈر میں انسرت ہوتا ہے اور سرچنگ فاسٹ ہوتی ہے۔

## *Binary Search*

بائنری سرچ بھی ایک سرچنگ Algorithm ہے جو سورٹڈ ڈیٹا کے ساتھ استعمال ہوتا ہے۔ اگر ہمارے اررے میں ایلیمنٹس سورٹڈ آرڈر میں ہیں تو ہم بائنری سرچ میٹھڈ کو استعمال کر کے ڈیٹا فائنڈ کر سکتے ہیں اررے میں۔ بائنری سرچ سورٹڈ اررے میں ایلیمنٹس کو فائنڈ کرنے کیلئے جو ٹائم لیتی ہے اس کو  $\log N$  Time کہتے ہیں۔ اگر ہمارے پاس 100000 ایلیمنٹس ہو اررے میں تو  $\log 100000$  کا صرف 20 بنے گا جو کہ بہت کم ہے 100000 کے مقابلے میں۔ اس لئے بائنری سرچ بہت فاسٹ ہے۔ بائنری سرچ ایسے ہے جیسے ڈائرکٹری میں فون نمبر کو تلاش کرنا یا ڈکشنری میں کسی لفظ کو تلاش کرنا۔ اگر ڈیٹا پہلے سے سورٹڈ فارم میں موجود ہے تو بہتر ہے کہ بائنری سرچ کے میٹھڈ کو ڈیٹا سرچ کرنے کیلئے استعمال کیا جائے بجائے اس کے ڈیٹا کو سٹارٹ سے لیکر اینڈ تک سرچ کیا جائے۔

# Lecture No. 39

Binary Search کے Algorithm کا ہم سڈو کوڈ دیکھتے ہیں۔

```
if ( value == middle element )  
    value is found  
else if ( value < middle element )
```

*search left half of list with the same method*

*else*

*search right half of list with the same method*

یہاں پر یہ بات اہم ہے کہ یہ algorithm ڈیٹا ٹائپ سے آزاد ہے مطلب اس میں ڈیٹا ٹائپ کچھ بھی ہو سکتا ہے چاہئے وہ Numeric ہو یا پھر سٹرنگ ٹائپ۔ اس Algorithm میں جو ایٹم ہم سرچ کر رہے ہیں اس کو ویلیو کہتے ہیں۔ ویلیو کا جو پہلا Comparison ہوگا وہ اررے کے مڈل ایلیمنٹ سے ہوگا، اگر دونوں برابر ہوئے تو اس کا مطلب ہے کہ جو سرچ ہم کرنا چاہتے تھے وہ ہمیں مل چکا ہے، جو کہ اررے کے مڈل میں موجود ہے۔ اگر یہ برابر نہ ہوئے تو پھر ویلیو اور مڈل ایلیمنٹ ایک جیسے نہیں ہیں۔

Else If کا پارٹ یہ چیک کرے گا کہ کیا ویلیو مڈل ایلیمنٹ سے کم ہے یا نہیں۔ اگر ویلیو مڈل ایلیمنٹ سے کم ہوگی تو لفٹ ہاف پارٹ اررے کا مزید سرچ ہوگا۔ اس سرچ کو ہم Non-Recursive کیوری یا Algorhtim کہتے ہیں۔ اس Algorithm کا آخری پارٹ اس کیس کو ڈیل کرتی ہے جس میں ویلیو مڈل ایلیمنٹ سے بڑی ہو۔

یہ پراسیس اررے کارائنٹ ہاف پراسیس ہوتا ہے۔ آئے اس کو ایک مثال سے سمجھنے کی  
کو کوشش کرتے ہیں۔

Case#1:

Value==a(mid)

Val:10

Low:0, high=8

Mid=(0+8)/2=4

|    |     |   |   |   |     |    |    |    |      |
|----|-----|---|---|---|-----|----|----|----|------|
| a: | 1   | 5 | 7 | 9 | 10  | 13 | 17 | 19 | 27   |
|    | 0   | 1 | 2 | 3 | 4   | 5  | 6  | 7  | 8    |
|    | ↑   |   |   |   | ↑   |    |    |    | ↑    |
|    | low |   |   |   | mid |    |    |    | high |

اس مثال میں ہم دیکھ سکتے ہیں کہ ہمارے پاس ایک اررے ہے جس کو ہم نے نام a  
دیا ہے اور اس اررے کے انڈکس 0 سے لیکر 8 تک ہیں اور ان انڈکس میں جو ویلیوز  
ہیں وہ 1، 5، 7، 9، 10، 13، 17، 19، 27 ہیں۔ یہ ویلیوز ہمارے ڈیٹا ایٹمز ہیں۔  
یہ ویلیوز اس ایڈنگ آرڈر یا نہی Increasing آرڈر میں سورٹڈ ہیں۔ اب ہمیں پتا کیسے  
چلے گا کہ ہم کون سی ویلیو تلاش کر رہے ہیں؟ val = 10 جو لکھا ہوا ہے اس کا مطلب  
ہے کہ ہم ویلیو نمبر 10 کو سرچ کر رہے ہیں۔ low: 0 اور high = 8 کا مطلب ہے  
کہ جو سرچ رینج ہے ہمارے پاس وہ 0 سے لیکر 8 تک کی ہے۔ اب اس ڈیٹا رینج میں  
مڈل پوزیشن کو اس Simple فارمولا سے Calculate کیا گیا ہے جو کہ ہے

(high+low)/2 یعنی  $(0+8)/2=4$  یہ اررے کا مڈل پوزیشن ہے۔ اب ہمارے پاس پوزیشن 4 پر جو ویلیو ہے وہ 10 ہی ہے جو ہم سرچ کر رہے ہیں۔ اس کیس میں ہم نے ویلیو کو اررے کے مڈل میں فائنڈ کر لیا ہے۔

Case#2

Val>a[mid]

Val=19

Low=0,high=8

Mid= (0/8)/2=4

New low=mid+1=5

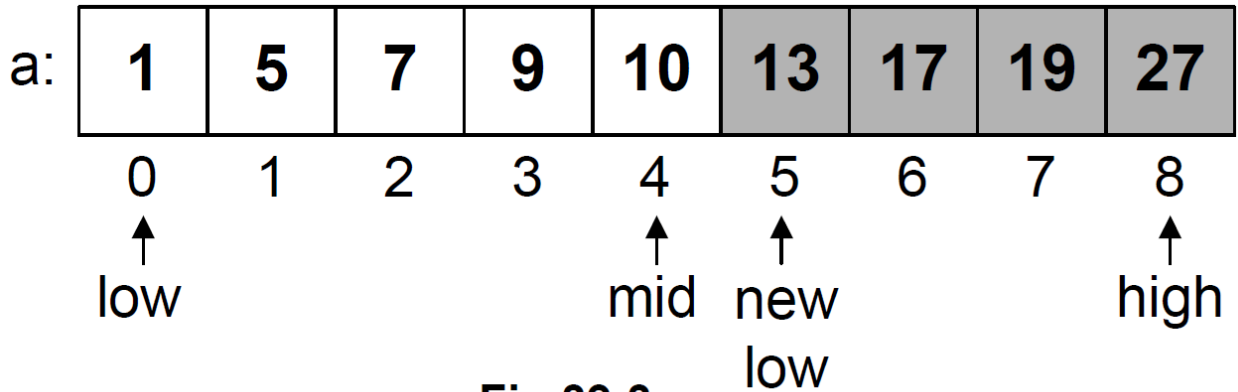


Fig 39.3

دوسرے کیس میں ہمارے پاس ویلیو مڈل ویلیو سے بڑی ہے۔ ڈیٹا ایٹم کی ریج پہلے کیس 1 والا ہی ہے۔ اس لئے اس کا مڈل پوزیشن بھی کیس 1 والا ہی ہے مگر val: 19 مڈل ویلیو 10 سے بڑی ہے۔ جیسے کہ یہ اررے سورٹڈ ہے اس لئے اررے کے لفٹ ہاف میں ویلیو نہیں ہونی چاہئے۔ ہمارے اس کیس میں ویلیو 19 مڈل ویلیو 10 سے بڑی ہے اس لئے اس کو کہیں اررے کے رائٹ ہاف میں ہونا چاہئے۔

اررے کا رائٹ ہاف پارٹ 5 سے لیکر 8 تک ہے، نیو پوزیشن اررے کے انڈکس کے 5 پر ہے جو کے ایلیمنٹ 13 بنتا ہے۔ اب اس نئے لو اور ہائی پوزیشن سے Algorithm اس رائٹ ہاف پر پھر سے Apply ہوگا۔

Binar Search Tree Case#3

Val < a[mid]

Val=7

Low=0, high=8

Mid=(0+8)/2=4

New high=mid-1=3

|    |     |   |   |      |     |    |    |    |      |
|----|-----|---|---|------|-----|----|----|----|------|
| a: | 1   | 5 | 7 | 9    | 10  | 13 | 17 | 19 | 27   |
|    | 0   | 1 | 2 | 3    | 4   | 5  | 6  | 7  | 8    |
|    | ↑   |   |   | ↑    | ↑   |    |    |    | ↑    |
|    | low |   |   | new  | mid |    |    |    | high |
|    |     |   |   | high |     |    |    |    |      |

اس کیس میں جو ویلیو سرچ کرنی ہے وہ 7 ہے، ڈیٹا رینج وہی ہے جو پہلے اور دوسرے کیس میں تھا یعنی 0 سے 8 تک۔ مڈل کی ویلیو اسی طریقے سے Calculate کی گئی ہے جس طرح پہلے اور دوسرے کیس میں کی گئی تھی، اور مڈل پوزیشن کی ویلیو مڈ کو val سے Compare کیا گیا ہے۔ val یعنی ویلیو 7 اس کیس میں مڈل کی ویلیو یعنی 10 سے چھوٹی ہے، اس اررے میں ڈیٹا سورتڈ ہے، اس لئے ویلیو جو کہ مڈل ویلیو سے کم ہوتی ہے وہ پھر اررے کے لوئر ہاف یعنی لفٹ ہاف میں کہیں ہوگی۔ اررے کا لفٹ ہاف low = 0 پوزیشن سے شروع ہوگا مگر ہائی پوزیشن اب اس کیس میں چینج ہوگا

کیونکہ mid-1 ہے اس کیس میں ہمارے جو کہ 1-4 یعنی کے 3 بنتا ہے۔ اب اس Algorithm کو ہم اسی اررے کے لفٹ ہاف پر پھر سے Apply کر کے output چیک کرتے ہیں۔

val = 7

|    |   |   |   |   |    |    |    |    |    |
|----|---|---|---|---|----|----|----|----|----|
| a: | 1 | 5 | 7 | 9 | 10 | 13 | 17 | 19 | 27 |
|    | 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  |

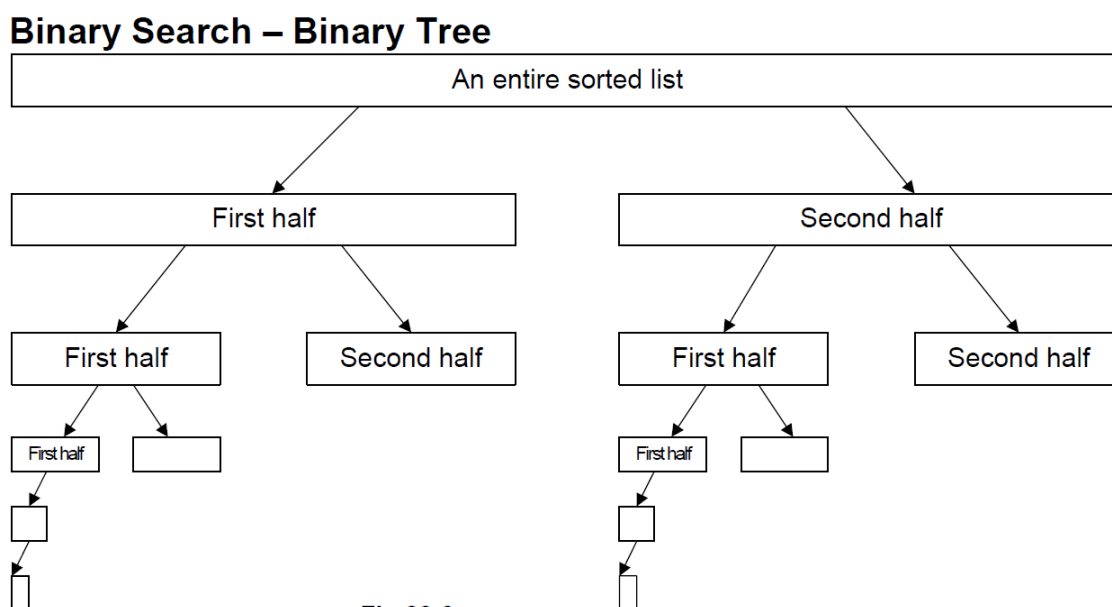
|    |   |   |   |   |    |    |    |    |    |
|----|---|---|---|---|----|----|----|----|----|
| a: | 1 | 5 | 7 | 9 | 10 | 13 | 17 | 19 | 27 |
|    | 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  |

|    |   |   |   |   |    |    |    |    |    |
|----|---|---|---|---|----|----|----|----|----|
| a: | 1 | 5 | 7 | 9 | 10 | 13 | 17 | 19 | 27 |
|    | 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  |

اب سب سے پہلے ہم 0 اور 3 کا مڈل Calculate کرے جو کہ 1 بنے گا، 1 اس لئے بنے گا ہم پہلے ہی  $(ہائی + لو) / 2$  کا فارمولا پڑھ چکے ہیں۔ اب جو ویلیو ہم سرچ کرے رہے ہیں وہ ویلیو ہم مڈل ویلیو سے Compare کرے گے۔ مڈل ویلیو ہمارے پاس 5 ہے اور جو ویلیو ہم سرچ کر رہے ہیں اس کیس میں وہ 7 ہے جو کہ ہمارے مڈل ویلیو 5 سے بڑی ہے، جب سرچ ہونے والی ویلیو مڈل ویلیو سے بڑی ہوتی ہے تو پھر ہم رائٹ ہاف ڈیٹارینج کو پراسیس کرتے ہیں۔ اب اس کیس میں ہمارے پاس low = 0

اور  $high = 3$  ہے، اب مڈل ویلیو mid کو Calculate کرنے کیلئے پھر سے فارمولا لگاتے ہیں،  $(2 + 3) / 2 = 2.5$ ، یعنی ہمارے پاس مڈل ویلیو 2 آگئی ہے، اب ہم اپنے آررے انڈکس میں چیک کرتے ہیں کہ انڈکس نمبر 2 پر ہمارے پاس کون سی ویلیو ہے؟ اگر ہم اپنے اس کیس کو دیکھیں تو ہمارے پاس انڈکس 2 پر وہی ایلیمینٹ ہے جس کو ہم سرچ کر رہے ہیں یعنی ویلیو 7 ہے۔

اس ٹری کو ایک نظر دیکھ لیں تاکہ آسانی سے سمجھ آ جائے



**Fig 39.6**

Remove آپریشن سے پہلے ہم فائنڈ آپریشن کرتے ہیں تاکہ جو ایلیمینٹ ڈیلیٹ کرنا ہے اس کو پہلے فائنڈ کیا جائے پھر اس کو ڈیلیٹ یا Remove کیا جائے۔

## Implementation 3 (of Table ADT): Linked List

یہ بھی ہو سکتا ہے کہ ہم لنکڈ لسٹ کو استعمال کر کے ٹیبل Abstract ڈیٹا ٹائپ کو Implement کریں۔ ہم ڈیٹا کو Unsorted کر کے رکھ سکتے ہیں اور نئے ایلیمینٹ جو بھی آئیں گے انکو لسٹ کے فرنٹ میں انسرت کرتے جائے گے۔ یہ بھی ممکن ہے کہ ڈیٹا کو سورٹڈ آرڈر میں رکھا جائے، اب اگر ڈیٹا سورٹڈ آرڈر میں ہوگا اور کوئی نیا ایلیمینٹ کو انسرت کرنا ہوا تو پھر ہم سب سے پہلے نئے ایلیمینٹ کیلئے پوزیشن کو فائنڈ کرے گے جیسے کہ سورٹڈ آرڈر میں کرتے ہیں، جب پوزیشن مل جائے گی تب نیا ایلیمینٹ کو انسرت کر دے گے۔ یاد رکھیں کہ بائسری سرچ صرف آرڈر کیلئے کام کرتا ہے۔ کیونکہ ہو سکتا ہے کہ لنک لسٹ میموری میں Contiguous نہ ہو، عام طور پر لنک لسٹ کے نوڈز ایک جگہ پر نہیں ہوتے اس لئے بائسری سرچ ٹری لنک لسٹ کیلئے کام نہیں کر سکتی ہے۔

TableNodes سورٹڈ یا انسورٹڈ سٹور ہوتی ہے۔ ٹیبل کیلئے پہلے پارٹ میں key اور دوسرے میں ریکارڈ کی انٹری ہوتی ہے۔

| Key | Entry |
|-----|-------|
|-----|-------|

لنک لسٹ ایک چوائس ہے ٹیبل Abstract ڈیٹا ٹائپ کو امپلمنٹ کرنے کیلئے۔ Unsorted ایلیمینٹس کیلئے انسیرشن فرنٹ سے ہوتی ہے اور انسرت آپریشن کانسٹنٹ ٹائم لے گا۔

## Implementation 4 (of Table ADT): Skip List

اس ٹائپ کے لسٹ میں سرچ اور اپ ڈیٹ کیلئے Linear ٹائم درکار ہوتا ہے۔

سورٹڈ Chain میں فاسٹ سرچنگ ہوتی ہے۔

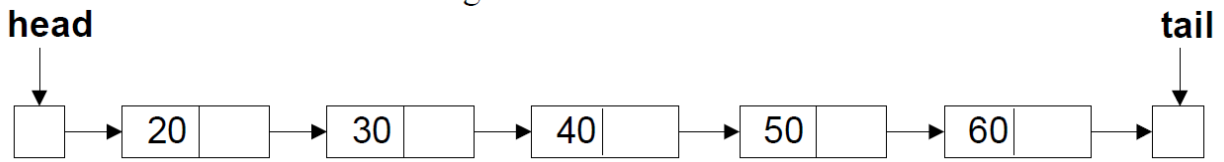
سکیپ لسٹ کو بل Pugh ان 1990 میں Proposed کیا تھا۔

یہ کچھ خاص خصوصیات تھی۔

اب سکیپ لسٹ کو تھوڑا ڈیٹیل میں دیکھتے ہیں۔

سکیپ لسٹ نمبرز اف Comparisons سے بہتر ہوتا ہے۔

Element in chain of length n



ہیڈ اور ٹیل نوڈز سکیپ لسٹ کے خاص نوڈز ہوتے ہیں، ہیڈ نوڈ سٹارٹ میں اور ٹیل نوڈ اینڈ یعنی آخر میں ہوتا ہے۔ اب اگر سے سکیپ لسٹ میں سے ہم نے نوڈ 60 کو سرچ کرنا ہوا تو ہیڈ سے شروع کرے گے اور نیکسٹ پوائنٹر کا استعمال کر کے سرچ کرتے جائے گے جب تک وہ نوڈ نہ آجائے جو ہم سرچ کر رہے ہیں یا ٹیل نہ آجائے۔ اب اگر 70 کو اس لسٹ میں سرچ کرنا ہوا تو پھر سے پوری لسٹ سرچ ہوگی اور پھر آخر میں پتا چلے گا کہ 70 ایلیمینٹ تو اس لسٹ میں ہے ہی نہیں۔ اب یہاں پر پروفیسر Pugh نے کچھ Suggest کیا ہے۔

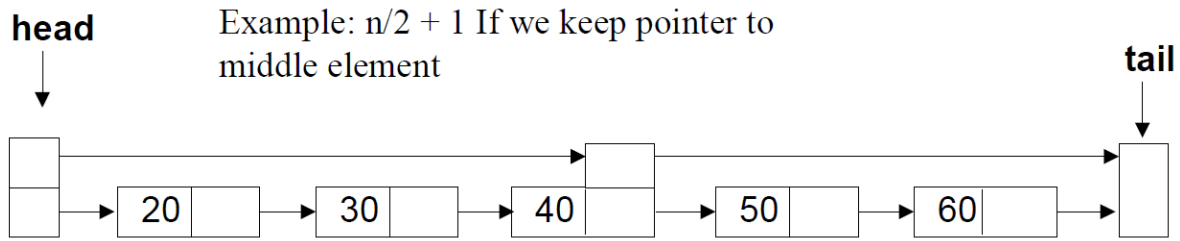


Fig 39.9

اب یہاں پر ہم نے دو پوائنٹرز استعمال کئے ہیں جو کہ ہیڈ اور ٹیل ہیں۔ اور پھر جو نوڈ  
 مڈل میں اس کے بھی دو نیکیسٹ پوائنٹرز ہیں، ایک اولڈ لنکڈ لسٹ کا پوائنٹر ہے جو کہ  
 اگلے نوڈ 50 کی طرف لیڈ کر رہا ہے، اور دوسرا پوائنٹر ٹیل کی طرف لیڈ کر رہا ہے۔  
 اب اس میں ہمارے پاس ہیڈ میں دو پوائنٹرز ہیں جس میں ایک اولڈ لنکڈ لسٹ پوائنٹر  
 ہے جو نوڈ 20 سے اگلے نوڈ کو پوائنٹ کر رہا ہے دوسرا مڈل ایلیمینٹ کو پوائنٹ کر رہا ہے  
 جو مزید ٹیل کی طرف پوائنٹ ہو رہا ہے۔

اب اگر ہم اس سکیپ لسٹ میں سے ویلیو 60 کو فائنڈ کرنا چاہتے ہیں تو اب اس کو  
 زیادہ فاسٹ طریقے سے سرچ کیا جاسکتا ہے۔ کیونکہ اب ہمارے پاس ایڈیشنل پوائنٹرز  
 ہیں اس لسٹ میں۔ سب سے پہلے ہم لسٹ کے مڈل ویلیو یعنی 40 پر آئے گے، اب  
 چونکہ مڈل ویلیو ہماری سرچ ویلیو سے کم اسلئے اب رائٹ ہاف پارٹ لسٹ کا سرچ ہوگا  
 ۔ اب یہ تو بالکل بائسری سرچ جیسا ہو گیا ہے مگر یہ ایسے کیوں ہوا؟ ایسے اس لئے ہوا  
 ہے کہ ہم نے اس میں ایڈیشنل پوائنٹرز رکھ دے ہیں جس سے سرچنگ فاسٹ ہو گئی  
 ہے۔

# Skip List - Higher Level Chains

- For general  $n$ , level 0 chain includes all elements
  - level 1 every other element, level 2 chain every fourth, etc.
  - level  $i$ , every  $2^i$  th element
- Level 0 chain is our old linked list chain
- سکیپ لسٹ chains کی Hierarchy رکھتا ہے۔

## Skip List - Formally

سکیپ لسٹ ایس سیٹ کی ڈسٹینکٹ key، ایلیمنٹ آئٹمز لسٹس کی سیریز ہوتی جو کہ  $S_0, S_1, \dots, S_h$  تک ہو سکتی ہے۔

ہر لسٹ  $S_i$  خاص Keys رکھتی ہے جو کہ  $-\infty$  and  $+\infty$  ہوتی ہے۔

لسٹ  $S_0$  ایسے keys کو Contains کرتا ہے جو Non-Decreasing آرڈر میں ہوتے ہیں، ہر لسٹ پچھلے لسٹ کی Subsequence ہوتی ہے۔ مطلب

$$S_0 \supseteq S_1 \supseteq \dots \supseteq S_h$$

لسٹ ایچ صرف دو خاص Contain کرتا ہے۔ سکیپ لسٹ کے بارے میں مزید ڈیٹیل میں سٹڈی کرنے کیلئے ٹیکسٹ بک کو چیک کریں۔ ہینڈ آؤٹ میں بھی یہی لکھا ہے۔

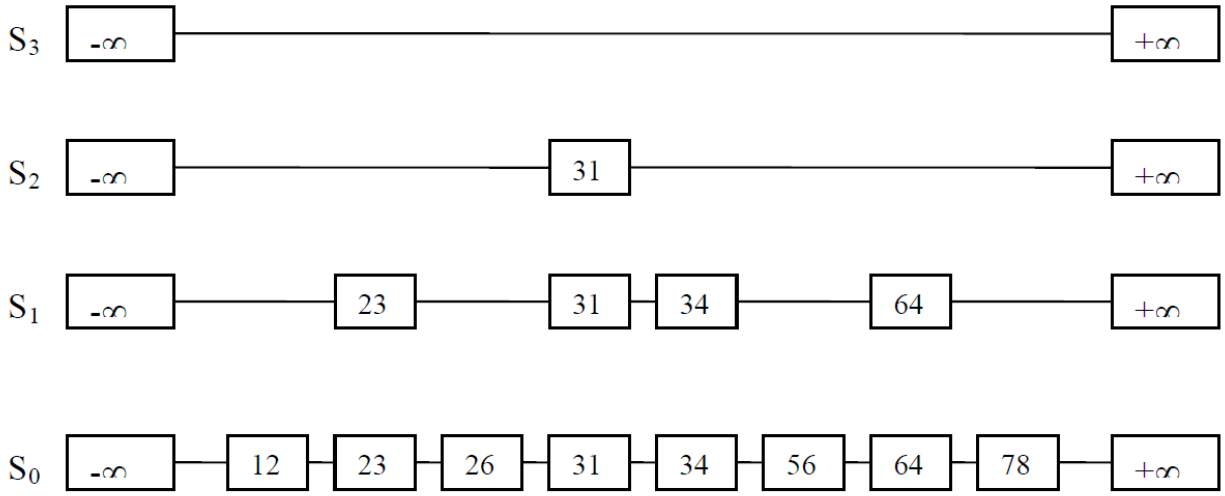
## Lecture No. 40

# Summary

- Skip List
- Skip List Search
- Insertion in Skip List
- Deletion from Skip List

سب سے پہلے ہمارے پاس  $S_0$  یعنی کے لنکڈ لسٹ ہے۔ ایس 0 کا پہلے اور آخری نوڈ  $-\infty$  and  $+\infty$  پر مشتمل ہوتا ہے۔ کمپیوٹر میں ہم یہ ویلیوز کو  $-\max(\text{int})$  اور  $\max(\text{int})$  سے پٹ کر سکتے ہیں۔ اب ویلیوز ہمارے پاس ڈیٹا ایٹم میں نہیں ہوں گے۔ ہمارے ڈسکشن کیلئے یہ  $-\infty$  اور  $+\infty$  Notations ہیں۔  $-\infty$  کی ویلیو ڈیٹا ایٹم کی ویلیو سے کم ہوتی ہے جبکہ  $+\infty$  کی ویلیو ڈیٹا ایٹم کی ویلیو سے زیادہ یعنی بڑی ہوتی ہے۔

$S_0$  کے نمبرز Non-Decreasing آرڈر میں ہوتے ہیں۔  $S_0$  پہلی لسٹ ہے جس میں تمام تر Keys موجود ہوتی ہیں۔ اب ہم لسٹ سے کچھ نوڈز لیتے ہیں اور انکو لنک کرتے ہیں۔ یہ نوڈز ہر فور تھ نوڈ نہیں ہوگا یہ کوئی بھی رینڈم سلیکشن ہو سکتی ہے۔



ہم دیکھ سکتے ہیں کہ  $S_1$  لسٹ سب سیٹ ہے لسٹ  $S_0$  کا۔ ایس 1 میں ہم نے نوڈز 34، 31، 23، اور 64 کو سلیکٹ کیا ہے۔ ان نوڈز کو ہم نے Randomly سلیکٹ کیا ہے نہ کہ کسی آرڈر کے۔ اب ہم نے ایس 1 سے لسٹ ایس 2 بنائی ہے جس میں ہم نے لسٹ ایس 1 ویلیو 31 کو سلیکٹ کی ہے۔ اب یہاں پر ایڈیشنل پوینٹرز Move

ہوں گے جو کہ  $-\infty$  to 31 اور 31 to  $+\infty$  ہیں۔ اب لسٹ  $S_3$  کو

بیانا گیا جو کہ سب سیٹ ہے  $S_2$ ۔ اب جیسے کہ لسٹ  $S_2$  میں ایک ایلیمینٹ 31 ہے اس لئے لسٹ  $S_3$  میں صرف اسپیشل Keys ہوں گی۔ ان تمام لسٹس میں پوینٹرز استعمال ہوئے ہیں اور ان پوینٹرز سے نوڈز کو لسٹ  $S_0$  میں لنک کیا گیا ہے۔ پھر ایڈیشنل پوینٹرز سے ان لسٹس کو دوسری لسٹوں کے ساتھ لنک کیا گیا ہے۔ یہ لسٹس انرست آپریشن کی مدد سے بنائی گئی ہیں۔ ان لسٹ میں ڈیٹا ڈپلیکیٹ نہیں گیا ہے اور نہ ہی ڈیٹا اسٹرکٹ کیا گیا ہے، جو ایلیمینٹس دو دو بار شو ہو رہے ہیں وہ صرف ایڈیشنل

پوینٹرز کی وجہ سے شوہر رہے ہیں۔ مثلاً ہمارے نوڈ 23 ہے اب اس کو یہ نہ سمجھ کے دو بار لکھا ہوا ہے اصل میں اس کے دو نیکسٹ پوینٹرز ہیں، ایک پوائنٹر 23 کو 26 کی طرف پوائنٹ کر رہا ہے اور دوسرا پوائنٹر 23 کو ویلیو 31 کی پوائنٹ کر رہا ہے۔ اسی طرح دوسرے ایلیمنٹس بھی ایسے ہی ایڈیشنل پوینٹرز کے ساتھ دو بار دیکھائی دے رہے ہیں۔ سکیپ لسٹ میں سرچ کرنے کیلئے کچھ رولز ہوتے ہیں۔ جو درج ذیل ہیں۔

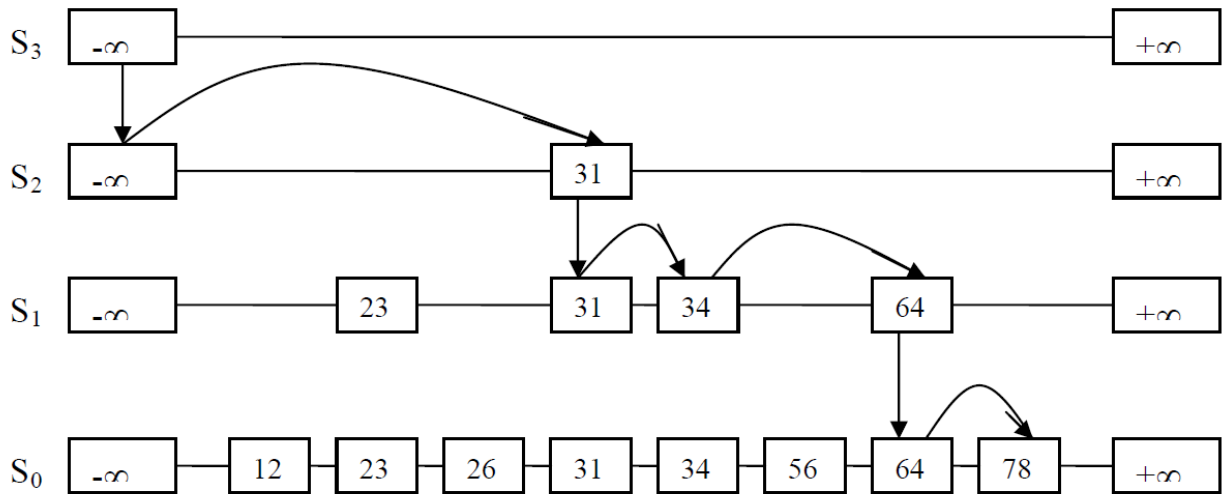
We search for a key  $x$  in the following fashion:

- We start at the first position of the top list
- At the current position  $p$ , we compare  $x$  with  $y \leftarrow \text{key}(\text{after}(p))$
- $x = y$ : we return  $\text{element}(\text{after}(p))$
- $x > y$ : we “scan forward”
- $x < y$ : we “drop down”
- If we try to drop down past the bottom list, we return  $\text{NO\_SUCH\_KEY}$

سکیپ لسٹ میں سرچ کرنے کیلئے لسٹ کے ٹاپ سے فرسٹ پوزیشن سے سرچ شروع کی جاتی ہے۔ مثال کے طور پر ہم ٹاپ سے مردلے لیتے ہیں ہمارے اوپر کے مثال میں S3 لسٹ ہے۔ کرنٹ پوزیشن ایلیمنٹ P ہوگی۔

فرض کریں اب ہم ویلیو 78 کو سرچ کرنا چاہتے ہیں، سب سے پہلے ٹاپ لسٹ یعنی S3 سے سرچ شروع ہوگی۔ 78 کی ویلیو کرنٹ نوڈ ہوگی اور اس کو P سے ڈی نوٹ کیا گیا ہوگا۔ اب ہم نوڈ میں سو ویلیو کو دیکھیں گے جو P کے بعد ہوگی۔ ہمارے کیس میں یہ ویلیو  $+\infty$  ہوگی اب  $+\infty$  کی ویلیو 78 سے بڑی ہے ہم لسٹ S2 پر ڈراپ ڈون کرے گے۔ جو ڈراپ down ہوا ہے وہ ور ٹیکل ہوا ہے۔ اب ایس 2 میں ہمارے پاس

ویلیو 31 ہے جو کہ سرچنگ ویلیو 78 سے چھوٹی ہے اب یہاں نہ ڈراپ down ہوگا نہ ڈراپ اپ ہوگا بلکہ سکیں فورڈ ہوگا جو کہ سکیپ سرچنگ رولز میں لکھا ہے۔ سکیں فورڈ ہونے کے بعد ہماری پاس ویلیو  $+\infty$  آئی جو کہ ایک بار پھر سرچنگ ویلیو 78 سے بڑی ہے، اب یہاں ایس 2 سے ایس 1 پر ڈراپ down ہوگا۔ ایس 1 میں ہمارے پاس کرنٹ پوزیشن کی ویلیو 34 ہے، یہ ویلیو ہماری سرچنگ ویلیو 78 سے چھوٹی ہے، اب یہاں پھر سکیں فارورڈ ہوگا، اب ہمارے پاس اگلی ویلیو 64 ہے جو کہ سرچنگ ویلیو سے چھوٹی ہے اب پھر سے نیکسٹ نوڈ کو دیکھا جائے گا جو ہمارے پاس  $+\infty$  اور یہ ویلیو ہماری سرچنگ ویلیو سے زیادہ ہے، تو اب یہاں ایس 1 سے ایس 0 پر ڈراپ down ہوگا۔ ایس 0 میں نیکسٹ ویلیو 64 کی ہے جو کہ 78 سے چھوٹی ہے، اور سکیں فارورڈ ہوگا اور ہم سرچنگ ویلیو 78 تک پہنچ جائے گے۔



## Insertion in Skip List

جب سکیپ لسٹ میں ہم ایٹم add کرتے ہیں تو ہم Randomized Algorithm کا استعمال کر کے انسیرشن کرتے ہیں۔

پہلا Algorithm یہ ہے کہ ہم تب تک Coin سے ٹاس کرتے ہیں جب تک ٹیل نہیں آجاتا، اور پھر ہم اس کو  $i$  کے نمبر سے ڈی نوٹ کر دیتے ہیں، جتنی بار بھی ہیڈ آیا ہوتا ہے۔ مثلاً اگر چار بار ٹاس کریں وار تین بار ہیڈ آئے تو اس  $i$  کو 3 سے ڈی نوٹ کرے گے۔ اس میں probability بچاس فیصد ہوتی ہے کہ ٹیل آیا گا یا ہیڈ۔ ہیڈ جتنی بار آیا گا اس کو  $i$  سے ڈی نوٹ کر دیا جائے گا۔ مثلاً اگر ہیڈ آیا تو  $i$  کی ویلیو 1 ہو جائے گی اور پھر ٹاس ہو گا اگر پھر ہیڈ آیا تو  $i$  کی ویلیو میں 1 جمع ہو جائے گا۔ یہ کاؤنٹ اس تک ہو گا جب تک ٹیل نہیں آجاتا۔ جب ٹیل آجائے گا تو ٹاس ہونا سٹاپ ہو جائے گا اور  $i$  کی ویلیو نوٹ کر لی جائے گی۔ اس کے بعد Algorithm کا دوسرا پارٹ کے کچھ رولز ہیں۔

- If  $i \geq h$ , we add to the skip list new lists  $S_{h+1}, \dots, S_{i+1}$ , each containing only the two special keys

یہاں ہم  $i$  کی ویلیو جو کہ ہیڈ کے کاؤنٹ ہونے کے بعد ہمارے پاس آئی ہے اس کو ایچ سے Compare کرے گے۔ اب اگر  $i$  بڑی یا برابر ہو ایچ کے تو ہم نئی لسٹ کو سکیپ لسٹ میں add کر دیتے ہیں، یہ لسٹ  $S_{h+1}, \dots, S_{i+1}$  ہوگی۔ مثلاً اگر  $i$  کی ویلیو 8 ہو اور ایچ کی ویلیو 4 ہو تو ہم ایڈیشنل lists 5، 6، 7، 8 اور 9

کو add کرے گے۔ یہ Lists دو خاص Keys پر مشتمل ہوں گی جو کہ  $+\infty$  and  $-\infty$  ہے۔

## Deletion from Skip List

deleting میتھڈ میں سب سے پہلے فائنڈ میتھڈ استعمال ہوگا۔ سکیپ لسٹ سے ایٹم کو remove کرنے کیلئے پروسیجر ہوتا ہے۔

We search for  $x$  in the skip list and find the positions  $p_0, p_1, \dots, p_i$  of the items with key  $x$ , where position  $p_j$  is in list  $S_j$ . This means that we look for the links of the item to be removed. We know that an item in the list has necessarily link in the list  $S_0$ . Moreover it may have links in other lists up to  $S_i$  or say  $S_j$ . After this, we remove positions  $p_0, p_1, \dots, p_i$  from the lists  $S_0, S_1, \dots, S_i$ . We remove all the lists except the list containing only the two special keys.

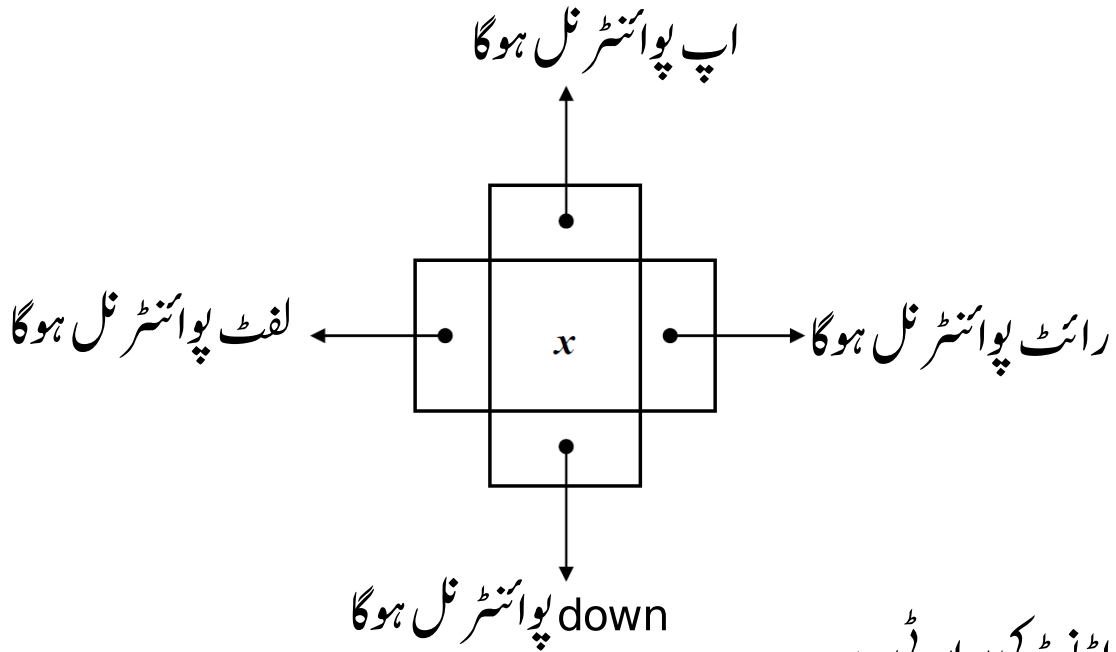
Let's consider the skip list shown in the figure below.

## Lecture No. 41

### Quad Node

اس میتھڈ میں اررے کے پوائنٹرز نہیں ہوتے ہیں۔ اس میں چار نیکسٹ پوائنٹرز ہوتے ہیں۔ کوآڈ نوڈ کو سمجھنے کیلئے پہلے اس کی پراپرٹی کو سمجھتے ہیں۔ کوآڈ نوڈ کیا سٹور کرتی ہے؟ یہ درج ذیل چیزیں سٹور کرتی ہیں۔  
ایٹم

نوڈ سے پہلے کا لنک  
 نوڈ کے بعد کا لنک نوڈ کے نیچھے لنک  
 نوڈ کے اوپر کے لنک کو سٹور کرتا ہے۔



یہ کوڈ نوڈ کی پراپرٹی ہے۔

اس میں ہمیں مختلف لیول پر Key کو کاپی کرنے کی ضرورت پڑی گی۔ اس میں ہمارے  
 اررے کے پوائنٹرز نہیں ہیں۔ اس لئے اس میں مختلف طریقے اپنا کر ملٹی لیول نوڈ کو  
 سکیپ لسٹ میں create کیا جاتا ہے۔ اس میں 6 لیولز درکار ہوتے ہیں، اب ہمیں 6  
 نوڈز create کرنے پڑے گے اور x کا ڈیٹا ایٹم کاپی کر کہ ان سب نوڈز میں انرسٹ کرنا  
 ہوگا۔ یہ insertion لنک لسٹ اسٹرکچر سے ہوگی۔

کوڈ نوڈ کے فائدے کیا ہیں؟

اس نوڈ میں ہم نیکسٹ پوائنٹز کا رے Allocate نہیں کرتے ہیں۔ ہر لسٹ نوڈ چار پوائنٹرز پر مشتمل ہوتا ہے۔ کوآ نوڈ کی فیکٹری وہ نوڈ ریٹرن کرے گی جس کے چار پوائنٹرز ہوتے ہیں۔ یہ ہماری ذمہ داری ہوگی کہ ہم نوڈ کو اپ، باٹم، لفٹ اور رائٹ پوائنٹرز کو دوسرے نوڈز سے لنک کرے۔ Previous پوائنٹر کی مدد سے ہم لسٹ میں backward بھی Move کر سکتے ہیں۔ یہ بیک move ہو سکتا ہے ڈبل سکیپ لسٹ کھلائے۔

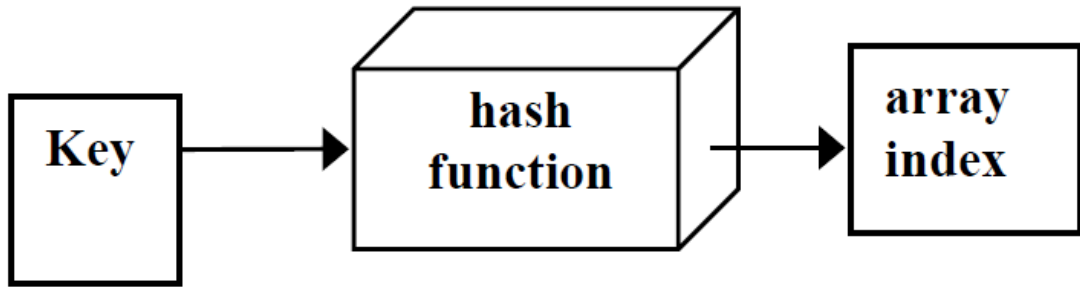
## Performance of Skip Lists

سکیپ لسٹ جب بنائی جاتی ہے تو کچھ میموری بھی ایٹمز کیلئے استعمال ہوتی ہے۔ lowest لیول اور دوسرے لیول لسٹ کو create کرنے کیلئے بھی میموری درکار ہوتی ہے۔ جتنے نمبر اف ایٹمز ہوں گے اتنی ہی میموری درکار ہوگی۔ این ایٹم کیلئے این میموری لوکیشن چاہئے ہوں گے۔ مثلاً اگر ہمارے 100 ایٹم ہوئے تو ہمیں میموری کی 100 لوکیشن چاہئے ہوں گی۔ پھر ہمیں نیکسٹ پوائنٹرز کیلئے میموری چاہئے ہوگی۔ نیکسٹ پوائنٹرز کیلئے 2 این میموری لوکیشن چاہئے ہوں گی۔

insertion، deletion اور سرچ Key کے بیس پر فارم کی جاتی ہے۔ نوڈ میں ہمارے پاس key اور ڈیٹا ایک ساتھ ہوتا ہے۔

## Hashing

ہاشنگ ایک Alogrithmic پروسیجر اور Methodology ہے۔ یہ نیا ڈیٹا اسٹرکچر نہیں ہے۔ یہ ایک طریقہ ہے پہلے سے موجود یعنی Existing ڈیٹا اسٹرکچر کو استعمال کرنے کا۔ ٹیبل کے میٹھڈ فائنڈ، انسرت اور remove کا انسٹنٹ ٹائم لے گے۔ ہاشنگ میں Internally ارے استعمال ہوگی۔ یہ ارے سٹیٹک یا ڈائنامک ہو سکتی ہے۔ مگر ہم ڈیٹا کو Consecutive لوکیشن پر سٹور نہیں کرے گے۔ سٹوریج کی جگہ کو Key اور ہاشنگ فنکشن سے Calculate کیا جائے گا۔ اب ہاش فنکشن ہمارے لئے نیا ہے۔



اب پہلے ہمارے پاس Key ہے، جس کو ہم ہاش فنکشن کو پاس کرے گے۔ ہاش فنکشن ایک Mathematical فنکشن ہے جو ارے انڈکس کو ریٹرن کرے گا۔ یا دوسرے الفاظ میں ہم کہہ سکتے ہیں کہ انٹیجر نمبر ریٹرن ہوگا۔ یہ نمبر کسی رینج میں ہوگا مگر Sequence میں نہیں ہوگا۔

اب اگر ہم ایمپلائڈ ڈیٹا کو انسرت کرنا چاہیں تو اس کی key ایمپلائڈ آئی ڈی ہوگی۔ ہم یہ Key ہاش فنکشن کو پاس کر دے گے اور ہاش فنکشن ایک انٹیجر ریٹرن کر دے گا۔ جو نمبر ہاش فنکشن ریٹرن کرے گا ہم اس کو ارے انڈکس کے طور پر استعمال کرے

گے۔ اب ہم ایمپلائی کا ڈیٹا اس ارے انڈکس پر add کرے گے۔ انسرت سٹوریج کے پلیس کو Calculate کرے گا اور اس کو ٹیبل نوڈ میں add کر دے گا۔

جب ہم نیا ڈیٹا ایٹم حاصل کر لے گے تو اسکی key ہاش فنکشن کی مدد سے Generate ہو جائے گی ارے کے انڈکس کو حاصل کرنے کیلئے۔ اب اس ارے انڈکس کو استعمال کر کے ہم ارے میں ڈیٹا انسرت کرے گے۔ اگر ہمارا ہاش فنکشن فاسٹ ہوگا تو انسرت آپریشن بھی فاسٹ ہوگا اور صرف ایک سٹیپ سے اپنا کام پر فارم کر دے گا۔

فائنڈ آپریشن ہاشنگ میں کانسنٹ ٹائم لیتا ہے۔ Remove میتھڈ سٹوریج کے پلیس کو Calculate کر کے اور کو Null سیٹ کر دیتا ہے۔ یہ key کو ہاش فنکشن کو پاس کرتا ہے، ارے انڈکس کو حاصل کرنے کیلئے۔ اب remove میتھڈ اس ارے انڈکس کو استعمال کر کے ایلیمینٹس کو remove کر دے گا۔

ایک بات کو خیال رکھیں کہ ہاش فنکشن ارے انڈکس کو Generate کرتا ہے اور پھر ہم اسی ارے انڈکس کو استعمال کر کے ٹیبل ارے میں ڈیٹا انسرت کرتے ہیں۔

## Lecture No. 42

### Collision

Collision کا استعمال تب ہوتا ہے جب دو یا دو سے زیادہ Keys ایک جیسا انڈکس پر وڈیوس کر دے۔ مطلب جب فنکشن ایک ہی ویلیو 2 ارے کو دے دیتا ہے تو اس کو Collision کہتے ہیں۔ Collision کو حل کرنے کے تین طریقے ہمارے ہینڈ آؤٹ میں دیئے گئے ہیں۔ **Collision اوپن ایڈرسنگ اور کلوز ہاشنگ بھی کہلاتا ہے۔** اوپن کا ورڈ ایڈرسنگ کے ساتھ استعمال ہوتا ہے اور کلوز ورڈ ہاشنگ کے ساتھ۔

The definition of collision is:

“When two values hash to the same array location, this is called a *collision*”

پہلا: اب اگر ہاش فنکشن نے کوئی ویلیو کسی ایٹم کو دے دی ہو اور پھر ہم اسی ویلیو پر کوئی اور ایٹم رکھنا چاہئے تو ہاش فنکشن پہلی والی ویلیو کو چینج نہیں کرے گا کیونکہ ہاش فنکشن فرسٹ come فرسٹ serve کی بنیاد پر کام کرتا ہے۔ اس نئے ایٹم کو ویلیو دینے کیلئے پہلے ہمیں خالی لوکیشن دیکھنے ہوگی پھر ہم ویلیو کو سٹور کرے گے۔ Collision کو Solve کرنے کے تین طریقے ہیں۔ پہلا لینئر پروبنگ، دوسرا: Quadratic پروبنگ اور تیسرا: لنک لسٹ Chaining۔

دوسرا: ہم یہ کر سکتے ہیں کہ ہاش کا کوئی اور فنکشن استعمال کرے، جب ہم نے پرائمری ہاش فنکشن میں ڈیٹا یا ایٹم پاس کی تو ہمیں ایک ویلیو ریٹرن میں ملی۔ مثال کے طور پر ہمیں ویلیو 6 ملی اور اس ویلیو 6 پر پہلے سے ہی کچھ سٹور ہوا ہے، اب ہم دوسرا ہاش فنکشن کال کرے گے اگر اس بار ہمیں پھر سے کوئی ایسی ویلیو ملی جس پر کوئی اور ایٹم سٹور ہے تو ہم پھر تیسرا ہاش فنکشن کال کرے اور اگر ویلیو ایسی ملی جو Empty تو ہم

اپنا ڈیٹا سٹور کر لے گے ورنہ پھر چوتھا ہاش فنکشن کال کرے گے، کالنگ تب تک ہوگی جب ہمیں کوئی خالی ویلیو نہ مل جائے جہاں ہم ڈیٹا کو سٹور کر سکیں۔

تیسرا حل: یہ ہے کہ ہم اررے لوکیشن کو لنکڈ لسٹ کے ویلیوز کے ہیڈر کے طور پر استعمال کرے جو کہ اس لوکیشن کو ہاش ہوئی ہوتی ہیں۔ اس سو لیوشن میں بھی ہم ڈیٹا ڈائریکٹ اررے میں سٹور نہیں کرے گے۔ ہماری اررے پوینٹرز کی اررے ہوگی ٹیبیل نوڈ کی۔ ہم لسٹ نوڈ create کرے گے اور پھر اس میں ڈیٹا سٹور کرے گے۔ اررے کا پوائنٹر نوڈ کی طرف ہوگا۔ اگر ہم لوکیشن 6 پر کوئی ڈیٹا سٹور کرنا چاہتے ہیں تو ہم لوکیشن 6 پر پوائنٹر سٹور کر دے گے اور یہ پوائنٹر اس نوڈ کو پوائنٹ کرے جس نے ڈیٹا ہولڈ کیا ہوگا۔ اب اگر کوئی اور ڈیٹا پوزیشن 6 پر ہوگا تو ہم لسٹ نوڈ Create کرے گے اور اس کو پچھلے نوڈ کے ساتھ ایچ کر دے گے۔

## Linear Probing

Collision کے Resolution کی جو اسٹریٹجی ہوتی ہے اس کو Linear پروبنگ کہتے ہیں۔ یہ اررے کو Sequentially سکین کرتا ہے تاکہ اررے میں Empty سیل کو سرچ کیا جاسکے۔ Collision میں ہم لوکیشن 6 پر کوئی ایٹم رکھنے لگے تھے تو ہمیں پتا چلا کہ وہ لوکیشن تو پہلے ہی کسی اور ایٹم کو دی جا چکی ہے، اس لئے ہمیں Collision

ملا۔ اور اس Collision کو ختم کرنے کیلئے پھر ہم نے اررے میں خالی لوکیشن کو تلاش کیا۔ اب اررے میں خالی لوکیشن کو تلاش کرنے کو لینتیر پر اہلم کہتے ہیں۔

|     |         |
|-----|---------|
| ... |         |
| 141 |         |
| 142 | robin   |
| 143 | sparrow |
| 144 | hawk    |
| 145 | seagull |
| 146 |         |
| 147 | bluejay |
| 148 | owl     |
| ... |         |

اس اررے میں ہمارے پاس کچھ پرندوں کے نام سٹور ہیں، ہمارے اس اررے میں کچھ انڈکس پر ڈیٹا سٹور ہے جو کہ 142 سے لیکر 148 تک کے انڈکس ہیں جہاں ڈیٹا سٹور ہے۔ فرض کرے کہ ہم نے ایک اور ایلیمینٹ یعنی سیگل اس ہاش ٹیبل میں add کرنا ہے تو کیسے کرے گے؟ ہم Seagull کو Key کے طور پر استعمال کرے گے اور اس کو ہاش کوڈ کو پاس کرے گے۔

$$\text{hashCode}(\text{"seagull"}) = 143$$

اب جو لوکیشن ہم نے سیگل کو سٹور کرنے کیلئے دی ہے وہ 143 ہے جو پہلے سے ہی استعمال میں ہے۔ اب فنکشن نیکسٹ لوکیشن پر جائے گا جس کیلئے فنکشن  $i+1$  کرے گا اب نیکسٹ لوکیشن 144 ہو جائے گی، یہ لوکیشن بھی پہلے سے استعمال میں ہے،

اب ہاش فنکشن  $2 + 144$  کر کے نیکسٹ لوکیشن کو چیک کرے گا۔ اب ہماری لوکیشن 146 ہو جائے گی جو کہ خالی ہے اور ہم اپنا ڈیٹا یہاں سٹور کر دے گے۔ اس انسیرشن میں ہمیں دو بار Collision کا سامنا کرنا پڑا، پہلی بار 143 انڈکس پر اور دوسری بار 144 انڈکس پر۔

اب اگر ہم کوئی ایسا ڈیٹا انسرت کرنا چاہیں جو پہلے سے ہی ہاش ٹیبل میں موجود ہے تو تب کیا ہوگا؟ فرض کریں ہم اس ہاش ٹیبل میں Hawk کو دوبارہ انسرت کرنا چاہتے ہیں۔ اب سب سے پہلے ہم یہ دیکھیں گے کہ جو ڈیٹا ہم انسرت کرنا چاہتے ہیں وہ پہلے سے add ہے یا نہیں؟ اس کیلئے ہم ہاش فنکشن کو کال کرے گے۔

`hashCode("hawk") = 143`

ہم نے انڈکس 143 دیا ہے اور ہم جانتے ہیں کہ اس میں پہلے سے ڈیٹا سٹور ہے، اور اگر اوپر دیئے گئے ٹیبل میں دیکھیں تو 143 انڈکس پر جو ڈیٹا سٹور ہے وہ Sparrow ہے جب کہ اس فنکشن میں ہم نے یہ بتایا ہے کہ انڈکس 143 پر hawk کو سٹور کرو۔ اب اس پوانٹ پر ہمیں فرض کرے نہیں پتا کہ انڈکس 143 پر کون سا ڈیٹا سٹور ہے اور نہ ہمیں یہ پتا ہے کہ hawk پہلے سے ہمارے ڈیٹا میں سٹور ہے۔ اب ہم لینئر پرابلم کرنے جارہے ہیں۔ کیونکہ ہو سکتا ہے کہ ڈیٹا پہلے سے سٹور ہو۔ اس لئے فیصلہ کرنے سے پہلے ہم مزید اس کو چیک کریں گے۔ اب ہم انڈکس کو چیک کرے گے جب

ہم نے انڈکس 143 کو چیک کیا تو وہاں تو ڈیٹا ہے اب ہم نے نیکسٹ انڈکس کو چیک کیا تو وہاں بھی ڈیٹا ہے مگر وہاں ڈیٹا وہ ہے جو ہم سٹور کرنا چاہتے ہیں۔ اب ہمارے پاس یہاں دو آپشن ہیں، ایک تو یہ کہ hawk پہلے سے سٹور ہے کچھ بھی نہ کرے، دوسرا یہ کہ ہو سکتا ہے کہ User ڈیٹا کو Modify کرنا چاہتا ہے جو hawk سے منسلک ہے۔

ہمارے انٹرئل آررے کا سائز فکس ہوتا ہے اور ہم نے اس کو کانسنٹنٹ ٹیبیل سائز میں سٹور کیا ہے۔

ہاشنگ ایک بہت وسیع ریسرچ فیلڈ ہے، جو ہاش فنکشن، سٹوریج اور Collision کے ایشوز کو Cover کرتی ہے۔ ہاشنگ میں انسرت، ڈیلیٹ یعنی Remove اور فائنڈ آپریشنز کانسنٹنٹ ٹائم لیتے ہیں، کانسنٹنٹ ٹائم سے مطلب ہے کہ اگر ڈیٹا سائز بعد میں بڑھ بھی جائے تو بھی ہاشنگ اسٹریٹجی میں ٹائم کانسنٹنٹ ہی لگے گا، البتہ اگر Collision کا مسئلہ شروع ہو جائے پھر یہ ٹائم کانسنٹنٹ نہیں رہے گا خاص طور پر لینئر پر ایلیم میں ہمیں آررے کو سورت کر کے ڈیٹا انسرت کرنا پڑتا جس میں ٹائم زیادہ لگے گا

## Lecture No. 43

# Hashing Animation

ہاشنگ animations جاوا پروگرام میں provide کی گئی ہیں۔ جاوا لینگویج اس لئے بھی زائدہ Famous ہے کہ انٹرنیٹ کیلئے facilities فراہم کرتی ہے۔ ہاشنگ Animation براؤزر میں شو کی جائے گی۔

کو مپلیر ہاش ٹیبل کو استعمال کرتا ہے تاکہ جو Variables اس میں Declared ہوئے ہیں انکو ٹریک کیا جائے۔ کو مپلیر ہاش ٹیبل کو ہی استعمال کر کہ Symbol ٹیبل کو Implement کرتا ہے۔ کو مپلیر جب پروگرام کو پروسس کرتا ہے تو Symbol ٹیبل میں Variables کو رکھ دیتا ہے۔ ہاش ٹیبل کو آن لائن سپلنگ چیکنگ کیلئے بھی استعمال کی جاسکتا ہے۔

ہاش ٹیبل بہت ہی مفید ثابت ہوتا ہے اگر بہت سری Searches ایک Reasonable اور stable ٹیبل میں چاہئے ہوتی ہو۔

اگر بہت ساری Insertions اور Deletions کرنے کی ضرورت بار بار پیش آئے تو ایسے صورت میں ہاش ٹیبل زیادہ مفید نہیں ہوگا۔ اس کیس میں اے وی ایل ٹری کا استعمال کرنا زیادہ بہتر ہے۔

سورٹنگ کا مطلب ہے ہوتا ہے کہ ڈیٹا کو ایک خاص آرڈر یا Sequence میں پٹ کرنا۔ سورٹنگ کمپیوٹر 80/90% اپلیکیشن میں Useful ہے۔

# Elementary Sorting Algorithms

تین قسم کے سورٹنگ Algorithm ہوتے ہیں۔

سلیکٹنگ سورٹ

انسیرشن سورٹ

بل سورٹ

سلیکشن سورٹ:

سب سے چھوٹا ایلیمنٹ فائنڈ کرتا ہے

پھر اس کو سب سے پہلی پوزیشن پر پٹ کر دیتا ہے

پھر دوسرا چھوٹا ایلیمنٹ فائنڈ کرتا ہے

اور پھر دوسرے چھوٹے ایلیمنٹ کو دوسرے نمبر پر پٹ کر دیتا ہے۔

سلیکشن سورٹنگ میں سب سے پہلے چھوٹے ایلیمنٹ کو فائنڈ کر کے اس کو پہلی پوزیشن

پر پٹ کیا جاتا ہے اور پھر دوسرے سب سے کم ویلیو کو سرچ کر کے اس کو پٹ کیا جاتا

ہے۔

|    |   |   |    |
|----|---|---|----|
| 19 | 5 | 7 | 12 |
|----|---|---|----|

ہمارے پاس یہ ایک اررے ہے جس میں کچھ ایلیمینٹس ہیں اور ان کو اب سلیکٹنگ سورٹنگ کی مدد سے سورٹ کرنا ہے۔ یہاں سب سے اہم بات ہے یہ کہ اس ٹائپ کے سلیکشن سرچ میں ہم سب سے پہلے سٹارٹ کے انڈکس سے سب سے کم ایلیمینٹ کو سرچ کرنا شروع کرتے ہیں۔ ہمارا جو اررے میں سب سے پہلا انڈکس 0 ہوتا ہے تو ہم اس 0 انڈکس سے سب سے کم ویلیو کے ایلیمینٹ کو سرچ کرنے لگے۔

انسیرشن سورٹ:

یہ اررے کے پہلے 2 ایلیمینٹس کو دیکھتا ہے اگر وہ آرڈر میں نہ ہوئے تو انکو Sawp کر دیتا ہے، پھر تیسری ایلیمینٹ کو دیکھتا ہے اور اگر وہ پروپر آرڈر میں ہو تو پھر پہلے 2 ایلیمینٹس بھی اس میں شامل کر کے پروپر آرڈر میں انسرت کر دے گا۔

اب جب پہلے تین ایلیمینٹس پروپر آرڈر میں انسرت ہو جائے گے تو یہ چوتھے ایلیمینٹ کو دیکھے گا اور اب پہلے تین ایلیمینٹ کو بھی چیک کر کے جو چھوٹا ایلیمینٹ ہے اس کو پروپر آرڈر میں انسرت کر دے گا۔ فرض کرے ہمارے پاس پہلے دو ایلیمینٹس 9، 8 تھے، اب سب سے پہلے کو انکو سورٹ کر کے پروپر آرڈر یعنی 9، 8 کر کے سورٹ کیا جائے گا اور پھر پٹ کر دیا جائے گا، اب اگر تیسرا ایلیمینٹ 2 ہو تو اب پہلے کے 9، 8 کو ملا کر نمبر دو کو سب سے پہلی پوزیشن پر پٹ کر دیا جائے گا، تو آرڈر 9، 8، 2 ہو جائے گا اب اگر چھوٹا ایلیمینٹ 10 ہو تو پہلے کے تین ایلیمینٹس 9، 8، 2 کو ملا کر چوتھے ایلیمینٹ کو اس کے پروپر اور نمبر 10، 9، 8، 2 پر سورٹ کر دیا جائے گا۔

|    |    |   |   |
|----|----|---|---|
| 19 | 12 | 5 | 7 |
|----|----|---|---|

اس اررے میں دیکھا جائے تو پہلے دو ایلیمنٹس 19 اور 12 ہیں اب سب سے پہلے ان کو Swap کی جائے گا ان کو Swap کر کے 12، 19 کر دیا جائے گا پھر تیسرے ایلیمنٹ کو دیکھا جائے گا جو کہ 5 ہے، اب ایلیمنٹ 5 کو پہلے دو ایلیمنٹس جو پہلے سے سورت کر دیئے گئے ہیں ان سے Compare کیا جائے گا، اب 5 کا جو ایلیمنٹ ہے وہ 12 اور 19 سے کم ہے اس لئے اس کو Swap کر کے پہلی پوزیشن پر لیا جائے گا۔ اسی طرح ایلیمنٹ 7 کو اب پہلے تین سورتڈ ایلیمنٹس سے Compare کر کے اس کے پروپر آرڈر میں پٹ کر دیا جائے گا۔

ببل سورت:

اس میتھڈ میں اررے میں سب سے چھوٹے نمبر کو اپ وارڈ لیکر آیا جاتا ہے اور بڑے نمبر کو downword لیکر جایا جاتا ہے۔ اگر اررے ور ٹیکل ہوگی تو ہم ببل سورت کرتے ہیں۔ چھوٹے نمبر کو اوپر کی طرف اور بڑے نمبر کو نیچے کی طرف کرنے کو ببل سورتنگ کہتے ہیں۔

ببل سورت کا Algorithm کہتا ہے کہ جو Neighbor نمبر ہوتے ہیں انکو ایکسچینج کرو تاکہ چھوٹا نمبر اوپر اور بڑا نمبر نیچے چلا جائے۔ اور اسی سٹیپ کو تب تک ریپیٹ کرو

جب تک سب سے چھوٹا نمبر Upword میں نہ آجائے اور بڑا نمبر downword میں نہ چلا جائے۔ اس Algorithm میں Pair wise سوائپنگ کی جاتی ہے۔

|    |    |   |   |
|----|----|---|---|
| 19 | 12 | 5 | 7 |
|----|----|---|---|

اس اررے میں اب سب سے پہلے 19 اور 12 کے Pair کو Swap کیا جائے گا، اور Swap ہونے کے بعد 5 پہلی پوزیشن پر آجائے گا اور 19 دوسرے اور 12 تیسرے نمبر پر سٹور ہوگا، اب دوسرے Pair کو دیکھا جائے جس میں اب ہمارے پاس 19 اور 12 ہیں، اب جب اس Pair کو Swap کی جائے گا تو 12 پہلے آجائے گا اور 19 بعد میں چلا جائے گا۔

|   |    |   |    |
|---|----|---|----|
| 5 | 12 | 7 | 19 |
|---|----|---|----|

تو ہمارا اررے کچھ یوں ہوگا اب تیسرے Pair کو swap کیا جائے گا جس میں اب ہمارے پاس 7 اور 19 ہے تو 7 پہلے آجائے گا اور 19 بعد میں ہوگا۔

|   |    |   |    |
|---|----|---|----|
| 5 | 12 | 7 | 19 |
|---|----|---|----|

اب پھر سے سوائپنگ ہوگی اور یہ تب تک ہوگی جب تک سب سے چھوٹا ایلیمینٹ سب سے پہلے یا Upword میں آجائے اور سب سے بڑا ایلیمینٹ سب سے آخری یعنی Downword میں نہ چلا جائے۔

بیل سورٹنگ Algorithm میں Outer لوپ اور انر لوپ ہوتے ہیں۔ Outer لوپ نمبر اف ایلیمینٹس ٹائم Executes ہوتا ہے جبکہ انر لوپ پہلی بار نمبر اف ٹائمز Executes اور پھر N-1 ہوگا پھر N-2 یعنی کم ہوتا جائے گا۔

| N       | $N^2$       | $N \log_2 (N)$ |
|---------|-------------|----------------|
| 10      | 100         | 33.21          |
| 100     | 10000       | 664.38         |
| 1000    | 1000000     | 9965.78        |
| 10000   | 100000000   | 132877.12      |
| 100000  | 10000000000 | 1660964.04     |
| 1000000 | 1E+12       | 19931568.57    |

اس چارٹ سے ہو سکتا ہے کہ MCQ's آجائے اس لئے ایک بار فارمولہ دیکھ لیں۔

## $N \log_2 (N)$ Algorithms

Algorithm کا درج ذیل Algorithm کو include کرتا ہے۔  $N \log_2 (N)$

پہلا : Merge سورٹ

دوسرا : کوئیک سورٹ

تیسرا : ہیپ سورٹ

یہ تینوں Algorithm کام کرنے کیلئے Proportional ٹائم لیتے ہیں جو کہ  $n \log_2 n$  کہلاتا ہے۔ یہ تینوں Algorithm تقسیم یعنی divide اور Conquer کے کیٹگری کے انڈر فال کرتے ہیں۔ یہ تقسیم اور فاتح کی اسٹریٹیجی جنگ میں بہت جانی جاتی ہے۔ اس فلسفہ کے مطابق اپنے دو شمنوں کو حصوں میں تقسیم کرو اور پھر ان کو فاتح کرو۔ کسی بھی چیز کو حصوں میں تقسیم کرنے کے بعد اس کو فاتح کرنا آسان ہو جاتا ہے۔ یہی فلسفہ ان تینوں Algorithm پر Apply ہوتا ہے۔ اس divide اینڈ conquer اسٹریٹیجی کو سمجھنے کیلئے ایک مثال دیکھتے ہیں۔

|    |    |   |   |   |    |   |   |
|----|----|---|---|---|----|---|---|
| 10 | 12 | 8 | 4 | 2 | 11 | 7 | 5 |
|----|----|---|---|---|----|---|---|

ہمارے پاس یہ ایک انسورٹڈ ارے ہے، Unsorted سے مرد کے ڈیٹا ترتیب سے نہیں ہے اس ارے میں۔ اب ہم اس ارے کو دو حصوں میں تقسیم کریں گے۔

|    |    |   |   |   |    |   |   |
|----|----|---|---|---|----|---|---|
| 10 | 12 | 8 | 4 | 2 | 11 | 7 | 5 |
|----|----|---|---|---|----|---|---|

اب اس اررے کے ہمارے پاس دو حصے آگئے ہیں، ہم نے دونوں پارٹس کو الگ سے سورٹ کر دیا ہے۔ فرض کریں ہم نے یہ سورٹنگ ایلیمینٹری سورٹ Algorithm سے کی ہے۔ اب ان پارٹس کو کچھ یوں سورٹ کیا جاسکتا ہے۔

|   |   |    |    |   |   |   |    |
|---|---|----|----|---|---|---|----|
| 4 | 8 | 10 | 12 | 2 | 5 | 7 | 11 |
|---|---|----|----|---|---|---|----|

اب دو حصوں میں سورٹ کرنے کے بعد ہم ان دونوں حصوں کو Merge کر دیں تاکہ ہمارا اررے سورٹڈ آرڈر میں ہمارے پاس جائے۔

|   |   |   |   |   |    |    |    |
|---|---|---|---|---|----|----|----|
| 2 | 4 | 5 | 7 | 8 | 10 | 11 | 12 |
|---|---|---|---|---|----|----|----|

اب ہمارا اررے ہمارے پاس سورٹڈ آرڈر میں آگیا ہے۔

## Lecture No. 45

پہلا : Merge سورٹ

دوسرا : کوئیک سورٹ

تیسرا : ہیپ سورٹ

یہ تینوں Algorithm کام کرنے کیلئے Proportional ٹائم لیتے ہیں جو کہ  $n \log_2 n$

کہلاتا ہے۔ ایلیمنٹری کے تین سورٹنگ Algorithm سورٹنگ کیلئے جو ٹائم لیتے تھے اس کو  $n^2$  کہا جاتا ہے۔ مطلب جتنے نمبر ہوں گے انکی پاور 2 کر کے ٹائم لگتا تھا۔ اس لئے یہ نئے Algorithm ایلیمنٹری Algorithm سے زیادہ فاسٹ ہیں اور  $n \log_2 n$  ٹائم لیتے ہیں۔ سرچنگ ٹائم میں ہم ٹائم کو کم کرنا چاہتے ہیں اس لئے N سے ٹائم کو کم کر کے  $\log_2 n$  کرنے کی کوشش کی گئی ہے۔

اب ان تینوں Algorithm کوئیک سورٹنگ، ہیپ سورٹنگ اور Merge سورٹنگ کو تھوڑا ڈیٹیل سے سمجھنے کی کوشش کرتے ہیں۔ ہم نے اپنی ڈسکشن کو تقسیم اور فاتح یعنی Divide اور Conquer سے شروع کی تھی، ہم نے ارے کو پہلے دو پارٹس میں تقسیم کیا پھر الگ الگ پارٹ کو سورٹ کیا پھر دونوں پارٹ کو Merge کر کے ارے کو فائنل شکل میں سورٹ کر لیا تھا۔ جس کی آؤٹ پٹ ہمارے پاس کچھ یوں آئی تھی۔

Split the list into two parts

|    |    |   |   |   |    |   |   |
|----|----|---|---|---|----|---|---|
| 10 | 12 | 8 | 4 | 2 | 11 | 7 | 5 |
|----|----|---|---|---|----|---|---|

Sort the two parts

|   |   |    |    |   |   |   |    |
|---|---|----|----|---|---|---|----|
| 4 | 8 | 10 | 12 | 2 | 5 | 7 | 11 |
|---|---|----|----|---|---|---|----|

Merge the two parts together

|   |   |    |    |   |   |   |    |
|---|---|----|----|---|---|---|----|
| 4 | 8 | 10 | 12 | 2 | 5 | 7 | 11 |
|---|---|----|----|---|---|---|----|

اب یہ دیکھتے ہیں یہ تقسیم اینڈ فاتح کے Algorithm کیسے مفید ہیں۔

Halves کو سورٹ کرنے کیلئے جو ٹائم لگتا ہے وہ ہے  $(n/2)^2 + (n/2)^2$   
 پھر دو Halves کو Merge کرنے کیلئے جو ٹائم لگتا ہے وہ N ہے  
 اس طرح اگر N یعنی نمبر ہمارے 100 ہوئے تو Divide اور Conquer کے  
 Algorithm جو ٹائم لے گیں اس کو اب ہم اس فارمولا کے ساتھ Calculate  
 کرے گا ہیں۔

$$\begin{aligned}
 &= (100/2)^2 + (100/2)^2 + 100 \\
 &= 2500 + 2500 + 100 \\
 &= 5100
 \end{aligned}$$

تو ہمارے پاس جو ٹائم آئے گا وہ 5100 ہوگا۔ اب اگر اسی کو ہم ایلیمینٹری سورٹنگ میں  
 دیکھیں تو این کی پاور 2 لیتا ہے۔

اب فرض کریں کہ ہم ایلیمینٹری Algorithm کو استعمال کر کے انسیرشن کر رہے ہیں۔

اب ہم Halves کو دو پارٹ میں تقسیم کرے گے اور جو ٹائم اب لگے گا وہ  $(n/2)^2 + (n/2)^2$ ۔

ہو جائے گا۔ اب مریجننگ کیلئے جو ٹائم درکار ہوگا وہ این یعنی نمبر 100 ہوگا۔ اب

فرض کریں این ہمارے پاس 100 ہے۔ اب جب 100 کی پاور دو ہوگی تو یہ

10000 ہو جائے گا اب اگر اس پر ہم تقسیم اور فاتح یعنی Divide اور Conquer کا

Algorithm کو Apply کریں تو پہلے ہاف کا ٹائم  $(100/2)^2$  ہو جائے گی۔ اس

طرح دوسرے ہاف کا ٹائم بھی ایسا ہی ہوگا۔ اور Merging کا ٹائم این یعنی 100 ہوگا۔

تو ہمارے پاس جو جواب آئے گا وہ 5100 آئے گا۔ اب تقسیم اور فاتح کے algorithm کا ٹائم یقیناً ہمارے پاس ایلیمنٹری ٹائم جو 10000 تھا سے کم ہے۔

## Mergesort

یہ بھی تقسیم اور فاتح کے Algorithm کی طرح ہی کام کرتا ہے۔

یہ لسٹ کو ہاف میں سپلٹ کر دیتا ہے۔

دو Halves کو MergeSorts کرتا ہے۔

اور پھر دو سورٹڈ Halves کو Merge کر دیتا ہے۔

Mergesort کو Recursively بھی امپلمنٹ کیا جاسکتا ہے۔

Merge کا Algorithm تین سٹیپس کو Involves کرتا ہے....

اگر ایٹم کے نمبر کو 0 یا 1 میں سورٹ کرنا ہے تو ریٹرن

Recursively پہلا اور دوسرا Halves الگ سے سورٹ کرتا ہے

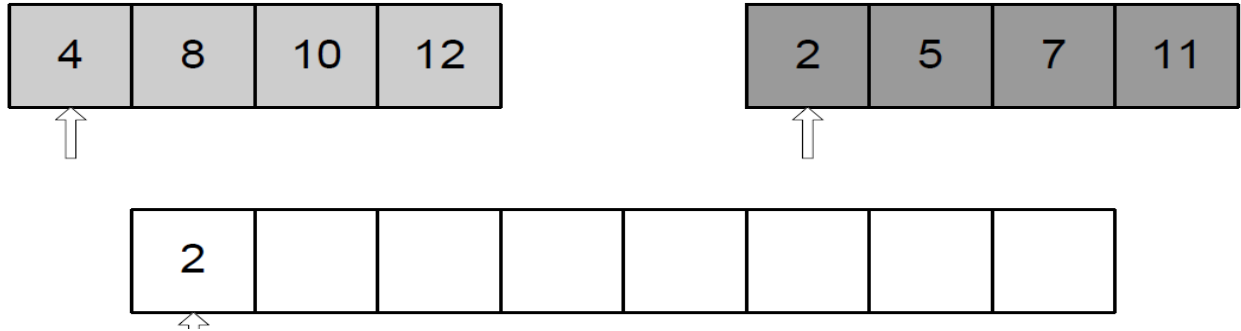
دو سورٹڈ Halves کو Merge ایک سورٹڈ گروپ میں اگر ڈیٹا کونسلٹ کرتا ہے۔

اگر ڈیٹا 0 یا 1 ایلیمنٹ پر Consist کرتا ہو تو پھر مزید کچھ بھی کرنے کی ضرورت نہیں

ہے۔ اگر نمبر یعنی ڈیٹا ایلیمنٹس 1 سے بڑا ہو تو پھر Divide اور conquer اسٹریٹجی کو

Apply کر کے ڈیٹا سورٹ کرنا ہوگا۔

MergeSort لسٹ کو دو ہاف حصوں میں تقسیم کر کہ الگ سے سورٹ کرتا ہے اور Recursion کو استعمال کر کہ ڈیٹا کو سورٹ کرتا ہے۔



ہمارے پاس دو سورٹڈ ارے ہیں اور ایک ارے خالی ہے جس کا سائز ان دو سورٹڈ ارے کے برابر ہے۔ یہ دونوں ارے internally سورٹڈ ہیں۔ اور تیسرے ارے میں 8 ایلیمنٹس ہوں گے۔ اب اس خالی ارے کے انڈکس 0 میں سے دو ارے میں سے جو ایلیمنٹ سب سے چھوٹا ہو گا وہ رکھا دیا جائے گا جیسے کہ ہمارے پاس اس میں ایلیمنٹ 2 دو ہے وہ ہمیں ارے کے انڈکس 0 پر نظر آ رہا ہے۔ اب دوسرا چھوٹا نمبر چیک ہو گا اور پھر ارے کے انڈکس 1 میں انسرت ہو جائے گا تو ہمارے پاس دوسرا چھوٹا ایلیمنٹ نمبر 4 ہے جو کہ انڈکس 1 پر سٹور ہو گا۔ اور اسی طرح ہم باقی کے نمبر کو ارے انڈکس میں انسرت کر جائے گے۔ اس کو ایک Simple لوپ کے ساتھ Manipulates کیا جائے گا جو کہ تین ارے کو Manipulate کرے گا۔

ایک بات یاد رکھیں کہ ہم Merging تب کرتے ہیں جب ہمارے پاس پہلے سے ہی دو سورٹڈ پارٹ موجود ہو۔

اب ہم ایک اررے کو دیکھتے ہیں جس کو ہم دو ہاف میں سپلٹ کرے گے اور Recursion کا استعمال کر کہ اس کو سورٹ بھی کرے گے۔

|    |   |   |    |    |   |   |   |
|----|---|---|----|----|---|---|---|
| 10 | 4 | 8 | 12 | 11 | 2 | 7 | 5 |
|----|---|---|----|----|---|---|---|

جب ہم MergeSort کے Algorithm کو Recursively کال کرتے ہیں تو صرف وہ مخصوص ہاف ہی اررے میں ارگنٹ پاس کرتے ہیں۔ اب ہم اپنے اس اررے کو دو ہاف میں تقسیم کرے گے۔

|    |   |   |    |    |   |   |   |
|----|---|---|----|----|---|---|---|
| 10 | 4 | 8 | 12 | 11 | 2 | 7 | 5 |
|----|---|---|----|----|---|---|---|

اس طرح ہم نے اپنے اررے کو دو حصوں میں تقسیم کر دیا ہے اب ہم MergeSort کے Algorithm کے مطابق اس ہاف کو مزید دو حصوں میں تقسیم کرے گے۔

|    |   |    |   |
|----|---|----|---|
| 10 | 4 | 10 | 4 |
|----|---|----|---|

اب ہم نے MergeSort کے مطابق اس کو مزید ہاف میں تقسیم نہیں کر سکتے ہیں اس لئے MergeSort Recursive کا Algorithm یہی ختم ہو جائے گا۔

اب اس کو سورٹ کرنا ہوگا۔

|   |    |   |    |
|---|----|---|----|
| 4 | 10 | 8 | 12 |
|---|----|---|----|

ایلیمنٹ 4 سب سے چھوٹا تھا جو کے پہلے آیا پھر 10 آیا، اب ان دونوں ہاف کو سورٹ کر کے پھر Merge کرے گے۔ ہمارے پاس اب 8 اور 12 کے ایلیمنٹ بھی ہیں جس کو بالکل اسی طرح سلیپٹ کریں گے جیسے کہ ایلیمنٹ 4 اور 10 کو کیا تھا۔ اور پھر ان دونوں ہاف کو سورٹ کر کے MergeSort کا Algorithm کو Apply کرے گے۔ یاد رکھیں کہ MergeSort کا Algorithm تب کام کرتا ہے جب ایلیمنٹس الگ الگ اور سورٹڈ کنڈیشن میں ہو۔

|   |   |    |    |
|---|---|----|----|
| 4 | 8 | 10 | 12 |
|---|---|----|----|

اب اسی طرح ارے کے دوسرے ہاف کو MergeSort کے Algorithm سے سپلٹ کرے گے اور پھر الگ الگ سورٹ کر کے Merge کرے گے۔ پہلے ہاف میں نے سورٹ کر کے دے دیا ہے دوسرا ہاف آپ لوگ پریکٹس کے طور پر خود سورٹ کر کے چیک کریں۔ یہ سورٹنگ کچھ اس طریقے سے ہوتی ہے

|    |   |   |    |    |   |   |   |
|----|---|---|----|----|---|---|---|
| 10 | 4 | 8 | 12 | 11 | 2 | 7 | 5 |
|----|---|---|----|----|---|---|---|

|   |   |    |    |
|---|---|----|----|
| 4 | 8 | 10 | 12 |
|---|---|----|----|

|    |   |   |   |
|----|---|---|---|
| 11 | 2 | 7 | 5 |
|----|---|---|---|

|    |   |
|----|---|
| 11 | 2 |
|----|---|

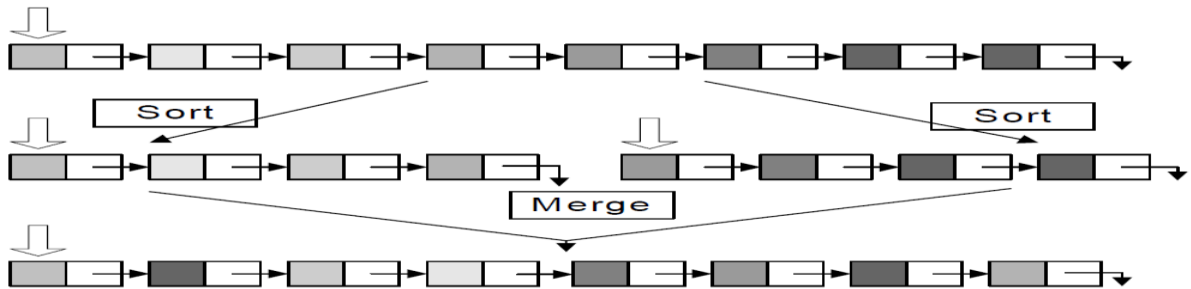
|    |   |
|----|---|
| 11 | 2 |
|----|---|

|   |    |
|---|----|
| 2 | 11 |
|---|----|

اس طرح دوسرے ہاف کو الگ سے سپلٹ کر کے سورت کرنا اور پھر دونوں ہاف کو Merge کر کے سورت کرنا ہوگا۔

## Mergesort and Linked Lists

Mergesorts اررے کے ساتھ ساتھ لنکڈ لسٹس کے ساتھ بھی کم کرتا ہے۔ اب ہم یہ دیکھیں گے کہ لنکڈ لسٹ کیسے سورت کرتے ہیں۔ فرض کریں ہمارے پاس سنگلی لنکڈ لسٹ ہے، ہم پہلے لنکڈ لسٹ کو دو ہاف میں تقسیم کرے گے، پھر دونوں ہاف کو Recursively سورت کرے گے۔



اس لسٹ میں ہمیں 8 ایلیمنٹس دیئے گئے ہیں جو کے نمبر ہو سکتے ہیں، اب پہلے اس لنکڈ لسٹ کو دو ہاف میں تقسیم کرے گے پھر اس کو سورٹ کرے گے اور پھر سورٹ کرنے کے بعد دونوں کو Merge کر کے سورٹڈ لسٹ میں جواب آ جائے گا۔ ہم مرجنگ کرتے وقت ایک ایڈیشنل عرضی اررے کا استعمال کرتے ہیں، مرجنگ کرتے وقت ہم ایلیمنٹ کو اس ایڈیشنل اررے میں کاپی کرتے ہیں اور جب مرجنگ آپریشن کے بعد ایلیمنٹس کو عرضی اررے سے اصل اررے میں کاپی کر دیتے ہیں۔

مرجنگ سورٹنگ Algorithm کو ایک عرضی اررے کی ضرورت پڑتی ہے اور اس اررے کا سائز اتنا ہی ہوگا جتنا سائز کا اررے سورٹ کرنا ہوتا ہے۔ مطلب جتنے ایلیمنٹس ہمیں دیئے گئے ہوتے ہیں سورٹ کرنے کیلئے۔

- Mergesort is  $O(n \log_2 n)$
- Space?
- The other sorts we have looked at (insertion, selection) are **in-place** (only require a constant amount of extra space)
- Mergesort requires  $O(n)$  extra space for merging

کوئیک سورٹ Algorithm نہ صرف  $n \log_2 n$  کا Algorithm بلکہ اس کے ساتھ ساتھ inplace Algorithm بھی ہے۔

Inplace کے Algorithm میں جو سپیس درکار ہوتی ہے وہ کانسٹنٹ ہوتی ہے۔ کوئیک سورٹ Algorithm میں ہمیں ایڈیشنل عرضی اررے کی ضرورت نہیں پڑتی

# Quicksort

کوئیک سورٹ ایک اور تقسیم اور فاتح کا Algorithm ہے۔ کوئیک سورٹ پارٹیشن کے آئیڈیا پر بیس کرتا ہے۔

|   |    |    |   |   |   |    |   |   |
|---|----|----|---|---|---|----|---|---|
| 4 | 12 | 10 | 8 | 5 | 2 | 11 | 7 | 3 |
|---|----|----|---|---|---|----|---|---|

فرض کرے ہمارے پاس یہ اررے ہے اور اب ہم اس پر کوئیک سورٹ کا Algorithm اپلائی کر کہ چیک کرتے ہیں۔ کوئیک سورٹ میں ایک Pivot ویلیو ہوتی جو کہ اررے کا وہ ایلیمنٹ جو سینٹر میں ہوتا ہے۔ اس کیس میں Pivot کی ویلیو 5 بنے گی۔

5

pivot value

مڈل ویلیو سلیکٹ کرنے کے بعد اب ہم اس ویلیو کو اررے کے آخری ایلیمنٹ یعنی 3 کے ساتھ Swap کرے گے۔

|   |    |    |   |   |   |    |   |   |
|---|----|----|---|---|---|----|---|---|
| 4 | 12 | 10 | 8 | 3 | 2 | 11 | 7 | 5 |
|---|----|----|---|---|---|----|---|---|

اب ہم اس اررے میں 2 انڈکس Low اور ہائی کا استعمال کرے گے۔ Low انڈکس اررے کے پوزیشن 0 سے سٹارٹ ہوگا اور  $n-1^{th}$  position تک جائے گا۔



↑  
low

↑  
high

5

pivot value

اب لوپ میں جو ویلیو Pivot ویلیو سے بڑی ہوگی وہ سرچ ہو ہوگی اور لو انڈ کس مزید انکریمنٹ ہو جائے گا۔ جیسے کے نیچے اررے میں دیکھا جاسکتا ہے۔



↑  
low

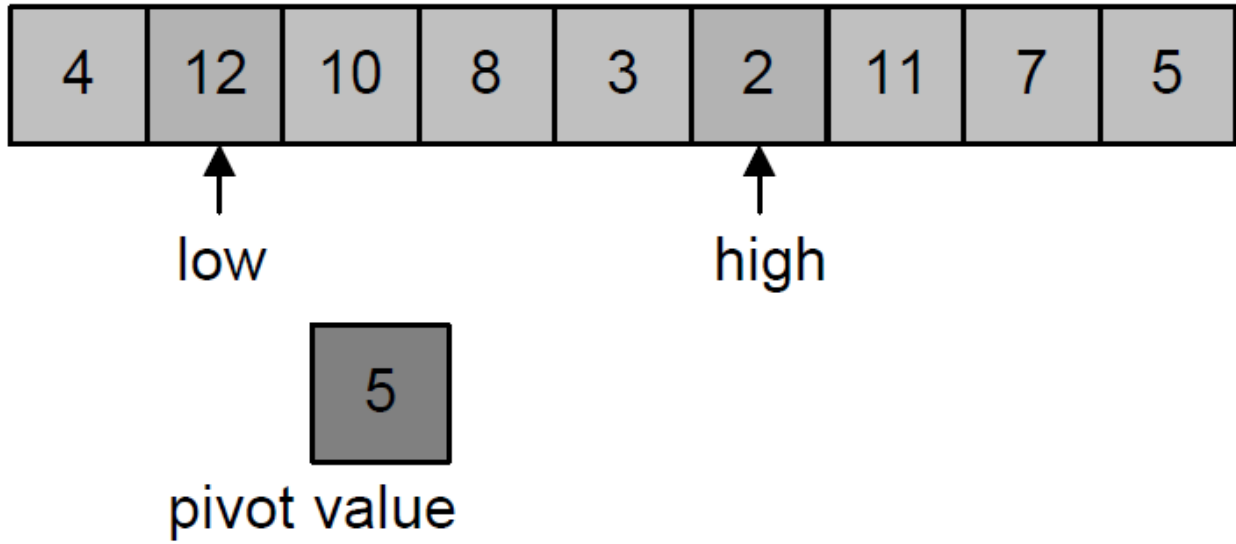
↑  
high

5

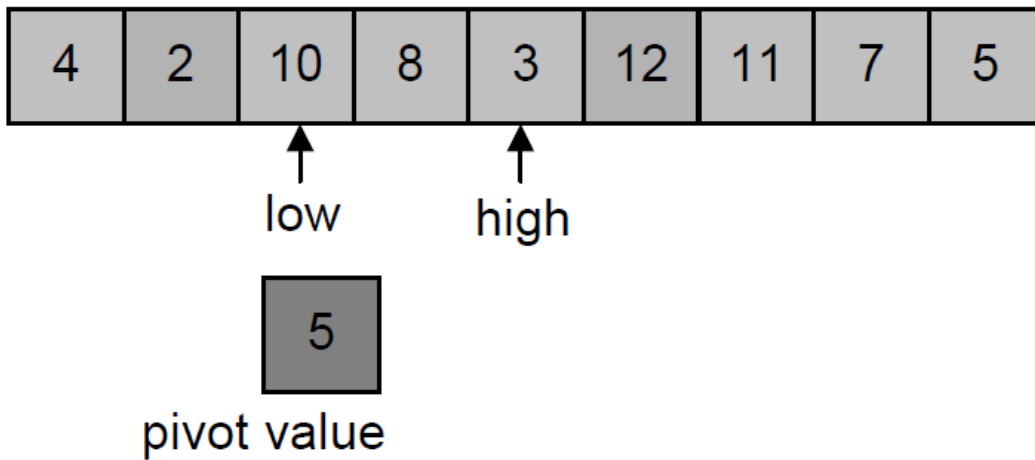
pivot value

اس اررے میں دیکھا جاسکتا ہے کہ اب لو انڈ کس 12 ہے، ایلیمنٹ 4 کیوں کے Pivot ویلیو یعنی 5 سے کم ہے اس لئے لو ویلیو انکریمنٹ ہو گئی ہے۔ اب 12 پر لو انڈ کس Stop ہو جائے گا کیونکہ ایلیمنٹ 12 کی ویلیو Pivot کی ویلیو سے بڑی ہے۔

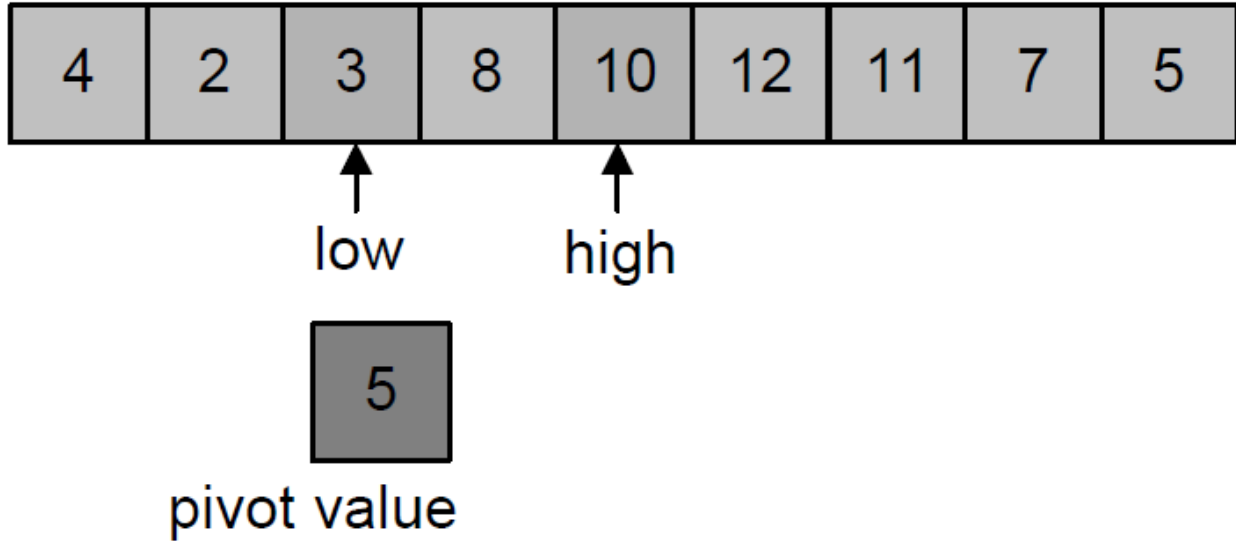
اب ہائی پوزیشن رائٹ سائیڈ سے لفٹ لفٹ یعنی انڈکس 0 کی طرف Move ہوگا جو کہ  $n-1^{th}$  پوزیشن تک جائے گا۔ اب رائٹ سے لفٹ سائیڈ کے اس Move میں جب ہم ایسے ایلیمینٹ پر پہنچ جائے گے جو کہ 5 سے چھوٹا ہوگا تو انڈکس Stop ہو جائے گا اور ہمارے اس اررے کے کیس میں انڈکس 2 پر Stop ہوگا۔



دونوں انڈکس لو اور ہائی اب Stop ہو چکے ہیں، اب نیکیٹ سٹیپ میں ہم ان دونوں ایلیمینٹس یعنی لو اور ہائی کو ایک دوسرے کے ساتھ Swap کرے گے۔

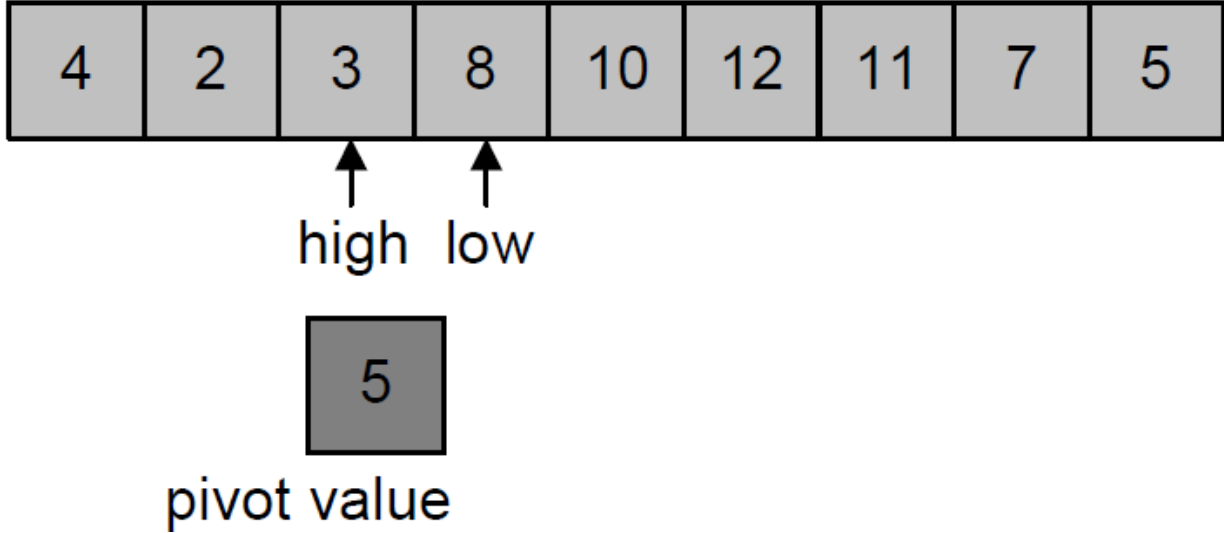


ہماری Pivot ویلیو ابھی بھی 5 ہی ہے۔ اب ہم پھر سے لو انڈ کس کو رائٹ سائیڈ پر Move کرے گے اور کوشش کرے گے کہ ایسا ایلیمینٹ مل جائے جو ہماری Pivot کی ویلیو 5 سے بڑا ہو۔ اس اررے میں جو اگلا نمبر یا ایلیمینٹ 10 ہے وہ ہماری Pivot کی ویلیو سے بڑا ہے اس لئے لو انڈ کس یہاں stop ہو جائے، اب ہائی انڈ کس رائٹ سے لفٹ کو move کرے گا اور ایسا انڈ کس تلاش کرے گا جو ہماری Pivot کی ویلیو سے کم ہو، ہمارے اس اررے میں کم ویلیو کا ایلیمینٹ 3 ہے اور اب ہائی انڈ کس یہاں روک جائے گا۔ اب دونوں انڈ کس روک جانے کے بعد ہم ایلیمینٹس 3 اور 10 کو ایک دوسرے کے ساتھ Swap کرے گے۔

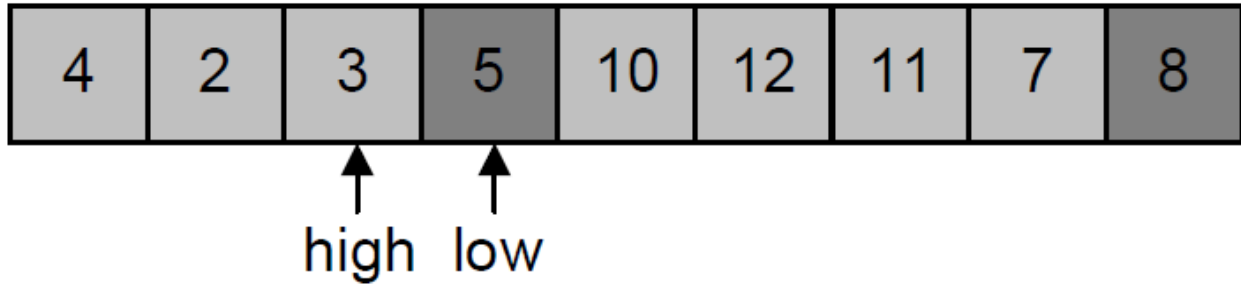


اب نیکسٹ Move یا Iteration میں دونوں لو اور ہائی انڈ کس ایک دوسرے کو کرس کرے گے۔ جب ہائی پوائنٹر لو پوائنٹر کو کرس کرے گا تو ہم اس کو مزید Move نہیں

کرے گے۔ اب ہمارے پاس ہائی ویلیو 8 اور لو ویلیو 3 ہو جائے گی۔ اور ہم پھر ان دونوں ایلیمنٹس کو ایک دوسرے کے ساتھ Swap کر دے گے۔



اب کر سنگ پوزیشن ایلیمنٹ 8 پر ہو رہی ہے، اب پھر سے ہم Pivot ویلیو کو کر سنگ پوزیشن کی ویلیو 8 کے ساتھ Swap کر دے گے۔



یہ ارے ابھی سورت نہیں ہوا ہے مگر ایلیمنٹ 5 نے اپنا Destination ڈھونڈ لیا ہے۔ ایلیمنٹ 5 کے لفٹ سائیڈ پر جو نمبر ہوں گے وہ Pivot ویلیو یعنی ایلیمنٹ 5 سے چھوٹے ہوں گے اور جو رائٹ سائیڈ پر ہوں گے وہ بڑے ہوں گے۔

|   |   |   |   |    |    |    |   |   |
|---|---|---|---|----|----|----|---|---|
| 4 | 2 | 3 | 5 | 10 | 12 | 11 | 7 | 8 |
|---|---|---|---|----|----|----|---|---|

Quicksort the left part

Quicksort the right part

اب یہاں پر یہ بھی دیکھا جاسکتا ہے کہ جو چھوٹے ایلیمنٹس لفٹ سائیڈ پر ہیں وہ بھی internally سورت نہیں ہیں اور نہ ہی رائٹ سائیڈ پر جو بڑے ایلیمنٹس ہیں وہ internally سورت ہیں۔ اب اگلے مرحلے میں ہم Recursively کوئیک سورت سے لفٹ اور رائٹ پارٹس کو سورت کرے گے۔

اررے اور اس کا سائز کوئیک سورت فنکشن میں ارگمنٹس کے طور پر پاس ہوتے ہیں۔ فنکشن لوکل variable کو ڈیکلاریڈ کرے گا اور اررے کا سائز اگلے سٹیمنٹ میں چیک ہوگا۔ اگر اررے کا سائز ایک یا ایک سے زیادہ ہو تو فنکشن Recursive Mechanisam کو کال کرتا ہے اررے کو سورت کرنے کے لئے۔ یہ اررے کو دو پارٹس میں divide کرتا ہے Pivot ایلیمنٹ کی ویلیو کو choose کرنے کیلئے۔

Subsequent کالز میں سب سے پہلے لفٹ سائیڈ سورت ہوتی ہے اور پھر رائٹ سائیڈ سورت ہوتی ہے۔ کوئیک سورت Algorithm بہت Elegant ہے اور یہ کسی بھی سائز کے اررے کو سورت کر سکتا ہے۔ کوئیک سورت Algorithm کو سورٹنگ کیلئے ایک اچھا Algorithm سمجھا جاتا ہے۔